Physics-based Flight Simulator

A2 Computer Science Project

Vian Garg - GEMS Founders School

Zipped Code Directory: https://bit.ly/VianGargFlightSimulator

Table of Contents

Analysis

Problem Definition

The main purpose of this project is to bridge the gap between high-end flight simulators and simpler web-based flight games by creating a physics-accurate flight simulation program that is accessible to users on standard hardware.

Current flight simulators often fall into two distinct categories:

High-End Simulators: Programs like Microsoft Flight Simulator and X-Plane 11 offer highly realistic physics and environmental simulations but demand substantial computing resources, making them inaccessible to many users, especially students and educational institutions.

Web-Based Games: These simpler simulations, usually available online, prioritise entertainment and ease of use over realism. As a result, they often lack essential physical mechanics, reducing their effectiveness as educational tools.

This project aims to fill this gap by developing an accessible, physics-focused flight simulator that:

Prioritizes Educational Value: By simulating accurate flight mechanics (e.g., angle of attack, stall conditions), users gain hands-on experience in flight physics, a feature critical for students or enthusiasts interested in the science of flight.

Improves Accessibility: It can be run on regular hardware, such as school computers or personal laptops, allowing greater accessibility for students, teachers, and aviation enthusiasts. It is also accessible to those with visual impairments, with a UI that employs a high-contrast color scheme to ensure readability for users with colorblindness. As a partially colorblind individual, I prioritized this design to meet my own needs and those of similar users.

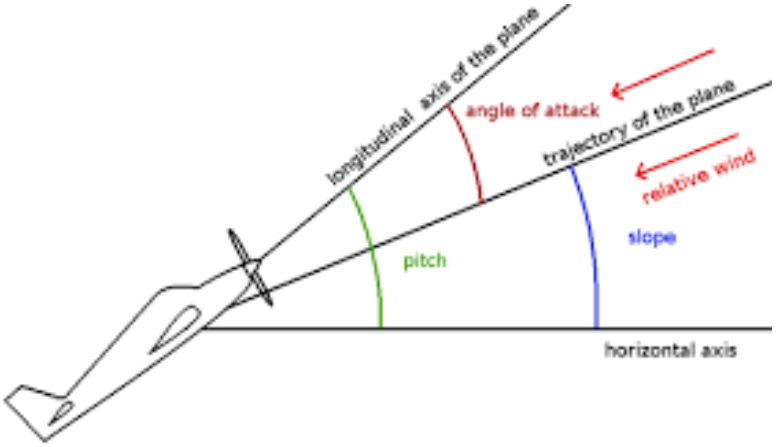
Balances Realism and Usability: While the program will incorporate physics to simulate realistic flight, it will maintain a user-friendly interface and moderate performance requirements to avoid overwhelming new users.

By building a program that can function both as an educational tool and as an accessible simulation for casual flight enthusiasts, this project hopes to contribute to the learning experience for users interested in the physics of flight.

Key-Term Definitions

Physics Engine: This component of the code is responsible for managing the simulation's physics, including the calculation of positions and velocities.

Angle of Attack (AoA): The angle between the plane and its instantaneous velocity, both in the forward direction.



Post-Flight Data: Information collected during the simulation, such as maximum G-load and AoA variation, which can be used for analysis and learning.

Stall: A sudden loss of lift when the airflow over the aircraft's wings separates from the surface. Typically occurs at low velocities and high AoAs.

Subsonic: Flight under the speed of Mach 0.8 or 273 metres per second.

Research

Interview - Ashley Colaco

My name is Ashley Colaco, I'm a 17 years old student born and brought up in Dubai, UAE. Some of my favourite subjects include maths, physics, chemistry, and engineering.

I've heard you've had an interest in aviation, Ashley?

Yes, ever since I was young, I've been interested in this subject. I have a huge plane collection of about 130 planes. I even have started a YouTube channel for this, where I post videos about my flight travels, model plane and unboxings.

What do you think of flight simulations?

Flight simulations are really fun and it helps pilots to train when they are flying in real life scenarios, including emergency situations. I've had limited exposure to flight simulators because they are inaccessible on my laptop. I have only tried them when I visited KidZania. And recently I went for an internship in Vision Concept Aviation Technology Institute, where on the last day of the internship, they had set up a flight simulator on the desktop. I tried different kinds of aircrafts like fighter jets, commercial aircrafts.

So you're saying that right now you're interested in flight simulators, but if you want to use one, you have to go outside somewhere like KidZania or to that internship of yours?

Yes, so that's the thing, it's inaccessible. But I really love to use flight simulators.

Would you be interested if someone made a product like a flight simulator that was physics accurate and was easier to run on regular hardware like the average laptop?

Yes, I am interested and it will be much more accessible. And in fact, I can do it anywhere I'd like in case I get bored, I could just carry my laptop, it's portable anyway. And so anytime I'm feeling bored, I can just try it on in my free time. It would help me to learn more about flying an aeroplane, especially because it's also physics based.

So you think it could also be used as an educational tool?

It can be. It depends also if you showcase the quantities of, for example, the knots, the aircraft speed, the engine speed. So there should be everything that an aircraft could possibly have. In fact, even the controlling of the ailerons, the elevators, and even the rudders. So all of these should be incorporated into a flight simulator.

And you're saying that post-flight data is important for educational purposes?

Post-flight data is important as well.

Alright. Thank you, Ashley.

Interview - Mr. Ebran Nasir

I'm Mr. Ebran, a physics teacher in Dubai with a past as a pilot. I've always loved aviation and now enjoy teaching students how physics powers flight.

How did you get into aviation?

It was my childhood dream, watching planes at the airport. I learned to fly small aircraft years ago before switching to teaching physics. It's a way to keep that passion alive.

What do you think of flight simulations?

They're brilliant for learning. Simulators let you test physics in action, like how wings lift or why planes stall, without risks. I've used them in pilot training, but in school, they're either too basic, like games, or too heavy for our old computers. Students get excited, but we're stuck.

Are flight simulators accessible for your students?

Not at all. Our lab has outdated PCs, barely enough for simple apps. Good simulators need strong hardware, so we rarely use them. Students might try one at an event or aviation workshop, but that's not regular enough to learn properly.

Would you be interested in a flight simulator that's accurate but runs on average school laptops?

Yes, that'd be perfect. If it models physics well, say, drag or pitch, and works on our basic machines, I could use it in class. Students could play with concepts like velocity or lift anywhere, even at home. It'd make lessons fun and practical.

Could it work as an educational tool?

For sure. Physics can feel dry, but a simulator showing real-time data, speed, altitude, control angles, would bring it alive. Students could experiment, like adjusting flaps to see lift change. It'd help them grasp tough ideas better than diagrams.

Is post-flight data important for education?

Very. If they can see graphs after flying, like speed or height over time, they'll understand what went wrong or right. It's like a lab report for flight, tying back to Newton's laws or energy. They'd learn by analyzing their own data.

Thank you, Mr. Ebran.

Interview Observations

- Accessibility Concerns; simulation technologies such, as Microsoft Flight Simulator demand costly hardware setups that may hinder access, for numerous interested users.
- Many current simulators prioritise hyper realism and intricate details; however simple flight mechanics are more suitable, for beginners.
- A flight simulator based on physics could be a resource, for learning purposes and would be even more beneficial if it offers features such as providing data after the flight.
- We need a simulator that's user friendly and works well on laptops while still maintaining a balance between realism and ease of use due to the growing interest in it.

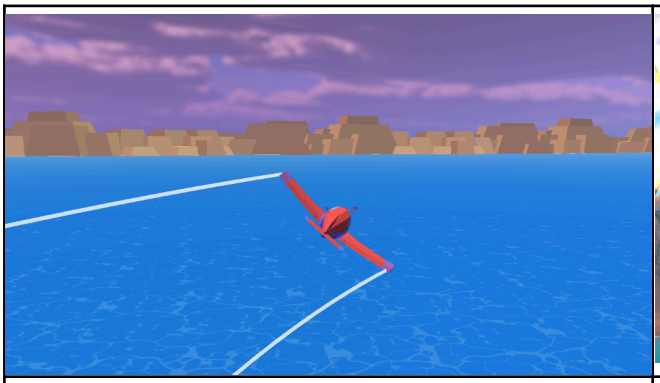
Additional stakeholders

IT staff, who require low-resource software for school computers, parents, who value educational outcomes, and schools, who fund software integration.

- Discussions with a school IT administrator confirmed the need for compatibility with 4 GB RAM systems.
- Parents emphasized the simulator’s role in engaging students, while schools prioritized cost-effective tools. These insights shaped the project’s focus on accessibility and efficiency.

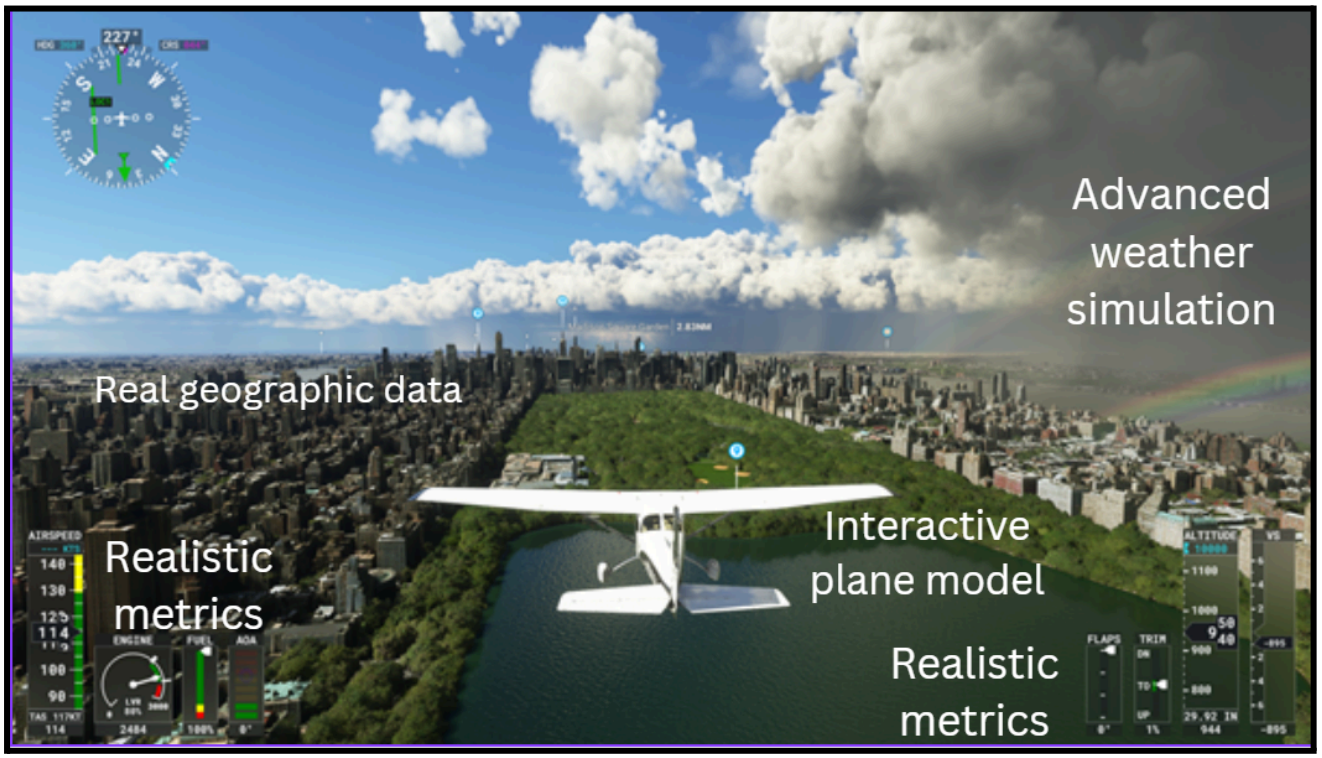
Current Systems - Overview

	
Microsoft Flight Simulator 2020	XPlane 11
Pros: Extremely Graphically and Physically Detailed, providing the optimal experience.	Cons: Costly, extremely processing power intensive & can't be run on most hardware

	
Pilot Royale	Trailblazers
Pros: Free, can run on most devices with a modern browser, & easy to control.	Cons: Requires internet connection & isn't physics based. No data; limited educational potential.

Current Systems - Deep Dive

Microsoft Flight Simulator 2024



Microsoft Flight Simulator (MSFS) 2024 is a premier high-end flight simulation platform that sets a benchmark in realism and immersion. Its advanced aerodynamic modeling, detailed environmental systems, and photorealistic graphics cater to professional pilots, aviation enthusiasts, and simulation hobbyists alike. However, its demanding hardware requirements and complex interface highlight the accessibility and usability gaps for broader audiences, especially students or educational institutions.

Feature	**MSFS 2024**	**Proposed Flight Simulator**
Physics Accuracy	Highly realistic	Realistic, focused on fundamentals
Hardware Requirements	High-end PCs or consoles: 150 GB i7 - 10700K RTX 2080 32 GB 50 Mbps	Standard school computers or personal laptops
User Interface	Advanced, professional	Simplified, education-friendly
Target Audience	Professionals, aviation enthusiasts	Students, educators, casual enthusiasts

XPlane 11

Download size	Recommended CPU	Rec. GPU	Rec. RAM	Rec. Bandwidth
150 GB	i7 - 10700K	RTX 2080	32 GB	50 Mbps

X-Plane 11 is a leading flight simulation platform renowned for its exceptional realism and immersion. With sophisticated aerodynamic modeling, intricate environmental systems, and highly detailed visuals, it appeals to professional pilots, aviation enthusiasts, and dedicated simmers. However, its steep hardware demands and intricate interface may pose challenges for wider accessibility, particularly for students or educational settings.

Feature	MSFS 2024	Proposed Flight Simulator
Physics Accuracy	Highly realistic	Realistic, focused on fundamentals
Hardware Requirements	High-end PCs or consoles	Standard school computers or personal laptops
User Interface	Advanced, professional	Simplified, education-friendly
Target Audience	Professionals, aviation enthusiasts	Students, educators, casual enthusiasts

Current Systems - IPSO (Input, Process, Storage, Output) Analysis

High End Simulator (Microsoft Flight Simulator 2024 and XPlane 11):

Inputs:

- User controls (joystick, yoke, rudder pedals, throttle) ~ precise analog input
- Keyboard commands, mouse interactions (cockpit controls, menus)
- Weather data (live/simulated), time of day ~ METAR data for real-time wind speed
- Aircraft and airport/location selection, configuration

Process:

- Physics, flight model, aerodynamic calculations ~ Blade element theory for lift force
- Weather and terrain processing ~ Volumetric clouds via ray marching
- Aircraft systems simulation, collision detection
- Graphics rendering, sound processing (spatial audio) ~ DirectX 12, HRTF audio

Storage:

- Aircraft data, world/terrain maps, airport/navigation database
- Weather patterns, user settings, flight plans
- Game state, flight logs, user profiles ~ SQLite database for pilot stats
- Graphics textures, 3D models, sound assets ~ 4K albedo textures, engine sounds

Output:

- Visuals (aircraft, environment, instruments) ~ 4K terrain with dynamic shadows
- Cockpit displays, instrument readings ~ PFD showing real-time airspeed
- Sound effects, engine noise ~ 5.1 surround jet turbine rumble

Web-Based Games (Pilot Royale and Trailblazers):

Inputs:

- Keyboard for controls, mouse for aiming ~ WASD for aircraft, mouse for shooting
- Menu navigation via mouse clicks ~ Left-click for settings menu

Process:

- Input-driven movement & card drafting ~ Vector-based velocity for aircraft
- Collision detection scoring ~ 2D circle collision for scoring
- WebGL & 2D graphics rendering ~ WebGL 2.0 for 3D trails

Storage:

- 3D models, 2D textures ~ glTF aircraft models, PNG sprites
- Audio files ~ shot.mp3, propellor.mp3, crash.mp3

Outputs:

- 3D aircraft, environments ~ 1080p
- HUD, score displays, instruments ~ 2D health bar
- Engine, explosion, ambient sounds ~ Web Audio API for explosion effects

Proposed Systems - IPSO (Input, Process, Storage, Output)

Inputs:

- Keyboard for flight controls (W/S: elevator, Q/E: aileron, A/D: rudder).
- Mouse for menu navigation and UI interaction.

Process:

- State-space model (4x4 matrices) for 6DOF dynamics, updated at 60 FPS.
- Matrix functions ~ addition, multiplication, determinants via Laplace expansion
- Numerical integration via Euler's Method.
- Ground collision check using aircraft Y-position vs. ground.
- 3D rendering with Ursina engine.

Storage:

- Aircraft physics matrices in text files, parsed into Matrix class.
- 3D models (.obj/.bam), textures (.png), SQLite for user data.

Outputs:

- 3D visuals of aircraft, terrain, skybox ~ 1080p, 60 FPS, adjustable window
- High-contrast UI with altitude, control surface, vertical velocity displays.

Language selection

Python 3 was selected for this project due to its simplicity and readability, making it ideal for debugging and beginner-friendly compared to C++. While C++ is faster for advanced simulations, it’s complex and demands more effort for details like memory management. Python offers built-in tools and libraries, such as SQLite, Ursina, and Matplotlib, that speed up and streamline development.

Python’s portability ensures compatibility across most systems, including low-end computers, broadening accessibility for schools and hobbyists. The extensive Python community provides robust resources and support, simplifying troubleshooting and program enhancement.

In summary, Python’s blend of simplicity, flexibility, and powerful features makes it the optimal choice for this educational, accessible flight simulation project.

Hardware selection

As the crux of this project is accessibility, I will choose to use solely keyboard and mouse inputs. While advanced users may desire joystick compatibility, it would take developmental efforts away from the core project without contributing to its completeness.

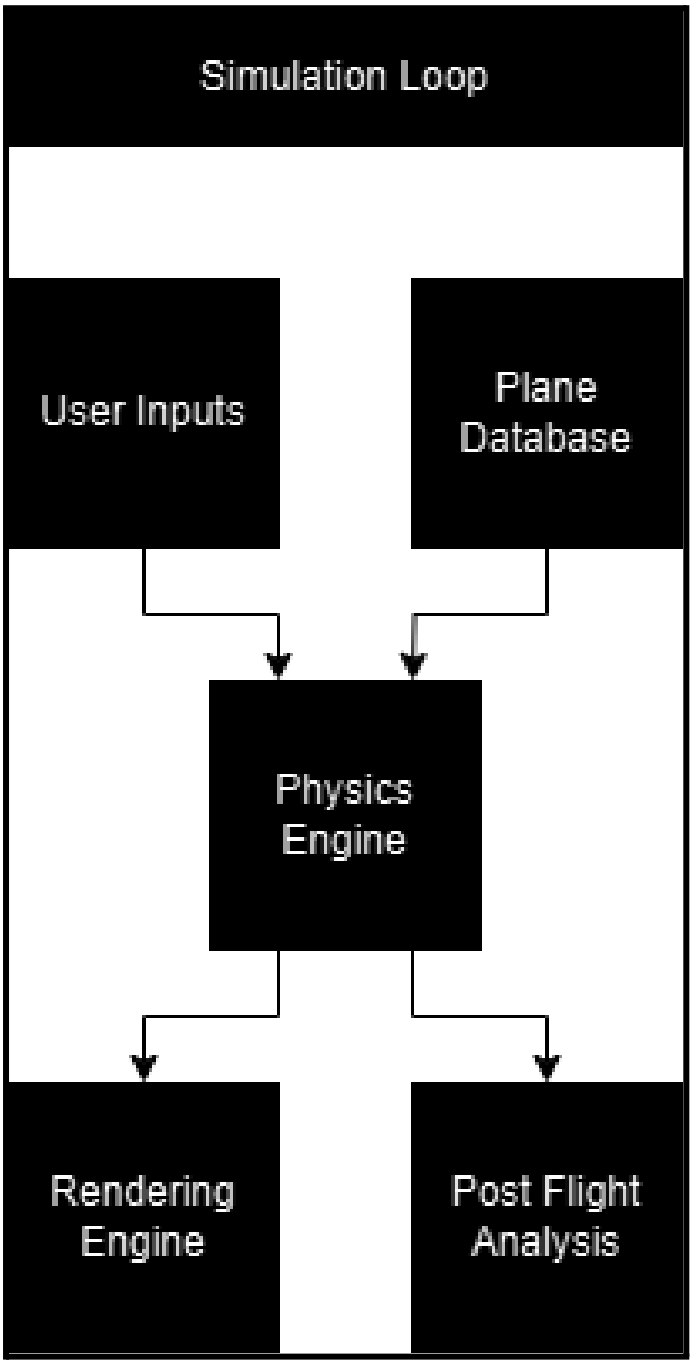
Download size	Recommended CPU	Rec. GPU	Rec. RAM
100 MB	i3 - 4130	Integrated	4 GB

The program will run efficiently on standard school computers or laptops with at least 4 GB RAM and a dual-core processor. This allows 12 year old hardware to access this resource and maximise usability without compromising physics.

Overview of proposed solution

The system delivers an interactive platform for mastering flight dynamics through a user-friendly simulator. It begins with a database of aeroplanes, each with detailed descriptions, allowing users to select an aircraft that aligns with their preferences or learning goals. The main simulation loop drives the experience with real-time physics for authentic flight behavior, paired with vibrant graphics for a visually captivating environment.

Complementing this is a post-flight screen with in-depth data from the session, fostering understanding of flight mechanics. This helps users evaluate performance and refine skills. The program demystifies flight dynamics, offering an engaging, accessible experience. By balancing realism with usability, it caters to enthusiasts and learners, sparking curiosity and confidence in aviation.



Program Components

Physics Engine

The engine will calculate the aircraft's longitudinal motion (pitch dynamics) by solving a set of linear equations that represent changes in key flight parameters such as velocity, pitch rate, and angle of attack. The state vector includes variables like forward speed (Δu), vertical speed (Δw), pitch rate (Δq), and pitch angle (Δθ). These variables change based on the aircraft's aerodynamic coefficients and control inputs.

- **State-Space Representation**: The system dynamics are modelled using matrices A and B, where the matrix A defines how the state variables evolve over time due to aerodynamic forces and stability derivatives. The matrix B represents the control inputs (such as elevator deflection) and their influence on the state variables.
- **Control Inputs**: The control vector (η) includes inputs such as elevator deflection (Δδ) which will affect the aircraft's trajectory and stability.
- **Time Evolution**: The system will solve for the derivatives of the state variables x* based on these matrices, updating the aircraft's motion over time, simulating real-time behaviour during flight.

$\begin{bmatrix} \dot{u} \\ \dot{w} \\ \dot{q} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} -0.04225 & -0.11421 & 0 & -32.2 \\ -0.20455 & -0.49774 & 317.48 & 0 \\ 0.00003 & -0.00790 & -0.39499 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} u \\ w \\ q \\ \theta \end{bmatrix} + \begin{bmatrix} 0.00381 \\ -24.4568 \\ -4.51576 \\ 0 \end{bmatrix} \eta$	Assumption: - Airplane is rigid - Airplane is symmetrical about its centerline plane (x-z plane is a plane of symmetry) - Airplane mass and moments of inertia are constant.
10. Longitudinal Dynamics Stability Part 2: Flight Dynamics and Control Lecture Flight Dynamics Principles | aerocastle	$x_{n+1} = x_n + \dot{x}_n \cdot \Delta t$

Plane Database

Plane selection menu with short description of each plane
The Database of planes must contain the following for each plane:

1. Flight Dynamics Matrices text file path, which will include:
    1.1. Longitudinal A
    1.2. Longitudinal B
    1.3. Lateral A (aka C)
    1.4. Lateral B (aka D)
2. Menu thumbnail image file path
3. Object file path
4. Texture image file path
5. Description text file path

Rendering Engine

The simulator will use Ursina 3D for rendering, loading OBJ files and BAM files from the plane database for aircraft models. Ursina's built-in optimizations will ensure smooth performance and dynamic animations based on user input. A third-person camera will track the aircraft, providing clear visuals.

User Management

Users should be able to login easily and intuitively.
Users should be able to create an account easily within a minute.
The user management system should be secure and robust, invulnerable to rookie haking attempts.

User Interface (UI)

Accessibility is a key consideration for the simulator’s UI to support diverse students, including those with visual impairments. The planned UI should employ a bright, high-contrast color scheme to ensure readability for users with colorblindness. I myself am partially color blind, which is why I prioritized this design to meet my own needs and those of similar users, informed by my experience and discussions with peers about UI clarity.

Main Menu containing the following:

1. Plane selection buttons
2. Plane thumbnails
3. Plane descriptions
4. Plane locked/unlocked
5. Play button
6. Quit button

Inputs will include:

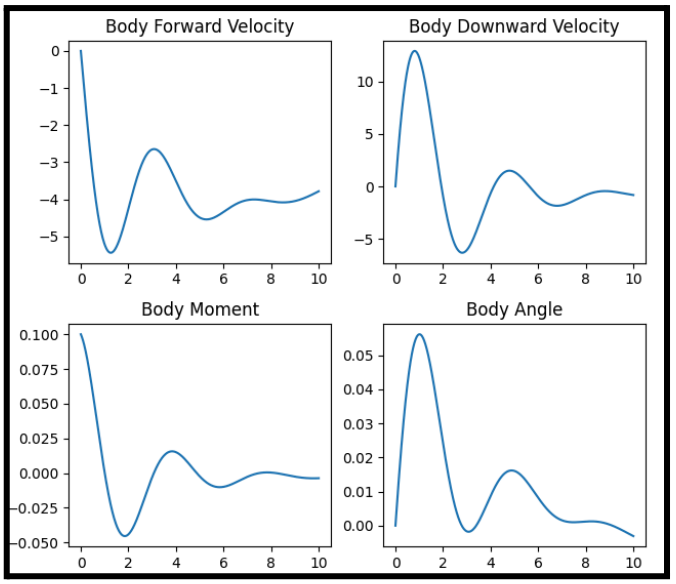
1. Elevators (W and S)
2. Ailerons (Q and E)
3. Rudder (A and D)
4. Pause and menu (ESC)

UI during the simulation should display some basic information such as fall rate, altitude, velocity and a Primary Flight Display
Transition to post-flight metrics

Post Flight Analysis

The post flight analysis will allow users to learn about their respective interests and possible mistakes with graphical feedback. It will display four graphs at once and utilise the matplotlib library.

Included graphs: AoA against time, Velocity against time, G-Force against time, altitude against time.



Limitations

Skill Set

I have only been consistently coding for 3 years now and the vast majority of my experience and knowledge lies within Python. I have previous experience with PyGame and MatPlotLib which will be helpful, however I will have to develop an understanding of 3 dimensional rendering independently. Additionally, as a crucial part of this project is its ability to run on low-end hardware, I must further my skill set in regards to program efficiency.

Lack of Available Data

Regarding the Plane Database, I will need 8 pieces of data: the 4 linearised matrices, a thumbnail image, an object file, and the textures of the plane. The thumbnail image will be easy to source, however both accurate object files and especially the matrices will be hard to find. This may lead to a limited variety of aircraft or I may have to independently calculate or estimate the matrices myself.

Simplified User Interface (UI)

The user interface (UI) will be designed to be simple and intuitive, but it may not feature the complexity or customization options that more advanced flight simulators offer. Detailed cockpit configurations or customizable HUDs are not necessary for most users outside of professional flight training. Additionally, accessibility features, like support for screen readers or alternative input methods, may not be as developed, potentially limiting the accessibility for users with specific needs.

Project Objectives

1. General

1.1. The program must be a downloadable Python application, with all supporting files, that is executable on Windows, macOS, and Linux systems, provided Python and the necessary libraries are installed.

1.2. The program must feature a clear, intuitive user interface with a consistent control system accessible to novice users.

1.3. The program must run efficiently on standard school computers or laptops with at least 4 GB RAM and a dual-core processor.

2. Physics Engine

2.1. The physics engine must calculate the aircraft's next state every frame with sufficient accuracy for educational demonstration of flight dynamics, using a fixed time step approximately equivalent to 60 FPS/ 60 Hz.

2.2. The physics engine must incorporate the following variables in its state-space model:

2.2.1. Last state of the system: Longitudinal and lateral state vectors (e.g., velocity, angle of attack, roll rate).

2.2.2. ***A*** Matrix from plane database

2.2.3. ***B*** Matrix from plane database

2.2.4. User Inputs: Elevator, aileron, and rudder deflections via keyboard controls.

2.3. The physics engine must perform the following matrix operations on 4x4 matrices:

2.3.1. Scalar Multiplication

2.3.2. Matrix Multiplication

2.3.3. Matrix Addition

2.3.4. Matrix Determinant

2.4. It should output the locational measures such that they can be rendered by the rendering engine

2.5. The physics engine must output positional and rotational data (x, y, z coordinates and Euler angles) for rendering.

3. Rendering Engine

3.1. The rendering engine should render the current aircraft state from the physics engine in a 3D environment every frame.

3.2. The rendering engine must display the following objects:

3.2.1. Aircraft model (loaded from .obj file)

3.2.2. Ground plane (textured surface)

3.2.3. Skybox (background environment).

3.3. The rendering engine must achieve a minimum of 30 FPS on standard hardware and provide visuals sufficient for educational purposes.

3.4. It should adapt resolution to system hardware

4. User Management

4.1. Users must log in and log out via a dedicated screen with SQLite-backend authentication, completing the process within 2 seconds on standard hardware.

4.2. Users must create an account through a registration screen, storing credentials securely.

4.3. Users must view and track their level and flight minutes via the main menu, with updates reflected after each flight.

5. User Interface

5.1. The program must feature a login screen as the entry point, transitioning to the main menu upon successful authentication.

5.2. The main menu must include:

5.2.1. Plane selection menu with short descriptions and thumbnails, locking planes based on user level.

5.2.2. Account details displaying pilot level and total flight minutes.

5.2.3. Exit button to close the application.

5.2.4. Play button to start the simulation with the selected plane.

5.3. Simulation inputs must include:

5.3.1. Elevators (W for down, S for up).

5.3.2. Ailerons (Q for left, E for right).

5.3.3. Rudder (A for left, D for right).

5.3.4. Pause and menu toggle (ESC key).

5.4. The simulation UI must display real-time data including vertical velocity, altitude, and control surface deflections (elevator, aileron, rudder) via graphical bars and text.

5.5. The program must transition seamlessly to a post-flight analysis screen upon flight completion or crash, within 1 second.

5.6. The program must have a UI with high-contrast colors to support accessibility for visually impaired users.

6. Post Flight Analysis

6.1. The post-flight screen must simultaneously display four graphs using Matplotlib:

6.1.1. Angle of Attack (AoA) vs. Time.

6.1.2. Velocity vs. Time.

6.1.3. G-Force vs. Time.

6.1.4. Altitude vs. Time.

6.2. The post-flight screen must update and display the user’s total flight minutes and level, reflecting flight performance (e.g., level up/down).

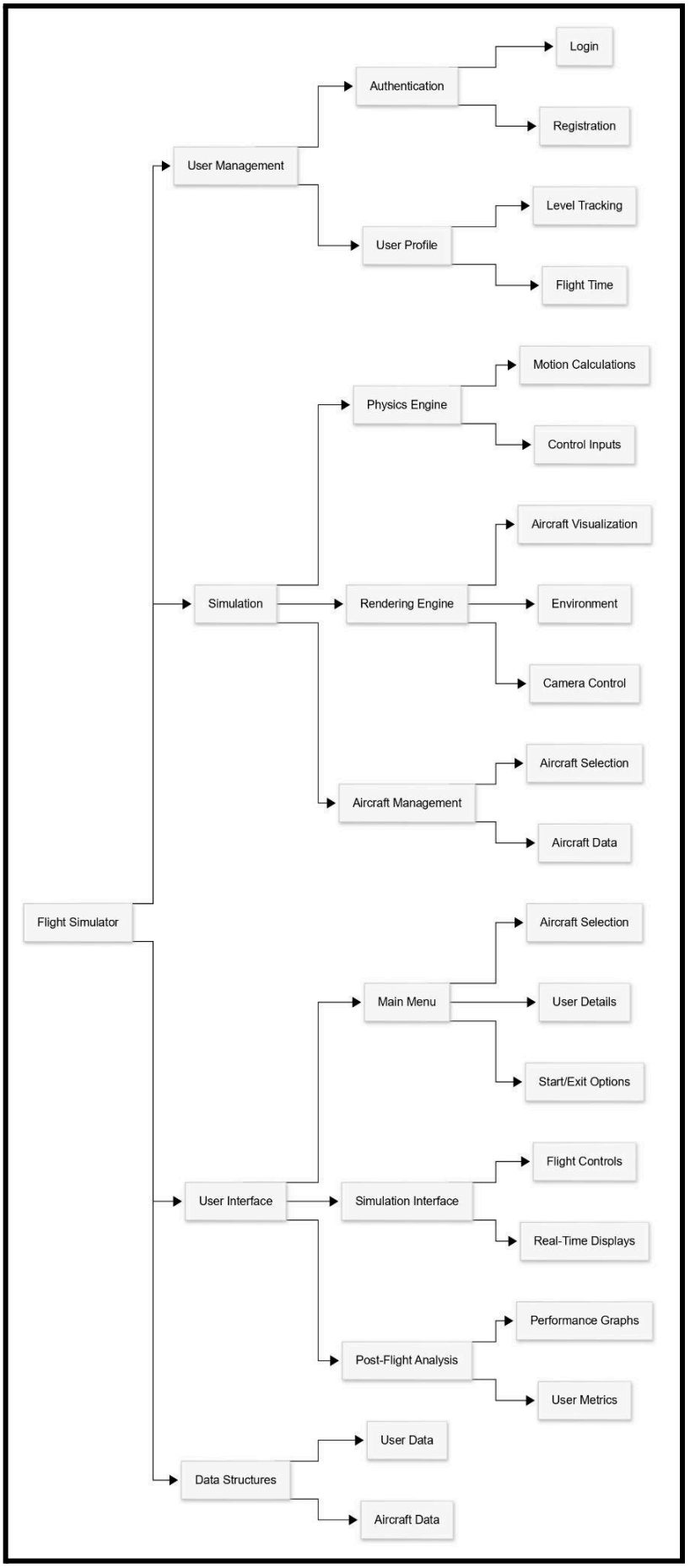
Design

High-Level Overview

I will be using the Urisa Python library to create my simulation. This library will allow me to easily render 3D object files (plane, ground, skybox), implement a GUI for login, pause, and menu screens, and easily interact with the user.

Hierarchy Chart

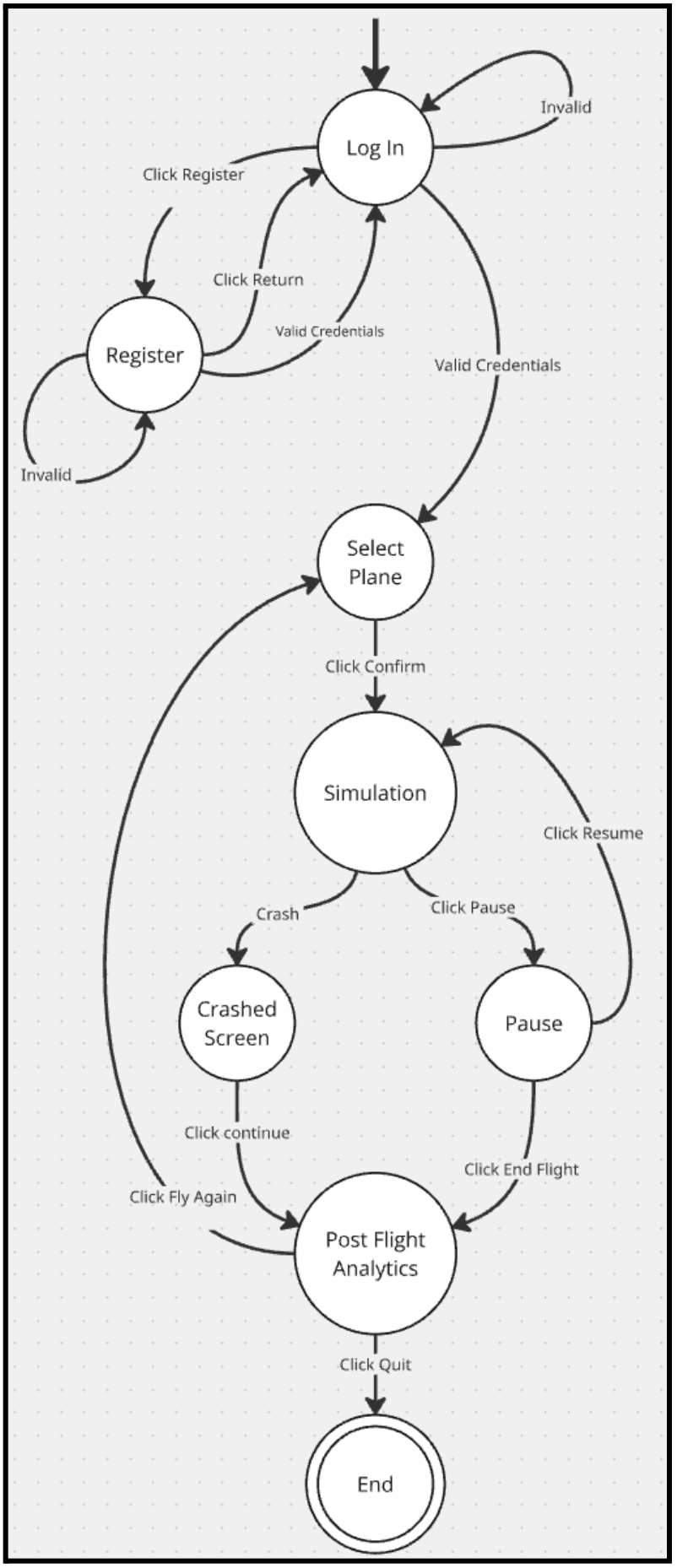
The following diagram shows the modular planned structure of my program, detailing the main components and their parts. It provides an eagle-eyed view of how the system is organized.



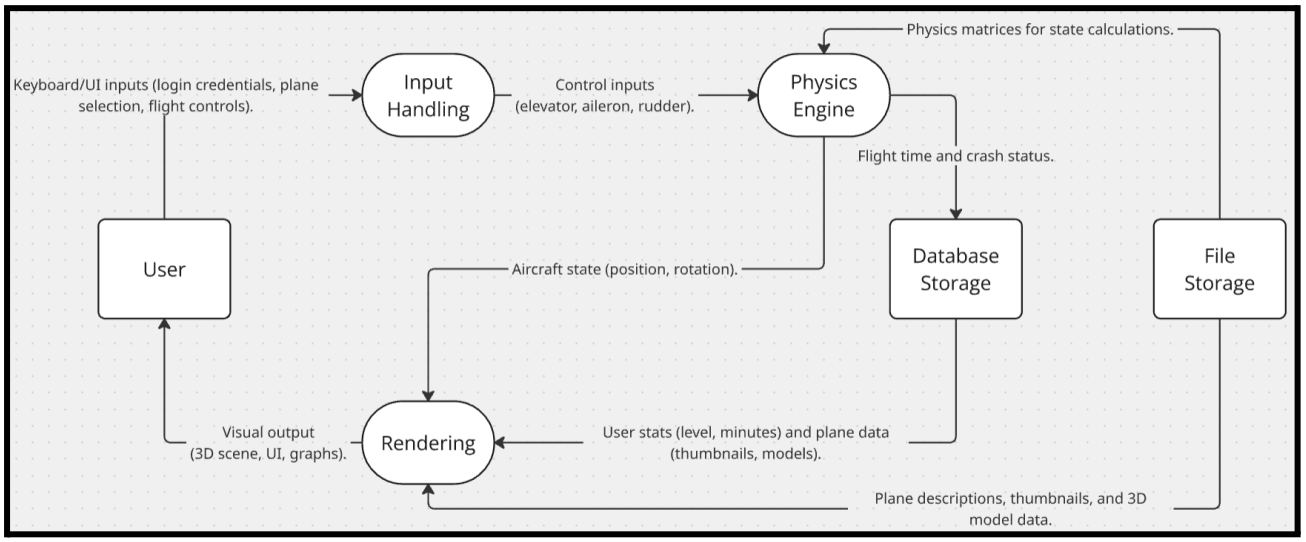
User Journey State Diagram

Below is a State Diagram based on the user experience. My chosen flow ensures security whilst maintaining simplicity to ensure the overall program is accessible to a wide range of users.

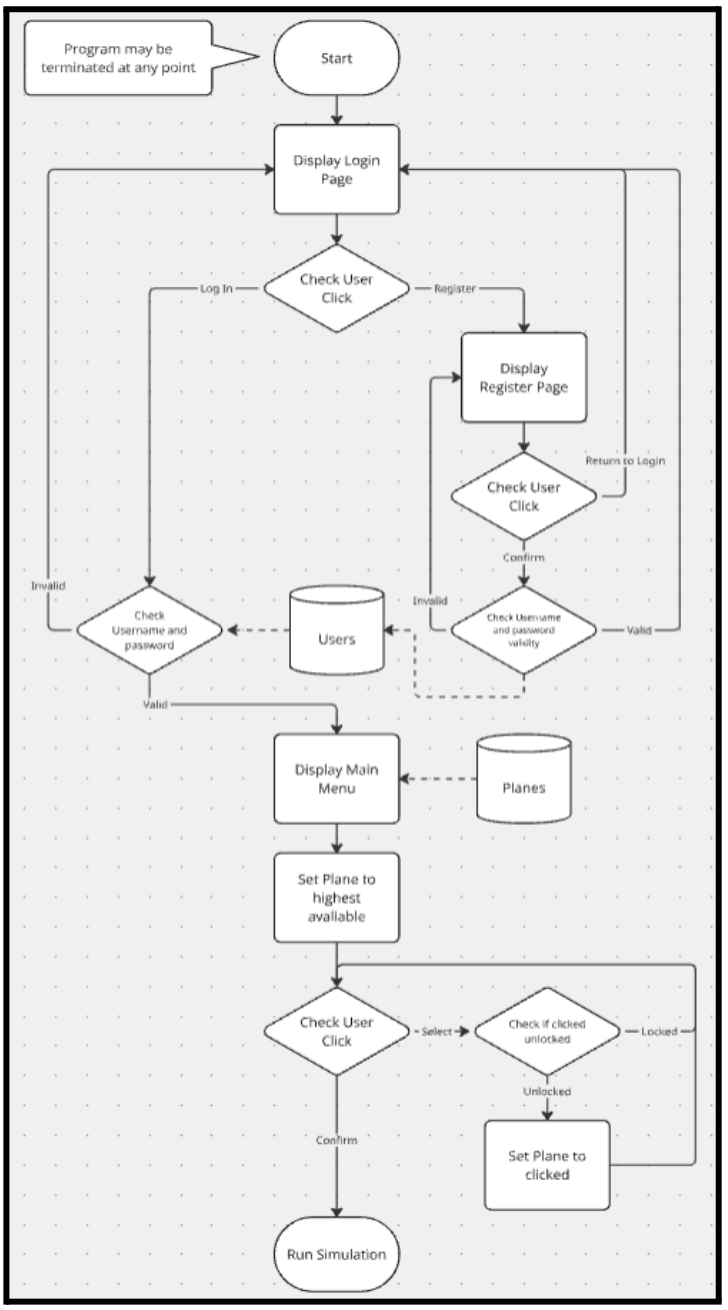
The diagram outlines the key states a user will navigate through, from starting the program to post-flight analytics and exiting. It incorporates essential security measures such as login authentication while maintaining an intuitive structure.



Data Flow Diagram



The above data flow diagram practically illustrates the movement of data through the flight simulator's core components, including user interfaces, physics calculations, and database interactions.



Menu

User Management

I have decided to use a user management system within my program. This will allow users to view data regarding their total flight time, prevent unwanted access to user accounts, allow multiple users to play on one device (not simultaneously), and allow me to implement a player skill based level system, where each player can level up and unlock more advanced planes. There will be a database consisting of User_ID, username, password_hash, level, flight_hours.

Menu Program Flowchart

Security Algorithms

User information will be stored in a database. This includes the hashed version of each user's password. Instead of storing the actual password, only a hashed version is saved, improving system security. Each password is salted to prevent rainbow table attacks, further improving security. Below is the pseudocode for the hashing system:

Salting function:

```
salt ← RANDOM_INT(31, 4294967295)      # Generate random salt in 32-bit range
RETURN salt                            # Return generated salt (INTEGER)
```

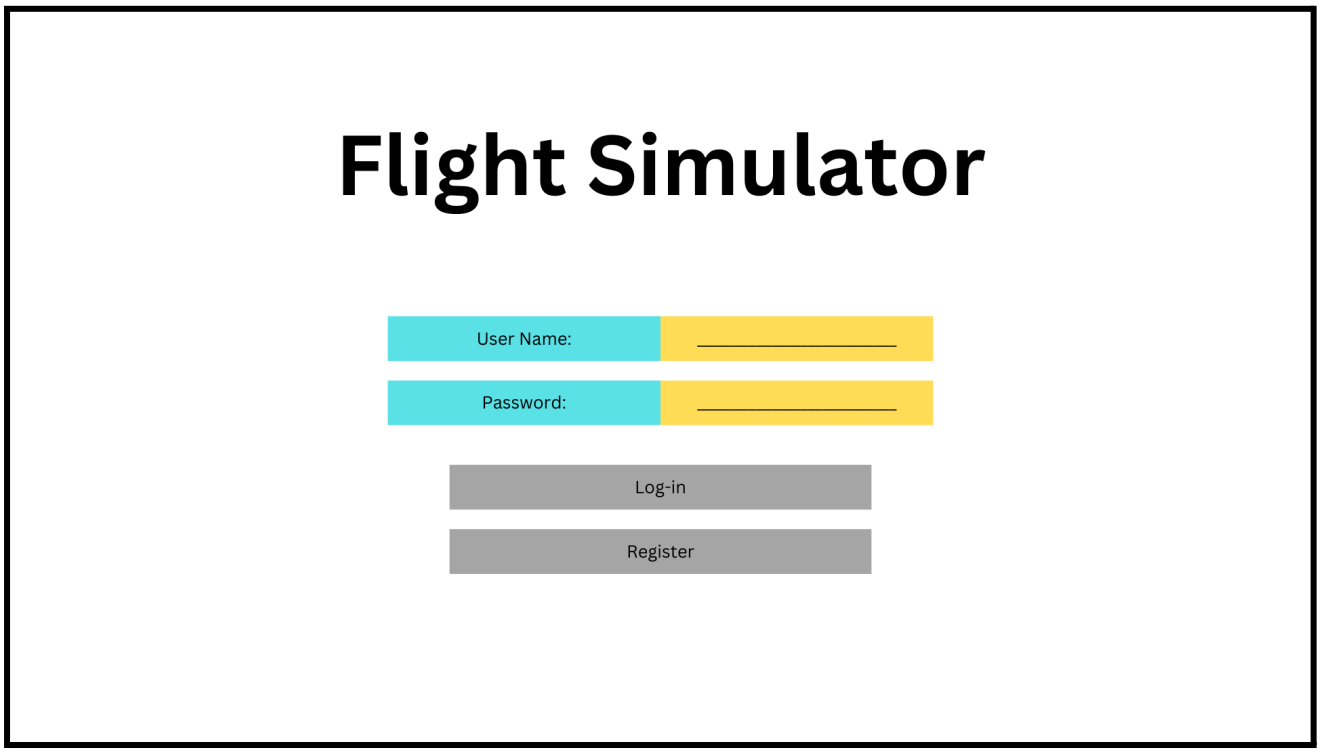
Hashing Algorithm:

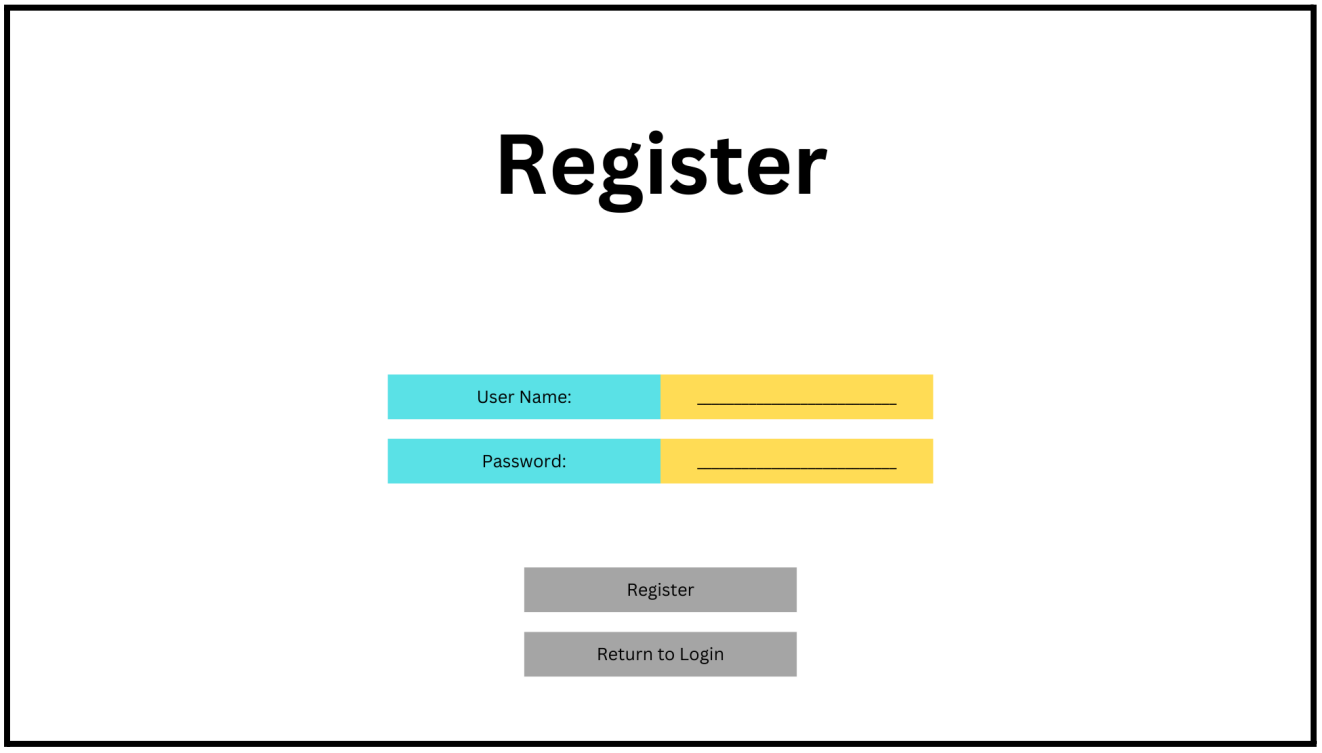
```
INPUT password                         # Receive password (type STRING)
INPUT salt                             # Receive salt (type INTEGER)

hash_val ← 867243217                   # Initial large seed prime

FOR char IN password DO
    ascii ← ASCII(char)                                  # Convert character to ASCII integer
    hash_val ← hash_val * 97                             # Multiply by a prime
    hash_val ← hash_val XOR (ascii + 144479)             # XOR with offset ASCII
    hash_val ← hash_val LEFT_SHIFT ((ascii MOD 5) + 1)   # Shift left by 1–5 bits
    hash_val ← hash_val XOR salt                         # XOR with salt
    hash_val ← hash_val + salt                           # Add salt
ENDFOR

hash_val ← hash_val MOD 387942806485727   # Modulo by large prime to fit within 63 bits

RETURN hash_val                # Return hashed password (INTEGER)
```

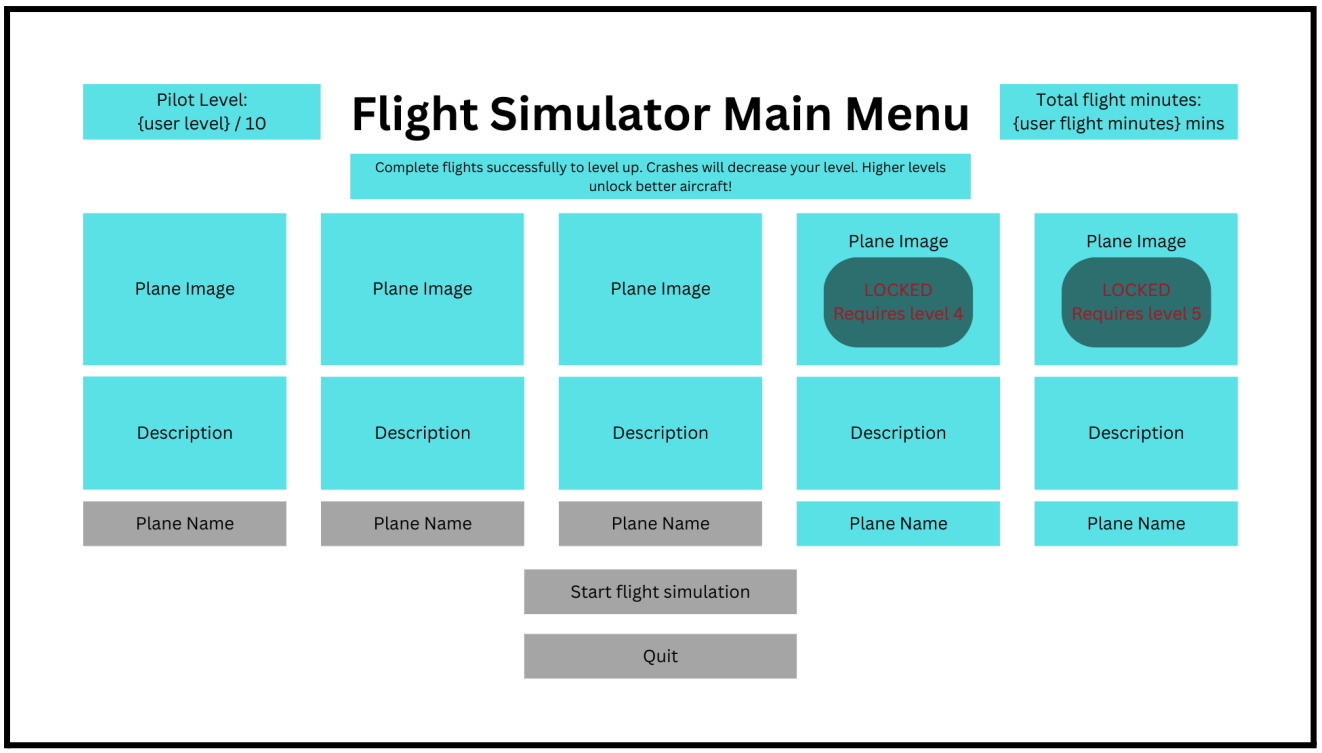
UI Design

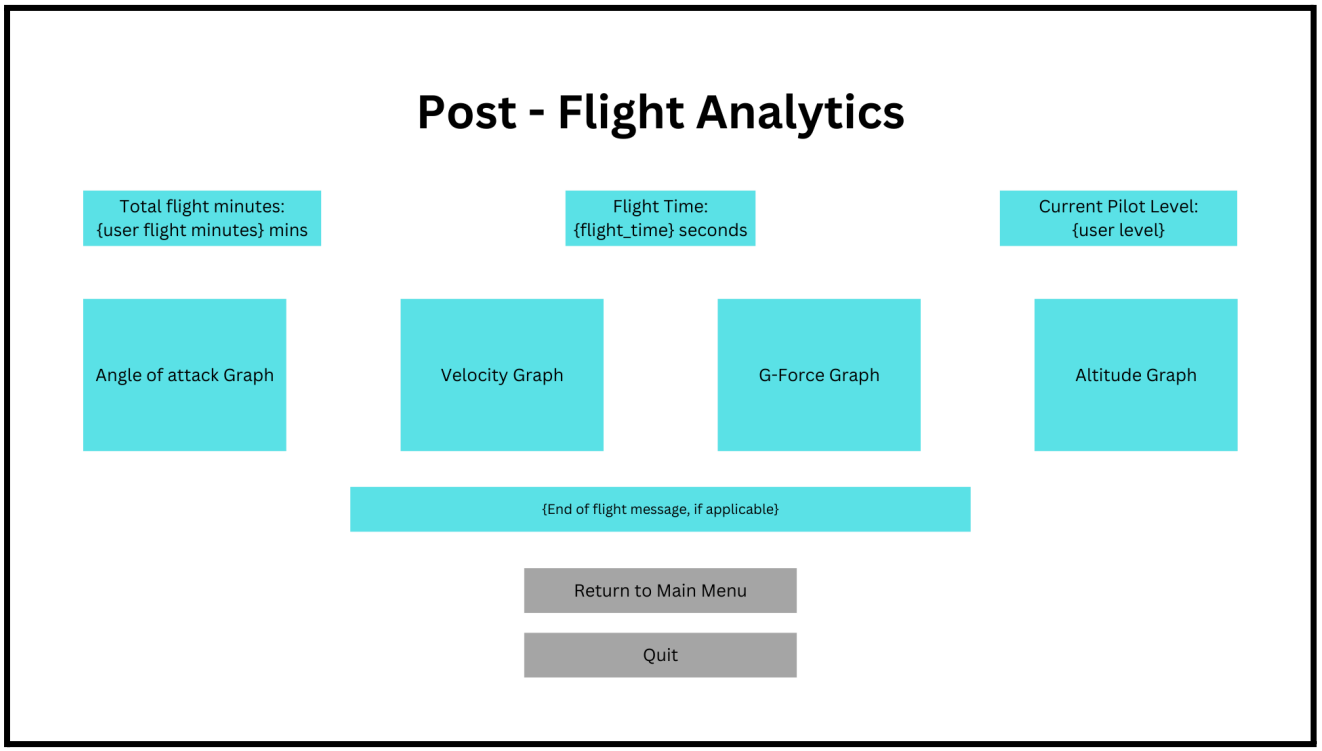
The following program pages are such that:

Both the red cross (present on all pages) and the Quit Program button quit the program.
Each grey box indicates a clickable button.
Each yellow box indicates a string enterable field.
Each blue box indicates display elements.






Pilot Level:
{user level} / 10
Flight Simulator Main Menu
Total flight minutes:
{user flight minutes} mins
Complete flights successfully to level up. Crashes will decrease your level. Higher levels unlock better aircraft!
Plane Image
Plane Image
Plane Image
Plane Image
LOCKED
Requires level 4
Plane Image
LOCKED
Requires level 5
Description
Description
Description
Description
Description
Plane Name
Plane Name
Plane Name
Plane Name
Plane Name
Start flight simulation
Quit


Post - Flight Analytics
Total flight minutes:
{user flight minutes} mins
Flight Time:
{flight_time} seconds
Current Pilot Level:
{user level}
Angle of attack Graph
Velocity Graph
G-Force Graph
Altitude Graph
{End of flight message, if applicable}
Return to Main Menu
Quit

Data Structures

User(User_ID, username, password_hash, salt, level, flight_hours)

Field	Type	Description
User_ID	Integer	Primary key
username	String	Username
password_hash	String	Hash of password
salt	Integer	Random salt
level	Integer	Skill level
flight_minutes	Float	Flight time

Simulation

The physics calculations use a state-space representation to determine aircraft behavior. These equations are 4 dimensional matrix differential equations, so in order to simulate flight, Eularian integration will be used to estimate the position of the plane.

This numerical integration technique is adequate for this application and works best with smaller time steps. This will be broken up into two tandem systems; longitudinal and lateral.

6DOF:

Aircraft motion is modeled with six degrees of freedom, 3 translational and 3 rotational.

Translation:

Forward: v * cos(β), where β is the sideslip angle (lateral misalignment).
Sideways: v * sin(β), lateral slip across the body axis.
Vertical: Derived from angle of attack α, approximates climb/descent rate.

Rotation:

Pitch (θ): Nose up/down tilt.
Yaw (ψ): Heading left/right.
Roll (ϕ): Wing bank angle.

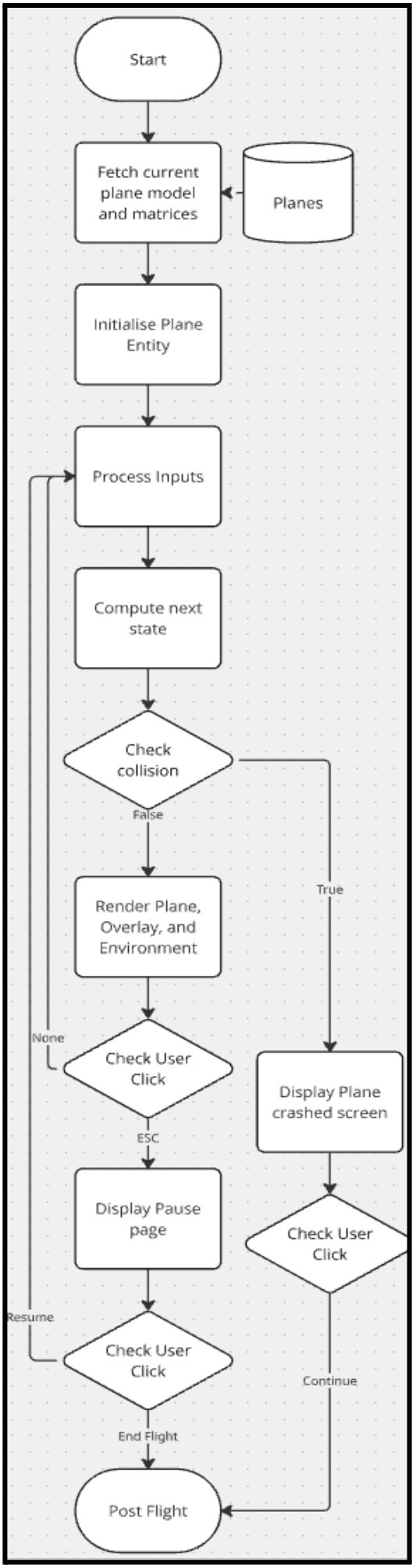
Longitudinal state space equation example:

$$\begin{bmatrix} \dot{u} \\ \dot{w} \\ \dot{q} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} -0.04225 & -0.11421 & 0 & -32.2 \\ -0.20455 & -0.49774 & 317.48 & 0 \\ 0.00003 & -0.00790 & -0.39499 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} u \\ w \\ q \\ \theta \end{bmatrix} + \begin{bmatrix} 0.00381 \\ -24.4568 \\ -4.51576 \\ 0 \end{bmatrix} \eta$$

Lateral state space equation example:

$$\begin{bmatrix} \dot{\beta} \\ \dot{p} \\ \dot{r} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} -0.127 & 0.0698 & -0.998 & 0.01659 \\ -2.36 & -1.02 & 0.103 & 0 \\ 11.1 & -0.00735 & -0.196 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \beta \\ p \\ r \\ \phi \end{bmatrix} + \begin{bmatrix} -0.00498 & 0.0426 \\ 28.7 & 5.38 \\ 0.993 & -6.90 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \delta_a \\ \delta_r \end{bmatrix}$$

Eulerian Integration:
$X_{t+\Delta t} = X_t + \Delta t \cdot X'_t$



Matrix Calculator

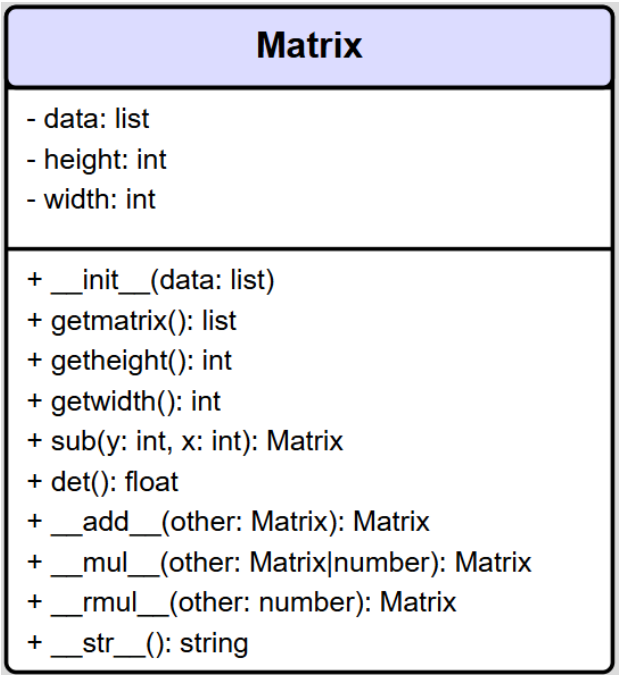
In order to work with the state space equation, my program must perform certain matrix calculations. It must handel:

Matrix-Matrix Multiplication
Matrix-Matrix Addition
Matrix-Scalar Multiplication
Matrix determinant calculation
Submatrix extraction (a part of matrix determinant calculation)

I will be implementing a matrix class to make these processes easier. I will implement the 5 above operations as methods of this matrix class, their Pseudocode algorithms are listed below.

The object will be made using a 2 dimensional array as input to the constructor. This class will also have private parameters height and width. There will be 3 methods, getwidth, getheight, and getmatrix, returning the width, height, and 2D array respectively. This implementation ensures the principle of **encapsulation** is followed

I will also be adding methods __add__ and __mul__ to this class. This will let me use python's inbuilt + and * operators on matrices, where they would normally display an error. For multiplication, the type (matrix or float) of input will be checked to see if matrix-matrix multiplication or matrix-scalar multiplication.



Matrix-Matrix Addition Algorithm

```
INPUT matrix_1, matrix_2
answer ← [ ]

FOR row ← 0 TO length(matrix_1) - 1 DO
   new_row ← [ ]
   FOR column ← 0 TO length(matrix_1 [0]) - 1 DO
      # Add corresponding elements from both matrices
      new_row.append(matrix_1 [row] [column] + matrix_2 [row] [column])
   ENDFOR
   # Add completed row to the result matrix
   answer.append(new_row)
ENDFOR

RETURN answer
```

Matrix-Scalar Multiplication Algorithm

```
INPUT matrix, scalar
answer ← [ ]

FOR row ← 0 TO length(matrix) - 1 DO
   new_row ← [ ]
   FOR column ← 0 TO length(matrix [0]) - 1 DO
      # Multiply each element by the scalar value
      new_row.append(matrix [row] [column] * scalar)
   ENDFOR
   # Store the scaled row in the result
   answer.append(new_row)
ENDFOR

RETURN answer
```

Matrix-Matrix Multiplication Algorithm

```
INPUT matrix_1, matrix_2
answer ← [ ]

FOR row ← 0 TO length(matrix_1) - 1 DO
   new_row ← [ ]
   FOR column ← 0 TO length(matrix_2 [0]) - 1 DO
      sum ← 0
      FOR x ← 0 TO length(matrix_1 [0]) - 1 DO
         # Compute dot product of row from matrix_1 and column from matrix_2
         sum ← sum + matrix_1 [row] [x] * matrix_2 [x] [column]
      ENDFOR
      # Append the computed sum to the new row
      new_row.append(sum)
   ENDFOR
   answer.append(new_row)
ENDFOR

RETURN answer
```

Submatrix Extraction Algorithm

```
INPUT matrix, y, x
answer ← [ ]

FOR i ← 0 TO length(matrix) - 1 DO
   IF i ≠ x THEN
      new_row ← [ ]
      FOR j ← 0 TO length(matrix [0]) - 1 DO
         IF j ≠ y THEN
            # Include element only if it's not in the excluded row or column
            new_row.append(matrix [i] [j])
         ENDIF
      ENDFOR
      # Add filtered row to the result
      answer.append(new_row)
   ENDIF
ENDFOR

RETURN answer
```

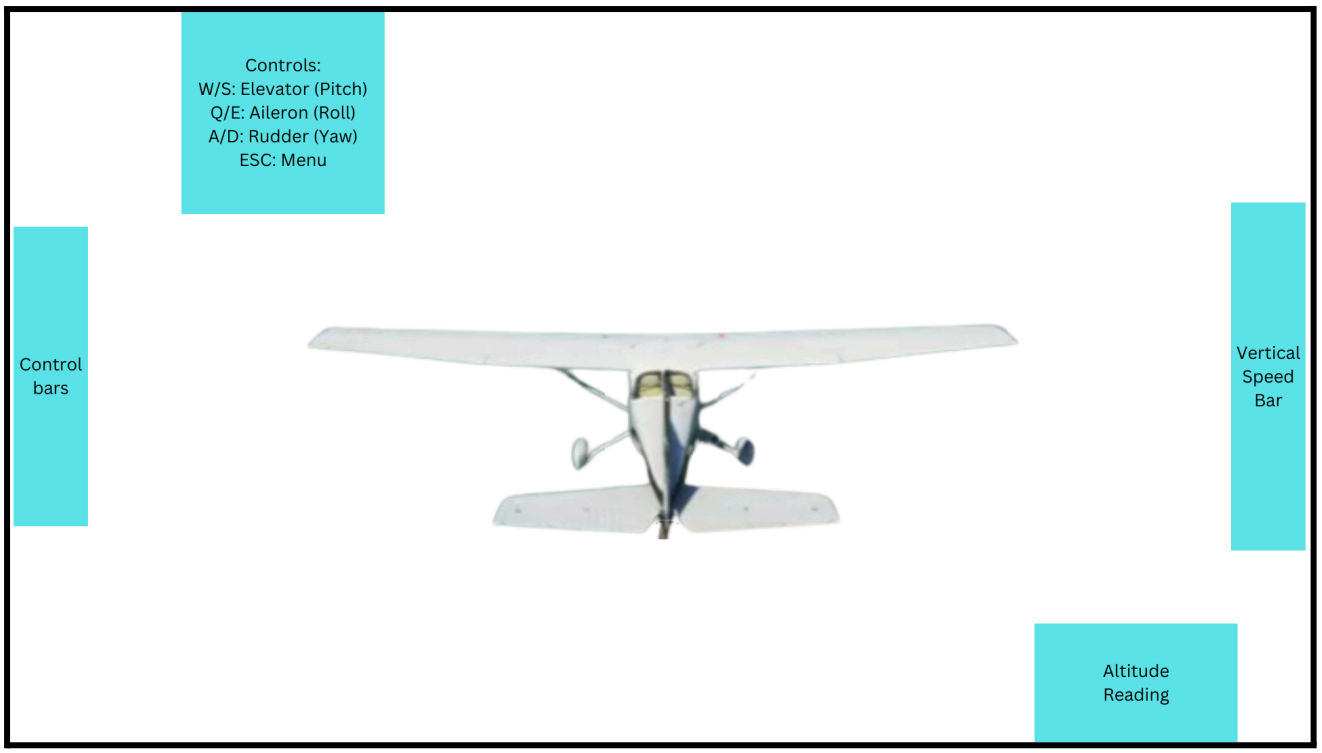
Matrix Determinant Algorithm - Recursive

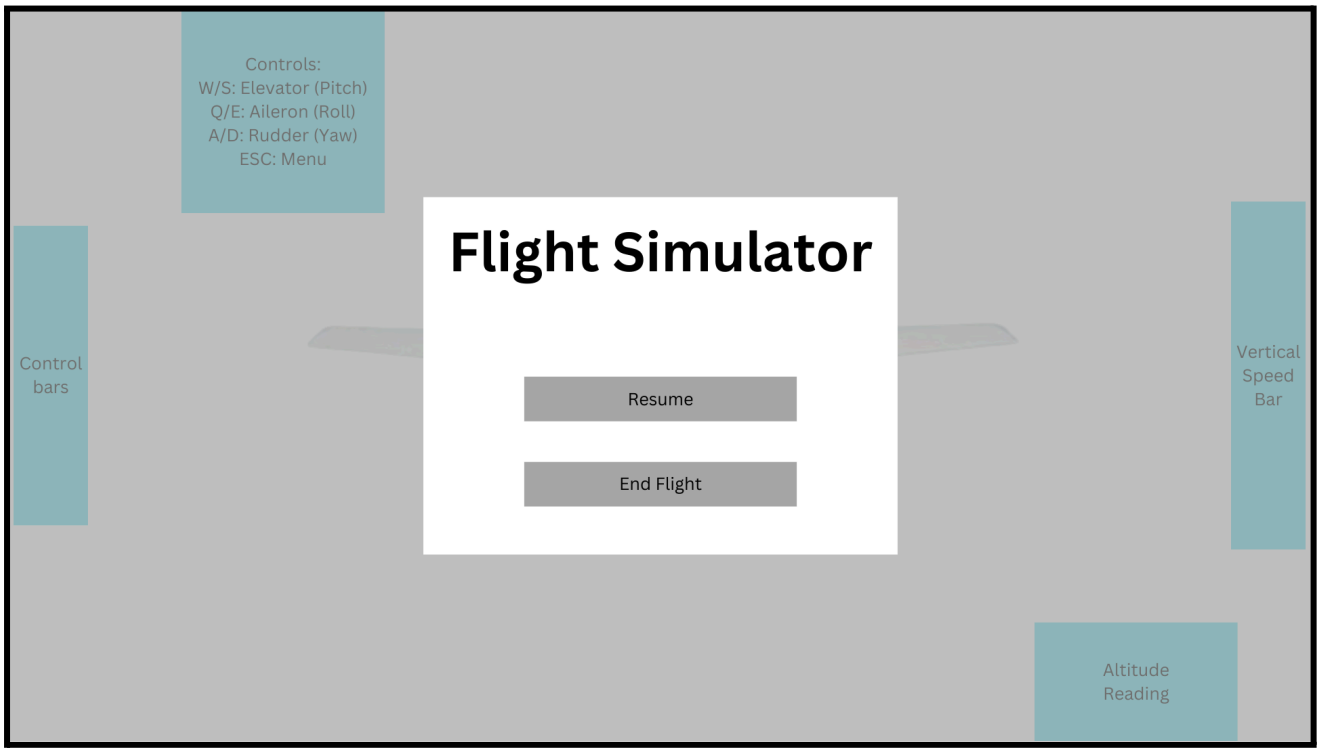
```
INPUT matrix
IF length(matrix) = 1 THEN
   # Base case: determinant of 1x1 matrix is the single element
   RETURN matrix [0] [0]
ELSE
   t ← 0
   FOR i ← 0 TO length(matrix) - 1 DO
      # Get submatrix excluding first row and current column
      submatrix ← Submatrix(matrix, 0, i)
      # Add term with alternating sign based on position
      t ← t + matrix [0] [i] * Determinant(submatrix) * ((-1) ^ i)
   ENDFOR
   # Return the accumulated sum as the determinant
   RETURN t
ENDIF
```

UI Design

The user interface includes various informative overlays on top of 3D rendering of the simulation itself. As for inputs, a conventional layout will be used as follows:

Key Pressed	Action
W	Pitch down
S	Pitch up
A	Yaw left
D	Yaw right
Q	Roll left
E	Roll right
Esc	Pause Menu


Controls:
W/S: Elevator (Pitch)
Q/E: Aileron (Roll)
A/D: Rudder (Yaw)
ESC: Menu
Control bars
Vertical Speed Bar
Altitude Reading


Controls:
W/S: Elevator (Pitch)
Q/E: Aileron (Roll)
A/D: Rudder (Yaw)
ESC: Menu
Flight Simulator
Resume
End Flight
Control bars
Vertical Speed Bar
Altitude Reading

Data Structures

The AIRCRAFT table stores essential data about each aircraft available in the simulator. It includes identification, difficulty level, flight characteristics, and file paths for visualization.

AIRCRAFT(Level, name, description, along_matrix, blong_matrix, alat_matrix, blat_matrix, image_path, model_path)

Field	Type	Description
Level	Integer	Primary key, unlock level
name	String	Aircraft name
description	String	File path to desc .txt file
matrix_path	String	File path to matrix .txt file
image_path	String	File path to thumbnail image
model_path	String	File path to .obj model

This structure ensures efficient data retrieval, supporting both the physics engine (for real-time calculations) and the rendering engine (for visualization). It also allows easy implementation of the plane selection process and the locked planes.

Technical Solution

File Structure

Below is the final structure of my project files:

		Page
./		
├── main.py	(Main game logic, UI, and flight simulation)	41
├── matrix.py	(Matrix operations for physics)	59
├── plane_management.py	(Manages aircraft data and database)	62
├── security.py	(Password hashing and salting)	63
├── user_management.py	(Manages aircraft data and database)	64
├── matrix_tests.py	(Tests for matrix module)	67
├── user_management_tests.py	(Tests for user_management module)	68
├── planes/	(Aircraft assets folder)	
│ ├── Cessna-172/		
│ │ ├── cessna.obj	(Model in OBJ format)	
│ │ ├── texture.png	(Model texture)	
│ │ ├── desc.txt	(Description for UI)	
│ │ ├── matrix.txt	(Physics matrix data)	
│ │ └── thumbnail.jpg		
│ ├── Boeing-737/		
│ │ ├── boeing.bam	(Model in BAM format)	
│ │ ├── texture.png		
│ │ ├── desc.txt		
│ │ ├── matrix.txt		
│ │ └── thumbnail.jpg		
│ ├── Spitfire/		
│ │ ├── spitfire.obj		
│ │ ├── texture.png		
│ │ ├── desc.txt		
│ │ ├── matrix.txt		
│ │ └── thumbnail.jpg		
│ ├── ORCA/		
│ │ ├── ORCA.bam		
│ │ ├── texture.png		
│ │ ├── desc.txt		
│ │ ├── Newsletter.txt		
│ │ └── thumbnail.png		
│ └── X-Wing/		
│ ├── xwing.obj		
│ ├── texture.png		
│ ├── desc.txt		
│ ├── matrix.txt		
│ └── thumbnail.jpg		
├── icon.ico	(Window icon)	
├── users.db	(User data database, autogenerated)	
└── planes.db	(Aircraft data database, autogenerated)	

Sources (excluding libraries)

https://stackoverflow.com/, (used to help understand python and fix some bugs)
https://aerocastle.wordpress.com/, (used for sourcing physics matrices)

Skills Used

Skill	Description	Page No.
Complex Scientific/Mathematical Model	Simulates aircraft dynamics using numerical integration of multiple state-space matrix differential equations.	47–54
Real-Time 3D Graphics and Physics Simulation	Combines 3D rendering and physics for an immersive simulation, showcasing game development expertise.	41
Complex User-Defined OOP	Implements classes for user, Ascending, plane, matrix, and UI with inheritance and polymorphism.	62, 60, 57, 41
Advanced Matrix Operations	Performs matrix calculations for physics simulations and validates them.	57, 65
Data Visualization with Graphs	Provides analytical insights through visual data representation, enhancing user experience.	41
Recursive Algorithms	Calculates matrix determinants using recursive Laplace expansion.	57
Dynamic UI Generation and Event Handling	Enables interactive and scalable UI, essential for user-facing applications.	41
Defensive Programming and Input Validation	Ensures software reliability and security, a hallmark of production-grade code.	41, 59, 64, 67, 68
Secure Password Policy Enforcement	Enhances security with robust input validation, critical for user authentication.	41
Complex User-Defined Algorithms	Handles password hashing, physics updates, and input validation.	61, 41
State Management in a Simulation Loop	Manages complex state in real-time, critical for simulation integrity and user flow.	41
Database-Driven Content Management	Supports scalable content delivery, key for dynamic applications.	41, 62, 64

Dynamic Generation of Objects Based on Complex OOP Model	Creates UI and 3D objects dynamically from database data.	41
Data Model in Database	Manages users and planes tables with primary keys and constraints using SQLite3.	62, 64-66
Simple Mathematical/Statistical Calculations (Advanced)	Computes G-force and velocity components for flight analytics.	41
Multi-Dimensional Arrays	Uses 2D lists for matrix data and state-space calculations.	57, 41
File(s) Organised for Direct Access	Reads matrix and description text files for plane data and physics.	41, 60
Writing and Reading from Files	Loads matrix and description data from text files.	41
Aggregate SQL Functions	Counts records to initialize the databases.	60
Simple Data Model in Database	Defines individual users and planes tables with basic structure.	62, 60
Simple OOP Model	Encapsulates user data with basic class structure.	62

Objectives

Objective	Relevant Page No.
General	
Develop a cross-platform Python application with all files, executable on Windows, macOS, and Linux with Python libraries, featuring an intuitive UI for novice users and efficient performance on 4GB RAM/dual-core hardware.	41, 57, 62, 63, 64
Physics Engine	
Calculate aircraft state every frame at ~60 FPS with accurate flight dynamics using a state-space model incorporating longitudinal/lateral vectors, A/B matrices from plane database, and user inputs (elevator, aileron, rudder).	47–54, 41, 57, 62

Perform scalar multiplication, matrix multiplication, addition, and determinants on 4x4 matrices.	57, 47–54
Output positional (x, y, z) and rotational (Euler angles) data for rendering.	41, 47–54
Rendering Engine	
Render the aircraft state in a 3D environment every frame, displaying aircraft model, ground plane, and skybox.	41
Achieve at least 30 FPS on standard hardware with visuals for educational purposes, adapting resolution to system hardware.	41
User Management	
Enable user login/logout and account creation via dedicated screens with SQLite-backend authentication within 2 seconds, storing credentials securely.	41, 63, 64
Allow users to view and track level and flight minutes in the main menu, updated after each flight.	41, 64
User Interface	
Provide a login screen as the entry point, transitioning to a main menu with plane descriptions, thumbnails, level-based locking, pilot level, flight minutes, and Exit/Play buttons.	41, 62
Support simulation inputs via WASDQE and ESC keys.	
Display real-time simulation data (vertical velocity, altitude, control surface deflections) via graphical bars and text.	49-54
Transition seamlessly to post-flight analysis screen within 1 second upon flight completion or crash.	55-56
Post Flight Analysis	
Display four Matplotlib graphs and update user’s total flight minutes and level based on flight performance on the post-flight screen.	56-58

Project Files

Zipped Code Directory: https://bit.ly/VianGargFlightSimulator

./main.py

Core flight simulator implementation using Ursina, integrating user interface screens, physics, and post-flight analytics. Contains Log In, Register, Main Menu, and Post-Flight Analysis screens along with the main flight simulation environment.

```
from ursina import *

from re import search
from matplotlib import pyplot

from matrix import Matrix
from user_management import User, UserManager
from plane_management import PlaneManager


##################################################################
################## GLOBAL VARIABLES/STATE ########################
##################################################################

current_user = None     # will store the logged-in user
flight_time = 0         # total flight time in seconds
crashed = False         # indicates if the plane crashed

##################################################################
#################### URSINA SCREENS ##############################
##################################################################

# Initialize the Ursina game engine with configuration settings

'''Ursina(title='ursina', icon='textures/ursina.ico', borderless:bool=None, fullscreen:bool=None, size=None,
    forced_aspect_ratio=None, position=None, vsync=True, editor_ui_enabled:bool=None, window_type='onscreen',
    development_mode:bool=None, render_mode=None, show_ursina_splash=False, use_ingame_console=False)
'''
app = Ursina(title = 'Flight Sim',
             development_mode = False,
             icon='icon.ico',
             borderless = False,
             size = [1920,1080],
             forced_aspect_ratio = True)
Text.default_origin = (0, 0)
Text.default_font = "VeraMono.ttf"

# ----------------------------
#    LOGIN SCREEN
# ----------------------------
class LoginScreen(Entity):
    """
    Login screen UI implementation.
    Demonstrates UI design and event handling in Ursina.
    """
    def __init__(self, user_manager):
        super().__init__()
        self.user_manager = user_manager
        # Create UI elements with precise positioning
        self.title = Text(text = 'Flight Simulator', position = (0, 0.3), origin = (0, 0), scale = (3, 3))
        self.username_txt = Text(text = 'Username:', position = (-0.25, 0.1), origin = (0, 0), scale = (1.3, 1.3))
        self.password_txt = Text(text = 'Password:', position = (-0.25, 0), origin = (0, 0), scale = (1.3, 1.3))
        self.username_field = InputField(position = (0, 0.1), scale = (0.3, 0.05))
        self.password_field = InputField(position = (0, 0), scale = (0.3, 0.05), hide_content = True)
        self.login_button = Button(text = 'Login', position = (0, -0.1), scale = (0.2, 0.05))
        self.register_button = Button(text = 'Register', position = (0, -0.2), scale = (0.2, 0.05))

        # Set up event handlers
        self.login_button.on_click = self.attempt_login
        self.register_button.on_click = self.go_to_register

        self.info_text = Text(text = '', position = (0, -0.3), origin = (0, 0), scale = 1.5)
```

```
    def attempt_login(self):
        """
        Validates user credentials and handles login attempt.
        Uses the UserManager for authentication.
        """
        username = self.username_field.text
        password = self.password_field.text
        user = self.user_manager.login(username, password)
        if user:
            global current_user
            current_user = user
            self.info_text.text = 'Login successful!'
            self.info_text.origin = (0, 0)
            self.start_main_menu()
        else:
            self.info_text.text = 'Login failed. Please try again.'
            self.info_text.origin = (0, 0)

    def clear_page(self):
        """
        Cleans up UI elements when transitioning to another screen.
        Demonstrates proper resource management.
        """
        # destroy each UI element
        destroy(self.title)
        destroy(self.username_field)
        destroy(self.password_field)
        destroy(self.username_txt)
        destroy(self.password_txt)
        destroy(self.login_button)
        destroy(self.register_button)
        destroy(self.info_text)

    def go_to_register(self):
        """
        Transitions to the registration screen.
        """
        self.clear_page()

        # Transition to Register Screen
        RegisterScreen(self.user_manager)

    def start_main_menu(self):
        """
        Transitions to the main menu screen after successful login.
        """
        self.clear_page()

        # Transition to the main menu
        MainMenu()
```

```
# ---------------------------
#       REGISTER SCREEN
# ---------------------------
class RegisterScreen(Entity):
    """
    Registration screen UI implementation.
    Demonstrates form validation and password policy enforcement.
    """
    def __init__(self, user_manager):
        super().__init__()
        self.user_manager = user_manager
        self.username_txt = Text(text = 'Username:', position = (-0.25, 0.1), origin = (0, 0), scale = (1.3, 1.3))
        self.password_txt = Text(text = 'Password:', position = (-0.25, 0), origin = (0, 0), scale = (1.3, 1.3))

        self.title = Text(text = 'Register', position = (0, 0.3), origin = (0, 0), scale = (3, 3))
        self.username_field = InputField(default_value = '', position = (0, 0.1), scale = (0.3, 0.05))
        self.password_field = InputField(default_value = '', position = (0, 0), scale = (0.3, 0.05), hide_content = True)
        self.register_button = Button(text = 'Register', position = (0, -0.1), scale = (0.2, 0.05))
        self.back_button = Button(text = 'Back to Login', position = (0, -0.2), scale = (0.2, 0.05))

        self.register_button.on_click = self.attempt_register
        self.back_button.on_click = self.back_to_login

        self.info_text = Text(text = '', position = (0, -0.3), scale = 1.5)

    def attempt_register(self):
        """
        Validates input and attempts to register a new user.
        Implements password policy checks:
        - Minimum length
        - No repeated characters
        - No repeated patterns
        """
        username = self.username_field.text.strip()
        password = self.password_field.text.strip()

        # Form validation
        if not username or not password:
            self.info_text.text = "Username and password cannot be empty!"
            self.info_text.origin = (0, 0)
            return

        if len(password) < 8:
            self.info_text.text = "Password must be atleast 8 characters long!"
            self.info_text.origin = (0, 0)
            return

        # Password strength check 1: No three consecutive identical characters
        if search(r'(.)\1{2}', password):
            self.info_text.text = "Password cannot have 3 consecutive identical characters!"
            self.info_text.origin = (0, 0)
            return

        # Password strength check 2: No repeating patterns
        if search(r'(..)\1', password):
            self.info_text.text = "Password cannot have consecutive identical character pairs!"
            self.info_text.origin = (0, 0)
            return
```

```
        if not (search(r'\d', password) and search(r'[!@#$%^&*(),.?":{}|<>]', password) and search(r'[A-Z]', password)):
            self.info_text.text = "Password must contain at least 1 digit,\n 1 special character, and 1 uppercase letter!"
            self.info_text.origin = (0, 0)
            return

        # Attempt registration
        if self.user_manager.register_user(username, password):
            self.info_text.text = "Registration successful! Returning to login..."
            self.info_text.origin = (0, 0)
            invoke(self.back_to_login, delay = 1)  # Auto-return after success
        else:
            self.info_text.text = "Username already exists. Try another one."
            self.info_text.origin = (0, 0)

    def back_to_login(self):
        """
        Returns to the login screen.
        Demonstrates proper UI cleanup.
        """
        destroy(self.title)
        destroy(self.username_field)
        destroy(self.password_field)
        destroy(self.register_button)
        destroy(self.back_button)
        destroy(self.info_text)
        destroy(self.username_txt)
        destroy(self.password_txt)
        LoginScreen(self.user_manager)  # Go back to login screen
```

```
# ----------------------------
#       MAIN MENU SCREEN
# ----------------------------
class MainMenu(Entity):
    """
    Main menu with aircraft selection.
    Demonstrates dynamic UI creation and component interaction.
    """
    def __init__(self, plane_manager = PlaneManager()):
        super().__init__()
        camera.orthographic = True
        camera.fov = 1

        # Get planes from plane manager
        self.plane_manager = plane_manager
        self.planes = self.plane_manager.get_all_planes_info()

        # Set up UI
        self.title = Text(text = 'Flight Simulator Main Menu', position = (0, 0.4), origin = (0, 0), scale = (2, 2))

        # Display user level
        global current_user
        self.user_level_text = Text(
            text = f'Pilot Level:\n{current_user.level} / 10',
            position = (-0.66, 0.4),
            origin = (0, 0),
            scale = (1.3, 1.3),
            color = color.yellow)

        self.flight_time_text = Text(
            text = f'Total Flight Time:\n{current_user.flight_minutes:.2f} mins',
            position = (0.66, 0.4),
            origin = (0, 0),
            scale = (1.3, 1.3),
            color = color.yellow)

        # Add level system explanation
        self.level_info = Text(
            text = '''Complete flights successfully to level up.\n
                    Crashes will decrease your level.\n
                    Higher levels unlock better aircraft!''',
            position = (0, 0.3),
            origin = (0, 0),
            scale = (1, 1),
            color = color.white)

        self.start_button = Button(text = 'Start Flight Simulation', position = (0, -0.35), scale = (0.45, 0.05))
        self.start_button.on_click = self.start_simulation
        self.quit_button = Button(text = 'Quit', position = (0, -0.45), scale = (0.25, 0.05))
        self.quit_button.on_click = application.quit

        # Dictionary mapping plane names to handler functions
        self.button_dict = {}
        for plane in self.planes:
            self.button_dict[plane[1]] = self.set_plane

        self.buttons = {}  # Store button references for updating color
        self.descriptions = []
        self.images = []
        self.selected_plane = None  # Track selected aircraft
        self.locked_text = []  # Track locked plane text indicators
```

```
        # Create buttons in a horizontal layout
        position_x = -0.66      # Initial X position
        offset = position_x/-2  # Spacing between buttons


        # Create thumbnail images and descriptions
        for i in range(len(self.planes)):
            plane = self.planes[i]
            plane_level = plane[0]  # Level requirement for the plane
            plane_name = plane[1]   # Name of the plane
            self.display_plane(plane[2], plane[3], offset * i + position_x, plane_level)

        # Create plane selection buttons
        for i, plane in enumerate(self.planes):
            plane_name = plane[1]
            plane_level = plane[0]

            # Create the button
            button = Button(
                text = plane_name,
                scale = (0.25, 0.05),
                position = (position_x, -0.25),
                color = color.black90 if plane_level <= current_user.level else color.gray.tint(-0.4))

            # Set button behavior based on level requirement
            if plane_level <= current_user.level:
                button.on_click = Func(self.set_plane, plane_name)  # Enable selection
            else:
                button.disabled = True  # Disable the button

            self.buttons[plane_name] = button  # Store reference
            position_x += offset

        # Set default selection to the highest level plane available to the user
        available_planes = [p[1] for p in self.planes if p[0] <= current_user.level]
        if available_planes:
            self.set_plane(available_planes[-1])
        else:
            # Fallback to the first plane if somehow no planes are available
            self.set_plane(self.planes[0][1])

    def display_plane(self, image_path, description_path, x, level_req):
        """
        Displays a plane thumbnail and description.
        Demonstrates file I/O for loading descriptions and images.
        """
```

```
        # Show plane thumbnail
        self.images.append(Entity(model = 'quad', texture = image_path, position = (x, 0.12), scale = (0.25, 0.2), z = 2))

        # Read description from file
        f = open(description_path, "r")
        desc_text = f.read()
        f.close()

        # Add description text
        self.descriptions.append(Text(text = desc_text, position = (x-0.128, 0.01), wordwrap = 10, scale = 0.8))
```

```
        # Add "LOCKED" overlay for planes that require higher level
        if level_req > current_user.level:
            locked = Text(
                text = f"LOCKED\nRequires\nLevel {level_req}",
                position = (x, 0.12),
                origin = (0, 0),
                scale = (1, 1),
                color = color.red,
                background = True,
                background_color = color.black50,
                z = 3)
            self.locked_text.append(locked)


    def set_plane(self, plane_name):
        """
        Selects a plane and updates the UI.
        Demonstrates visual feedback in UI.
        """
        # Reset all buttons to default color
        for btn in self.buttons.values():
            if not btn.disabled:
                btn.color = color.black90

        # Highlight the selected button
        self.buttons[plane_name].color = color.gray
        self.selected_plane = plane_name

    def start_simulation(self):
        """
        Cleans up UI and transitions to the flight simulator.
        """
        for btn in self.buttons.values():
            destroy(btn)
        for img in self.images:
            destroy(img)
        for txt in self.descriptions:
            destroy(txt)
        for locked in self.locked_text:
            destroy(locked)

        destroy(self.title)
        destroy(self.user_level_text)
        destroy(self.flight_time_text)
        destroy(self.level_info)
        destroy(self.start_button)
        destroy(self.quit_button)
        FlightSimulator(self.selected_plane, self.plane_manager)
```

```
# --------------------------
#       FLIGHT SIMULATOR
# --------------------------
class FlightSimulator(Entity):
    """
    Core flight simulator implementation.
    Demonstrates advanced physics simulation using:
    - State-space model for aircraft dynamics
    - Matrix operations for state updates
    - 3D transformations for aircraft motion
    - Real-time control input handling
    """
    def __init__(self, selected_plane, plane_manager):
        super().__init__()
        self.plane_manager = plane_manager
        self.plane_physics = self.plane_manager.get_plane_physics(selected_plane)

        self.setup_physics()

        # Configure 3D environment
        self.sky = Sky(texture = 'sky_sunset')
        self.setup_ground()

        camera.orthographic = False
        camera.fov = 90

        # State vectors for aircraft dynamics:
        # Longitudinal: [velocity, angle_of_attack, pitch_angle, pitch_rate]
        # Lateral: [sideslip_angle, roll_rate, yaw_angle, roll_angle]
        self.state_long = Matrix([[0], [0], [0], [0]])
        self.state_lat = Matrix([[0], [0], [0], [0]])

        self.dt = 0.01667       # Time step (~60 FPS)
        self.flight_time = 0    # Local flight timer
        self.run = True
        self.menu_active = False  # Track if escape menu is open
        self.crashed = False      # Track if aircraft has crashed

        # Ground collision parameters
        self.ground_y = -100      # Ground altitude (matching setup_ground)
        self.crash_message = None # Placeholder for crash message

        # Control surface deflections
        self.elevator = 0  # Pitch control
        self.aileron = 0   # Roll control
        self.rudder = 0    # Yaw control

        # UI elements
        self.setup_overlay()
        self.setup_esc_menu()

        # Data collection lists for graphs
        self.time_data = []
        self.aoa_data = []        # Angle of Attack (state_long[1])
        self.velocity_data = []   # Velocity (state_long[0] + cruise_speed)
        self.gforce_data = []     # G-Force (approximated from vertical acceleration)
        self.altitude_data = []   # Altitude (plane.y - ground_y)
```

```
    def setup_ground(self):
        """
        Creates a large textured ground plane below the aircraft.
        Provides visual reference for altitude and motion.
        """
        # Create a large plane for the ground
        ground_size = 50000  # Size of the ground plane
        self.ground_y = -100 # Ground altitude

        # Create ground mesh
        self.ground = Entity(
            model = 'plane',
            scale = (ground_size, 1, ground_size),
            position = (0, self.ground_y, 0),
            collider = None,  # No need for physical collider, we'll handle collision manually
            texture = 'white_cube',  # Use a default texture
            texture_scale = (ground_size/20, ground_size/20),  # Repeat texture to avoid stretching
            color = color.green.tint(-0.3))  # Adjust color to look like terrain

    def check_ground_collision(self):
        """
        Checks if the aircraft has collided with the ground.
        Ends the flight if a collision is detected.
        """
        # Get the lowest point of the aircraft (assuming the plane's pivot is at its center)
        # Add a small buffer (e.g., 1 unit) to account for the size of the aircraft model
        aircraft_lowest_point = self.plane.y - 1

        # Check if the aircraft's lowest point is at or below ground level
        if aircraft_lowest_point <= self.ground_y:
            self.crashed = True
            self.run = False

            # Create crash message
            self.crash_message = Text(
                text = 'AIRCRAFT CRASHED!',
                position = (0, 0.2),
                origin = (0, 0),
                scale = 3,
                color = color.red)

            # Add "Continue" button to proceed to post-flight analysis
            self.continue_button = Button(
                parent = camera.ui,
                text = 'Continue to Analysis',
                position = (0, 0),
                scale = (0.4, 0.08),
                color = color.gray.tint(.2),
                highlight_color = color.gray.tint(.4),
                on_click = self.end_simulation)

            # Optional: Add dramatic visual/audio effects for crash
            # For example, change the plane color to indicate damage
            self.plane.color = color.red
```

```
    def setup_physics(self):
        """
        Sets up aircraft model and loads physics matrices.
        Demonstrates file I/O for loading simulation parameters.
        State-space model matrices A, B, C, D represent aircraft dynamics.
        """
        # Load 3D model and texture
        self.plane = Entity(
            model = self.plane_physics[0],
            texture = self.plane_physics[1],
            scale = 1,
            rotation = (0, 0, 0),
            position = (0, -2, 20))

        # Load state-space matrices from file
        matrix_path = self.plane_physics[2]
        f = open(matrix_path, "r")
        full_string = f.read()
        f.close()

        # Parse matrix data from file
        matrix_combo = [list(map(float, row.split(','))) for row in full_string.split('\n')]

        # Create state-space model matrices
        # A and B matrices for longitudinal dynamics (pitch, velocity)
        self.A = Matrix(matrix_combo[0:4])
        self.B = Matrix(matrix_combo[4:8])

        # C and D matrices for lateral dynamics (roll, yaw)
        self.C = Matrix(matrix_combo[8:12])
        self.D = Matrix(matrix_combo[12:16])

    def setup_overlay(self):
        """
        Initialize UI elements for flight information display.
        Demonstrates layered UI design with parent-child relationships.
        """
        # Explicitly create and assign each bar without setattr
        self.elevator_bar_bg, self.elevator_bar_fill = self.create_bar(-0.85, color.green)
        self.aileron_bar_bg, self.aileron_bar_fill = self.create_bar(-0.80, color.blue)
        self.rudder_bar_bg, self.rudder_bar_fill = self.create_bar(-0.75, color.yellow)
        self.vv_bar_bg, self.vv_bar_fill = self.create_bar(0.85, color.red)

        # Add altitude display (moved outside loop)
        self.altitude_text = Text(
            text = 'Altitude: 0 m',
            position = (0.6, -0.4),
            origin = (0, 0),
            scale = 1.5)

        # Updated instructions text (moved outside loop)
        self.instructions = Text(
            text = 'Controls:\nW/S: Elevator (Pitch)\nQ/E: Aileron (Roll)\nA/D: Rudder (Yaw)\nESC: Menu',
            position = (-0.5, 0.4),
            origin = (0, 0),
            scale = 1.5)
```

```
    def create_bar(self, pos, fill_color):
        bg = Entity(parent = camera.ui, model = 'quad', color = color.gray, scale = (0.03, 0.5), position = (pos, 0))
        fill = Entity(parent = bg, model = 'quad', color = fill_color, scale = (0.8, 0), position = (0, 0))
        return bg, fill

    def setup_esc_menu(self):
        """
        Create a pause menu that appears when ESC is pressed.
        The menu allows the player to resume or exit the simulation.
        """
        # Create menu container (initially hidden)
        self.menu_panel = Entity(
            parent = camera.ui,
            model = 'quad',
            color = color.black66,
            scale = (0.4, 0.3),
            position = (0, 0),
            enabled = False)

        # Menu title
        self.menu_title = Text(
            parent = self.menu_panel,
            text = 'PAUSED',
            position = (0, 0.1),
            origin = (0, 0),
            scale = 2,
            color = color.white)

        # Resume button
        self.resume_button = Button(
            parent = self.menu_panel,
            text = 'Resume Flight',
            position = (0, 0),
            scale = (0.8, 0.1),
            color = color.azure,
            highlight_color = color.azure.tint(.2),
            on_click = self.resume_game)

        # Exit button
        self.exit_button = Button(
            parent = self.menu_panel,
            text = 'End Flight',
            position = (0, -0.12),
            scale = (0.8, 0.1),
            color = color.red.tint(.2),
            highlight_color = color.red.tint(.4),
            on_click = self.exit_game)

        # Set up input handler for ESC key
        self.input_handler = Entity()
        self.input_handler.input = self.handle_input

    def handle_input(self, key):
        """
        Handle keyboard input for the ESC key to toggle the menu.
        """
        if key ==  'escape':
            self.toggle_menu()
```

```
    def toggle_menu(self):
        """
        Toggle the visibility of the pause menu and update the simulation state.
        """
        self.menu_active = not self.menu_active
        self.menu_panel.enabled = self.menu_active

        # Pause/resume the simulation
        self.run = not self.menu_active

        # Disable/enable mouse control for menu interaction
        mouse.locked = not self.menu_active
        mouse.visible = self.menu_active

    def resume_game(self):
        """
        Resume the game when the resume button is clicked.
        """
        self.menu_active = False
        self.menu_panel.enabled = False
        self.run = True
        mouse.locked = True
        mouse.visible = False

    def exit_game(self):
        """
        Exit the simulation when the exit button is clicked.
        """
        self.run = False
        self.end_simulation()
```

```
    def update(self):
        """
        Main update loop for the flight simulator.
        Called every frame by the Ursina engine.
        Implements the simulation time step and physics updates.
        """
        if not self.run:
            if not self.menu_active and not self.crashed:  # If not running and not in menu/crash state
                return

        if self.run:  # Only update simulation if running
            self.flight_time += self.dt

            # Update control inputs based on keyboard
            self.update_control_inputs()

            # Update aircraft state using state-space model
            self.update_state_vectors()

            # Update aircraft position and rotation
            self.update_plane()

            # Check for ground collision
            self.check_ground_collision()

            # Update camera position to follow aircraft
            self.update_camera()

            # Update UI elements
            self.render_overlay()

            # Update data collection lists for graphs
            self.update_data()

            # End simulation after 120 seconds
            if self.flight_time > 120:
                self.run = False
                self.end_simulation()

    def update_control_inputs(self):
        """
        Update elevator, aileron, and rudder based on user input.
        Demonstrates real-time control input handling.
        """
        self.elevator = self.update_control_surface(self.elevator, 'w', 's', 5)
        self.aileron = self.update_control_surface(self.aileron, 'e', 'q', 5)
        self.rudder = self.update_control_surface(self.rudder, 'a', 'd', 5)
```

```python
def update_control_surface(self, control, increase_key, decrease_key, maximum):
    """
    Helper method to update a control surface based on user input.
    Implements proportional control with spring-back behavior.
    """
    # Apply input based on key presses within limits
    if held_keys[increase_key] and control < maximum:
        control += 3 * self.dt
    elif held_keys[decrease_key] and control > -maximum:
        control -=  3 * self.dt
    # Spring-back effect when no keys are pressed
    elif control > 0:
        control -=  5 * self.dt
    elif control < 0:
        control += 5 * self.dt
    # Snap to zero for very small values
    if abs(control) < 10 * self.dt and not (held_keys[increase_key] or held_keys[decrease_key]):
        control = 0
    return control

def update_state_vectors(self):
    """
    Updates longitudinal and lateral state vectors using state-space model.

    This implements a discrete-time linear state-space model:
    x[k+1] = x[k] + (Ax[k] + Bu[k])*dt

    Where:
    - A, B, C, D are the state-space matrices representing aircraft dynamics
    - x is the state vector (position, velocity, angles, rates)
    - u is the control input vector (elevator, aileron, rudder)
    - dt is the time step

    Demonstrates advanced aerospace concepts:
    - State-space representation of dynamic systems
    - Discrete-time integration using Euler method
    - Aircraft stability and control theory
    """
    # Update longitudinal state using matrix operations
    # State vector: [velocity, angle_of_attack, pitch_angle, pitch_rate]
    # Control input: elevator deflection (affects pitch primarily)
    self.state_long += (self.A * self.state_long + self.B * self.elevator) * self.dt

    # Update lateral state using matrix operations
    # State vector: [sideslip_angle, roll_rate, yaw_angle, roll_angle]
    # Control inputs: aileron (roll) and rudder (yaw) deflections, constructed in to an input matrix
    self.state_lat += (self.C * self.state_lat + self.D * Matrix([[self.aileron], [self.rudder]])) * self.dt
```

```
    def update_plane(self):
        """
        Updates plane position and rotation based on state vectors.

        Demonstrates advanced aerodynamics and flight physics:
        - Coordinate system transformations between body and earth frames
        - Six degrees of freedom (6DOF) aircraft movement
        - Euler angle rotations and their applications
        - Velocity decomposition into forward, vertical, and horizontal components

        Uses the custom Matrix class for all calculations, showing practical
        application of the OOP principles implemented earlier.
        """
        # Scale factor to convert physical units to visual representation
        movment_scale = 1e-3

        # Extract state variables and convert from degrees to radians where needed
        # 57.2958 is the conversion factor (180/π)
        sidelip_angle = self.state_lat.getmatrix()[0][0] / 57.2958  # Sideslip angle in radians
        pitch_angle = -self.state_long.getmatrix()[2][0] / 57.2958  # Pitch angle in radians (note: negated)
        yaw_angle = self.state_lat.getmatrix()[2][0] / 57.2958      # Yaw angle in radians
        roll_angle = self.state_lat.getmatrix()[3][0] / 57.2958     # Roll angle in radians

        # Calculate velocity components
        # Adds cruise speed to current velocity perturbation (state-space models work with perturbations)
        cruise_speed = self.A.getmatrix()[1][2] / movment_scale # Extract cruise speed from A matrix (encapsulation)
        v = self.state_long.getmatrix()[0][0] + cruise_speed # Total airspeed

        # Decompose velocity into components using trigonometric relationships
        fv = v * cos(sidelip_angle)  # Forward velocity component
        vv = self.state_long.getmatrix()[1][0]  # Vertical velocity component (from angle of attack)
        hv = v * sin(sidelip_angle)  # Horizontal (sideways) velocity component

        # Update aircraft orientation using Euler angle transformations
        # These equations implement a simplified form of the rotation matrix transformations
        # Combines pitch and yaw effects based on current roll angle
        self.plane.rotation_x += 57.2958 * (pitch_angle * cos(roll_angle) + yaw_angle * sin(roll_angle)) * self.dt
        self.plane.rotation_y += 57.2958 * (pitch_angle * sin(roll_angle) + yaw_angle * cos(roll_angle)) * self.dt
        self.plane.rotation_z = 57.2958 * roll_angle  # Direct mapping for roll angle

        # Update aircraft position using velocity components in aircraft body axes
        # Multiply by time step (dt) and scaling factor to get position change
        # Uses Ursina's vector operations through forward, up, and right vectors
        self.plane.position += self.plane.forward * fv * self.dt * movment_scale
        self.plane.position += self.plane.up * vv * self.dt * movment_scale
        self.plane.position += self.plane.right * hv * self.dt * movment_scale
```

```
    def render_overlay(self):
        """
        Updates the UI elements based on current flight parameters.
        """
        # Elevator bar (original)
        normalized_elevator = -self.elevator / 10
        self.elevator_bar_fill.scale_y = 0.9 * abs(normalized_elevator)
        self.elevator_bar_fill.position = (0, 0.9 * normalized_elevator / 2)

        # Aileron bar
        normalized_aileron = -self.aileron / 10
        self.aileron_bar_fill.scale_y = 0.9 * abs(normalized_aileron)
        self.aileron_bar_fill.position = (0, 0.9 * normalized_aileron / 2)

        # Rudder bar
        normalized_rudder = -self.rudder / 10
        self.rudder_bar_fill.scale_y = 0.9 * abs(normalized_rudder)
        self.rudder_bar_fill.position = (0, 0.9 * normalized_rudder / 2)

        # Vertical velocity bar
        # Scale vertical velocity (state_long[1]) to a reasonable display range
        # Adjust divisor based on expected vv range
        if self.state_long.getmatrix()[1][0] < 0:
            normalized_vv = 1 / (-0.001 * self.state_long.getmatrix()[1][0] + 2) - 0.5
        else:
            normalized_vv = - 1 / (0.001 * self.state_long.getmatrix()[1][0] + 2) + 0.5
        self.vv_bar_fill.scale_y = 0.9 * abs(normalized_vv)
        self.vv_bar_fill.position = (0, 0.9 * normalized_vv / 2)

        # Calculate altitude (y position above ground level)
        altitude = self.plane.y - self.ground_y # Offset by ground level position

        # Add warning for low altitude (change text color when below 100m)
        if altitude < 75:
            self.altitude_text.color = color.red
        else:
            self.altitude_text.color = color.white

        self.altitude_text.text = f'Altitude: {int(altitude)} m'

    def update_data(self):
        """
        Updates data for postflight graphs.
        """
        self.time_data.append(self.flight_time)
        self.aoa_data.append(self.state_long.getmatrix()[1][0])  # Angle of Attack in degrees
        cruise_speed = self.A.getmatrix()[1][2]
        self.velocity_data.append(self.state_long.getmatrix()[0][0] + cruise_speed)  # Total velocity
        # Approximate G-force as vertical acceleration (dv/dt) / 9.81
        if len(self.velocity_data) > 1:
            dv_dt = (self.velocity_data[-1] - self.velocity_data[-2]) / self.dt
            g_force = dv_dt / 9.81  # Assuming vertical component dominates
        else:
            g_force = 0
        self.gforce_data.append(g_force)
        self.altitude_data.append(self.plane.y - self.ground_y)  # Altitude above ground
```

```
    def end_simulation(self):
        """
        Cleans up resources and transitions to post-flight analysis.
        """
        global flight_time, crashed

        flight_time = self.flight_time

        # Record crash status for post-flight analysis
        crashed = self.crashed

        # Store data for post-flight analysis
        self.flight_data = {
            'time': self.time_data,
            'aoa': self.aoa_data,
            'velocity': self.velocity_data,
            'gforce': self.gforce_data,
            'altitude': self.altitude_data}

        # Clean up all entities including menu elements and ground
        destroy(self.plane)
        destroy(self.sky)
        destroy(self.ground)

        # Clean up UI elements
        destroy(self.instructions)
        destroy(self.altitude_text)
        destroy(self.elevator_bar_bg)
        destroy(self.aileron_bar_bg)
        destroy(self.rudder_bar_bg)
        destroy(self.vv_bar_bg)
        destroy(self.menu_panel)
        destroy(self.input_handler)

        # Clean up crash-specific elements if they exist
        if self.crash_message:
            destroy(self.crash_message)
            destroy(self.continue_button)

        # Transition to post-flight analysis
        PostFlight(self.flight_data)
```

```
# ---------------------------
#   POST-FLIGHT ANALYTICS
# ---------------------------
class PostFlight(Entity):
    def __init__(self, flight_data = None):
        super().__init__()

        title_y = 0.4
        stats_y = 0.325
        graph_y = 0.05
        message_y = -0.225

        self.title = Text(text = 'Post-Flight Analytics',
                          position = (0, title_y),
                          scale = (2, 2),
                          origin = (0, 0))

        # Display flight time
        global flight_time, current_user, crashed
        minutes_to_add = flight_time / 60  # Convert seconds to minutes

        self.flight_time_text = Text(
            text = f'Flight Time: {flight_time:.2f} seconds',
            position = (0, stats_y), origin = (0, 0))

        # Initialize level-up related attributes
        self.level_up = False
        self.level_down = False
        self.new_level = current_user.level if current_user else 1
        self.new_total_minutes = 0

        # Update user's flight minutes and check for level up
        if current_user:
            user_manager = UserManager()

            # Update flight minutes
            self.new_total_minutes = user_manager.update_flight_minutes(current_user.user_id, minutes_to_add)
            # Total minutes needed for levels 1-10
            required_minutes = [0, 2, 4, 7, 10, 13, 16, 19, 22, 25]

            # Handle crash penalty
            if crashed and current_user.level > 1:
                self.level_down = True
                self.new_level = user_manager.update_level(current_user.user_id, current_user.level - 1)
            elif not crashed and minutes_to_add >=  1:  # Require at least 1 minute of flight time to level up
                # Check if total minutes qualify for the next level
                next_level = current_user.level + 1
                if next_level <= 10 and self.new_total_minutes >=  required_minutes[next_level - 1]:
                    self.level_up = True
                    self.new_level = user_manager.update_level(current_user.user_id, next_level)

            # Update current_user.level
            current_user.level = self.new_level
```

```
            # Display level information
            self.level_text = Text(
                text = f'Current Pilot Level: {current_user.level}',
                position = (0.5, stats_y),
                origin = (0, 0))

            self.total_time_text = Text(
                text = f'Total Flight Minutes: {self.new_total_minutes:.2f}',
                position = (-0.5, stats_y),
                origin = (0, 0))

            if self.level_up:
                self.level_up_text = Text(
                    text = f'CONGRATULATIONS! You\'ve reached Level {self.new_level}!',
                    position = (0, message_y), origin = (0, 0), color = color.yellow)

            elif self.level_down:
                self.level_down_text = Text(
                    text = f'Crash Detected! Level decreased to {self.new_level}',
                    position = (0, message_y), origin = (0, 0), color = color.red.tint(-0.2))

            elif current_user.level < 10:
                # Show progress to next level
                minutes_to_next = required_minutes[current_user.level] - self.new_total_minutes
                if minutes_to_next < 0:
                    minutes_to_next = 0
                self.progress_text = Text(
                    text = f'Minutes to next level: {minutes_to_next:.2f}',
                    position = (0, message_y), origin = (0, 0))

        # Navigation buttons
        self.menu_button = Button(text = 'Return to Main Menu', position = (0, -0.3), scale = (0.35, 0.05))
        self.menu_button.on_click = self.return_to_menu
        self.quit_button = Button(text = 'Quit', position = (0, -0.4), scale = (0.25, 0.05))
        self.quit_button.on_click = application.quit

        if flight_data:
            self.generate_graphs(flight_data, graph_y)
```

```
    def generate_graphs(self, flight_data, graph_y):
        """
        Generate four graphs side by side using Matplotlib and display them within Ursina UI.
        """
        graph_size = 1.7  # Graph display size in Ursina
        time = flight_data['time']

        plot_data = [
            (flight_data['aoa'], 'b-', 'Angle of Attack', 'AoA (deg)'),
            (flight_data['velocity'], 'g-', 'Velocity', 'Velocity (m/s)'),
            (flight_data['gforce'], 'r-', 'G-Force', 'G-Force (g)'),
            (flight_data['altitude'], 'm-', 'Altitude', 'Altitude (m)')] # Plot data: (data, color, title, y-label)

        fig, axs = pyplot.subplots(1, 4, figsize=(16, 4))  # 1 row, 4 columns
        fig.suptitle('Flight Performance Analysis', fontsize=16)

        # Configure each subplot
        for i in range(len(plot_data)):
            data, color, title, ylabel = plot_data[i]
            axs[i].plot(time, data, color, label=title)
            axs[i].set_title(title)
            axs[i].set_xlabel('Time (s)')
            axs[i].set_ylabel(ylabel)
            axs[i].grid(True)

        pyplot.tight_layout(rect=[0, 0, 1, 0.95])  # Adjust layout
        temp_file = 'temp_graphs.png'
        pyplot.savefig(temp_file, format='png', dpi=300)  # Save plot
        pyplot.close(fig)  # Free memory

        graph_texture = Texture(temp_file)  # Load as texture
        self.graph_entity = Entity(
            parent = camera.ui,
            model = 'quad',
            texture = graph_texture,
            scale = (graph_size, graph_size/4),  # Match figsize aspect ratio
            position = (0, graph_y),
            z = -1)  # Render behind UI elements

    def return_to_menu(self):
        destroy(self.title)
        destroy(self.flight_time_text)

        if current_user:    # Destroy level-related texts if they exist
            destroy(self.level_text)
            destroy(self.total_time_text)
            if self.level_up:
                destroy(self.level_up_text)
            elif self.level_down:
                destroy(self.level_down_text)
            else:
                destroy(self.progress_text)

        destroy(self.menu_button)
        destroy(self.quit_button)
        destroy(self.graph_entity)

        # Reset flight statistics for next session
        global flight_time, crashed
        flight_time = 0
        crashed = False
        MainMenu()
```

```
##########################################################################
######################### PROGRAM ENTRY ##################################
##########################################################################

if __name__ == '__main__':
    user_manager = UserManager()
    LoginScreen(user_manager = user_manager)
    app.run()
```

./matrix.py

Defines a Matrix class for linear algebra operations within flight physics calculations. Uses operator overloading to allow direct addition and multiplication of Matrices, along with determinants. Closely follows the OOP principle of encapsulation.

```
##############################################################
###################### MATRIX CLASS ##########################
##############################################################

class Matrix:
    """
    Custom Matrix implementation for linear algebra operations.
    Used for flight physics calculations (state-space model).
    Demonstrates OOP concepts like Encapsulation and Polymorphism.
    """
    def __init__(self, data):
        # Defensive Programming
        # data is expected as a 2D list (list of rows)
        if not isinstance(data, list) or len(data) == 0:
            raise ValueError("Input must be a non-empty 2D list")

        # Check if each element is a list
        for row in data:
            if not isinstance(row, list):
                raise ValueError("All elements must be lists")

        # Check if all rows have the same length
        width = len(data[0])
        for row in data:
            if len(row) != width:
                raise ValueError("All rows must have the same length")

        # Check if all elements are numbers
        for row in data:
            for x in row:
                if not isinstance(x, (int, float)):
                    raise ValueError("All elements must be numbers")

        self.data = data
        self.height = len(data)
        self.width = len(data[0]) if self.height > 0 else 0

    # Next 3 functions demonstrate the OOP principle of Encapsulation
    # by providing controlled access to private attributes

    def getmatrix(self):
        return self.data

    def getheight(self):
        return self.height

    def getwidth(self):
        return self.width
```

```
    def sub(self, x, y):
        """
        Creates a submatrix by removing specified row and column.
        Used in determinant calculation with the recursive minor method.
        """
        # Defensive Programming
        if not isinstance(y, int) or not isinstance(x, int):
            raise TypeError("Indices must be integers")
        if not (0 <= y < self.width and 0 <= x < self.height):
            raise IndexError("Indices out of bounds")
        r = []
        for i in range(self.height):
            if i != x:
                u = []
                for j in range(self.width):
                    if j != y:
                        u.append(self.data[i][j])
                r.append(u)
        return Matrix(r)

    def det(self):
        """
        Recursive determinant calculation using the Laplace expansion.
        Demonstrates advanced math concept - matrix determinants.
        Base case: 1x1 matrix determinant is the value itself.
        Recursive case: Sum of products of elements and cofactors.
        """
        if self.height != self.width:
            raise ValueError("Matrix must be square for determinant")
        if self.width == 1:
            return self.data[0][0]
        else:
            t = 0
            for i in range(self.height):
                # Calculate cofactor using submatrix
                submatrix = self.sub(0, i)
                # Laplace expansion formula: element * cofactor * sign

                # Debug Line Below
                # print(f't {t} += {self.data[0][i]} * det({submatrix}) * {((-1)**i)}')

                t += self.data[0][i] * submatrix.det() * ((-1)**i)
            return t
```

```
    # Operator overloading demonstrates OOP principle of Polymorphism
    # Allows matrices to be used with standard Python operators "+" and "*"
    def __add__(self, other):
        """
        Overloads + operator for matrix addition.
        Polymorphism - changing behavior of standard operator.
        """
        # Defensive Programming
        if not isinstance(other, Matrix):
            raise TypeError("Can only add Matrix with another Matrix")
        if self.height != other.height or self.width != other.width:
            raise ValueError("Matrices must have same dimensions for addition")
        result = []
        for i in range(self.height):
            row = []
            for j in range(self.width):
                row.append(self.data[i][j] + other.data[i][j])
            result.append(row)
        return Matrix(result)

    def __mul__(self, other):
        """
        Overloads * operator for matrix multiplication.
        Supports both matrix-matrix and matrix-scalar multiplication.
        Polymorphism - operator behavior changes based on operand type.
        """
        # If multiplying by another matrix:

        if isinstance(other, Matrix):
            # Defensive Programming
            if self.width != other.height:
                raise ValueError("Matrix dimensions incompatible for multiplication")
            result = []
            for i in range(self.height):
                row = []
                for j in range(other.width):
                    sum_val = 0
                    for k in range(self.width):
                        sum_val += self.data[i][k] * other.data[k][j]
                    row.append(sum_val)
                result.append(row)
            return Matrix(result)
        # Scalar multiplication
        elif isinstance(other, (int, float)):
            result = []
            for i in range(self.height):
                row = []
                for j in range(self.width):
                    row.append(self.data[i][j] * other)
                result.append(row)
            return Matrix(result)
        else:
            raise TypeError("Multiplication only supported with Matrix or scalar")

    def __rmul__(self, other):
        """
        Overloads right multiplication to support scalar * matrix syntax.
        Polymorphism - enables commutative property for scalar multiplication.
        """
        if not isinstance(other, (int, float)):
            raise TypeError("Right multiplication only supported with scalar")
        return self.__mul__(other)
```

```
    def __str__(self): # String representation of the matrix for debugging.
        return f'Matrix({self.data})'
```

./plane_management.py

Manages aircraft data in a SQLite database, handling plane attributes and file paths for 3D models and physics. Initialises database and fetches data when needed.

```
from sqlite3 import connect

##################################################################
#################### PLANE MANAGEMENT ############################
##################################################################

class PlaneManager:
    """
    Manages aircraft data and database operations.
    Demonstrates DAO pattern and file I/O for configuration.
    """
    def __init__(self, db_path = "planes.db"):
        self.conn = connect(db_path)
        self.cursor = self.conn.cursor()
        self.create_table()
        self.populate_planes()  # Populate planes right after creation

    def create_table(self):
        """
        Creates the planes table if it doesn't exist.
        Demonstrates SQL schema design with:
        - Primary key with auto-increment
        - Unique constraints
        - Multiple related file paths
        """
        self.cursor.execute("""
            CREATE TABLE IF NOT EXISTS planes (
                level INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT UNIQUE NOT NULL,
                obj_path TEXT NOT NULL,
                texture_path TEXT NOT NULL,
                description_path TEXT NOT NULL,
                matrix_path TEXT NOT NULL,
                thumbnail_path TEXT NOT NULL
            )""")
        self.conn.commit()

    def add_plane(self, name, obj_path, texture_path, description_path, matrix_path, thumbnail_path):
        """
        Adds a new plane to the database.
        Demonstrates the use of prepared statements for SQL injection prevention.
        """
        self.cursor.execute("""
            INSERT INTO planes (name, obj_path, texture_path, description_path, matrix_path, thumbnail_path)
            VALUES (?, ?, ?, ?, ?, ?)""", (name, obj_path, texture_path, description_path, matrix_path, thumbnail_path))
        self.conn.commit()

    def get_all_planes_info(self):
        """
        Retrieves all planes' information from the database.
        Returns a list of tuples containing plane data.
        """
        self.cursor.execute("SELECT level, name, thumbnail_path, description_path FROM planes ORDER BY level")
        return self.cursor.fetchall()

    def get_plane_physics(self, name):
        """
        Retrieves physics-related info of a specific plane by name.
        Used to load 3D model, texture, and flight dynamics matrices.
        """
        self.cursor.execute("SELECT obj_path, texture_path, matrix_path FROM planes WHERE name = ?", (name,))
        return self.cursor.fetchone()
```

```
    def populate_planes(self):
        """
        Populates the database with default planes if empty.
        Demonstrates batch data insertion and file path management.
        """
        self.cursor.execute("SELECT COUNT(*) FROM planes")
        if self.cursor.fetchone()[0] ==  0:
            planes = [
```

```
# Each tuple represents: (name, 3D model path, texture path, description file, matrix file, thumbnail)
("Cessna-172", "planes/Cessna-172/cessna.obj", "planes/Cessna-172/texture.png", "planes/Cessna-172/desc.txt", "planes/Cessna-172/matrix.txt", "planes/Cessna-172/thumbnail.jpg"),
("Boeing-737", "planes/Boeing-737/boeing.bam", "planes/Boeing-737/texture.png", "planes/Boeing-737/desc.txt", "planes/Boeing-737/matrix.txt", "planes/Boeing-737/thumbnail.jpg"),
("Spitfire",   "planes/Spitfire/spitfire.obj", "planes/Spitfire/texture.png",   "planes/Spitfire/desc.txt",   "planes/Spitfire/matrix.txt",   "planes/Spitfire/thumbnail.jpg"),
("ORCA",       "planes/ORCA/ORCA.bam",         "planes/ORCA/texture.png",       "planes/ORCA/desc.txt",       "planes/ORCA/matrix.txt",       "planes/ORCA/thumbnail.png"),
("X-Wing",     "planes/X-Wing/xwing.obj",      "planes/X-Wing/texture.png",     "planes/X-Wing/desc.txt",     "planes/X-Wing/matrix.txt",     "planes/X-Wing/thumbnail.jpg"),]
```

```
            for plane in planes:
                self.add_plane(plane[0], plane[1], plane[2], plane[3], plane[4], plane[5])
```

./security.py

Provides password security with salt generation and a custom hashing algorithm using several bitwise operations.

```python
from random import randint

############################################################
########### PASSWORD HASH AND SALTING FUNCTIONS ############
############################################################

def generate_salt():
    """
    Generates a random salt value for password hashing.
    Cybersecurity best practice: unique salt for each password.
    """
    return randint(31, 2**32 - 1)           # Generates a random salt in the 32-bit range

def hash_password(password, salt):
    """
    Custom password hashing algorithm implementation.
    Demonstrates cryptographic concepts for password security:
    - Salting: adds random value to prevent rainbow table attacks
    - Multiple transformations: bitwise XOR, binary shifts, and addition
    - Modulo prime: keeps hash value in manageable range
    """

    if not isinstance(password, str):
        raise TypeError("Password must be a string")
    if not isinstance(salt, int):
        raise TypeError("Salt must be an integer")
    if salt < 0:
        raise ValueError("Salt cannot be negative")

    hash_val = 867243217                    # Initial large "seed" prime
    for char in password:
        char = ord(char)                    # Convert from character to ASCII integer value
        hash_val = hash_val * 97            # Multiply by prime
        hash_val = hash_val ^ (char + 144479) # Imprint the ASCII value - Bitwise XOR
        hash_val = hash_val <<((char%5) + 1)  # Imprint the ASCII value - Binary shift
        hash_val = hash_val ^ salt          # Imprint the salt value - Bitwise XOR
        hash_val = hash_val + salt          # Imprint the salt value - ADD

    hash_val = hash_val % 387942806485727   # Mod by prime to ensures value fits in 63 bits
    return abs(hash_val)                    # Return the hash
```

./user_management.py

Implements user authentication and management with SQLite, utilising secure hashing algo. Allows uploading and fetching of user information. Initialises test user and database.

```
from sqlite3 import connect

from security import generate_salt, hash_password

################################################################
#################### USER MANAGEMENT ###########################
################################################################

class User:
    """
    Represents a user in the system.
    Demonstrates OOP concept of Abstraction by hiding complexity.
    """
    def __init__(self, user_id, username, password_hash, level = 1, flight_minutes = 0):

        # Defensive Programming

        if not isinstance(user_id, int) or user_id < 1:
            raise ValueError("User ID a must be a natural number")
        if not isinstance(username, str) or len(username) == 0:
            raise TypeError("Username must be a non-empty string")
        if not isinstance(password_hash, int) or user_id < 1:
            raise TypeError("Password hash must be a natural number")

        self.user_id = user_id
        self.username = username
        self.password_hash = password_hash
        self.level = level
        self.flight_minutes = max(0, flight_minutes)  # Ensure non-negative

class UserManager:
    """
    Manages user authentication and database operations.
    Demonstrates database integration with SQLite.
    Demonstrates SQL table creation, selection, and insertion.
    """
    def __init__(self, db_path = "users.db"):
        try:
            self.conn = connect(db_path)
            self.cursor = self.conn.cursor()
        except:
            raise RuntimeError("Failed to connect to database")
        self.create_table()
        self.add_default_user()

    def create_table(self):
        """
        Creates the users table if it doesn't exist.
        Demonstrates SQL schema design with:
        - Primary key with auto-increment
        - Unique constraints
        - Default values
        """
        self.cursor.execute("""
            CREATE TABLE IF NOT EXISTS users (
                user_id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT UNIQUE NOT NULL,
                password_hash INTEGER NOT NULL,
                salt INTEGER NOT NULL,
                level INTEGER DEFAULT 1,
                flight_minutes REAL DEFAULT 0)""")
        self.conn.commit()
```

```
    def add_default_user(self):
        """
        Adds a default test user if none exists.
        """
        username = "test"
        password = "password"
        salt = generate_salt()
        password_hash = hash_password(password, salt)

        # Check if user exists before inserting
        self.cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
        if not self.cursor.fetchone():
            self.cursor.execute("""
                                INSERT INTO users
                                    (username, password_hash, salt, level, flight_minutes)
                                VALUES (?, ?, ?, ?, ?)""", (username, password_hash, salt, 5, 12))
            self.conn.commit()

    def register_user(self, username, password):
        """
        Registers a new user with salted password hash.
        Returns True if successful, False if username exists.
        Demonstrates security best practices for password storage.
        """
        # Defensive Programming
        if not isinstance(username, str) or not isinstance(password, str) or len(username) == 0:
            raise TypeError("Username and password must be strings")
        self.cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
        if self.cursor.fetchone():
            return False

        salt = generate_salt()
        password_hash = hash_password(password, salt)

        self.cursor.execute("INSERT INTO users (username, password_hash, salt) VALUES (?, ?, ?)",
                            (username, password_hash, salt))
        self.conn.commit()
        return True  # Registration successful

    def login(self, username, password):
        """
        Authenticates a user by username and password.
        Returns User object if successful, None if failed.
        Demonstrates secure password verification technique.
        """
        self.cursor.execute('''
            SELECT user_id, username, password_hash, salt, level, flight_minutes FROM users
            WHERE username = ?''',
            (username,))
        user_data = self.cursor.fetchone()

        if user_data:
            user_id, username, stored_hash, salt, level, flight_minutes = user_data
            # Verify password by hashing with same salt and comparing
            if stored_hash ==  hash_password(password, salt):
                return User(user_id, username, stored_hash, level, flight_minutes)
        return None
```

```python
    def update_level(self, user_id, new_level):
        """
        Updates the user's level in the database.
        Ensures level stays within valid range (1-10).
        Returns the updated level.
        """
        # Ensure level is within bounds (1-10)
        # Defensive Programming
        if not isinstance(new_level, int):
            raise TypeError("Level must be an integer")
        if new_level < 1:
            new_level = 1
        elif new_level > 10:
            new_level = 10

        # Update the database
        self.cursor.execute("UPDATE users SET level = ? WHERE user_id = ?", (new_level, user_id))
        self.conn.commit()

        # Return the final level value
        return new_level

    def update_flight_minutes(self, user_id, minutes_to_add):
        """
        Updates the user's accumulated flight time in the database.
        Returns the new total flight minutes.
        """
        # Defensive Programming
        if not isinstance(minutes_to_add, (int, float)):
            raise TypeError("Minutes must be a number")

        # Fetch current flight minutes
        self.cursor.execute("SELECT flight_minutes FROM users WHERE user_id = ?", (user_id,))
        result = self.cursor.fetchone()  # Fetch only once and store the result

        # Defensive Programming
        if result is None:
            raise ValueError("User ID not found")

        # Calculate new total
        current_minutes = result[0]  # Access the first column of the result
        new_total = max(0, current_minutes + minutes_to_add)  # Ensure non-negative

        # Update the database
        self.cursor.execute("UPDATE users SET flight_minutes = ? WHERE user_id = ?", (new_total, user_id))
        self.conn.commit()

        # Return the new total
        return new_total
```

./matrix_tests.py

Tests Matrix class functionality, ensuring correct matrix operations and error handling for invalid inputs.

```
from matrix import Matrix

#  matrix tests
m1_ = Matrix([[1, 2], [3, 4]])
m2_ = Matrix([[5, 6], [7, 8]])

# Basic functionality tests
print(" Height OK" if m1_.getheight() == 2 else " Height FAIL")
print(" Width OK" if m1_.getwidth() == 2 else " Width FAIL")
print(" Data OK" if m1_.getmatrix() == [[1, 2], [3, 4]] else " Data FAIL")
print(" Add OK" if (m1_ + m2_).getmatrix() == [[6, 8], [10, 12]] else " Add FAIL")
print(" Mul OK" if (m1_ * m2_).getmatrix() == [[19, 22], [43, 50]] else " Mul FAIL")
print(" Scalar OK" if (m1_ * 2).getmatrix() == [[2, 4], [6, 8]] else " Scalar FAIL")
print(" Det OK" if m1_.det() == -2 else " Det FAIL")
print(" Sub OK" if m1_.sub(0, 0).getmatrix() == [[4]] else " Sub FAIL")
print()

# Error handling tests for
try:
    Matrix([])  # Empty list
    print(" Empty list: FAIL")
except ValueError:
    print(" Empty list: OK")

try:
    Matrix([[1, 2], [3]])  # Unequal rows
    print(" Unequal rows: FAIL")
except ValueError:
    print(" Unequal rows: OK")

try:
    Matrix([[1, "2"], [3, 4]])  # Non-numeric
    print(" Non-numeric: FAIL")
except ValueError:
    print(" Non-numeric: OK")

try:
    Matrix([1, 2])  # Non-2D list
    print(" Non-2D: FAIL")
except ValueError:
    print(" Non-2D: OK")

try:
    m1_.sub("0", 0)  # Invalid submatrix type
    print(" Sub bad type: FAIL")
except TypeError:
    print(" Sub bad type: OK")
```

```
try:
    m1_.sub(2, 0)  # Invalid submatrix index
    print(" Sub bad index: FAIL")
except IndexError:
    print(" Sub bad index: OK")

try:
    m1_ + 5  # Invalid addition type
    print(" Add bad type: FAIL")
except TypeError:
    print(" Add bad type: OK")

try:
    m1_ * "2"  # Invalid scalar type
    print(" Scalar bad type: FAIL")
except TypeError:
    print(" Scalar bad type: OK")

input()
```

./user_management_tests.py

Tests user management functionality, verifying registration, login, and data updates with error handling.

```
from user_management import UserManager, User
from random import randint

manager = UserManager()

# Helper function to print test results
def print_result(test_no, passed, details = ""):
    if passed:
        status = "PASSED"
    else:
        status = "FAILED"

    print(f"Test {test_no}: {status} - {details}")

# Test 1.1: Register new user
i = randint(1,100000)
un = 'newuser' + str(i)
result = manager.register_user(un, "Password123!")
passed = result is True
manager.cursor.execute("SELECT username FROM users WHERE username = ?", (un,))
exists = manager.cursor.fetchone() is not None
print_result("1.1", passed and exists, "Register new user")

# Test 1.2: Register existing user
result = manager.register_user("test", "Password123!")
print_result("1.2", result is False, "Register existing user 'test'")

# Test 1.3: Register with empty username
try:
    manager.register_user("", "Password123!")
    print_result("1.3", False, "Empty username should raise TypeError")
except TypeError:
    print_result("1.3", True, "Empty username raises TypeError")

# Test 1.4: Login with correct credentials
user = manager.login("test", "password")
passed = (user is not None and isinstance(user, User) and user.username == "test" and user.level == 5)
print_result("1.4", passed, "Login with correct credentials")

# Test 1.5: Login with incorrect password
user = manager.login("test", "wrongpass")
print_result("1.5", user is None, "Login with incorrect password")

# Test 1.6: Login with non-existent user
user = manager.login("nonuser", "password")
print_result("1.6", user is None, "Login with non-existent user")
```

```
# Test 1.7: Update flight minutes (positive)
manager.cursor.execute("SELECT flight_minutes FROM users WHERE user_id = ?", (1,))
initial = manager.cursor.fetchone()[0]
new_total = manager.update_flight_minutes(1, 5.0)
print_result("1.7", new_total == initial + 5.0, "Update flight minutes +5.0")

# Test 1.8: Update flight minutes (negative)
manager.cursor.execute("UPDATE users SET flight_minutes = ? WHERE user_id = ?", (10.0, 1))
manager.conn.commit()
new_total = manager.update_flight_minutes(1, -5.0)
print_result("1.8", new_total == 5.0, "Update flight minutes -5.0")

# Test 1.9: Update level to 3
new_level = manager.update_level(1, 3)
manager.cursor.execute("SELECT level FROM users WHERE user_id = ?", (1,))
db_level = manager.cursor.fetchone()[0]
print_result("1.9", new_level == 3 and db_level == 3, "Update level to 3")

# Test 1.10: Update level above max
new_level = manager.update_level(1, 11)
manager.cursor.execute("SELECT level FROM users WHERE user_id = ?", (1,))
db_level = manager.cursor.fetchone()[0]
print_result("1.10", new_level == 10 and db_level == 10, "Update level to 11 (capped at 10)")

# Test 1.11: Update level below min
new_level = manager.update_level(1, 0)
manager.cursor.execute("SELECT level FROM users WHERE user_id = ?", (1,))
db_level = manager.cursor.fetchone()[0]
print_result("1.11", new_level == 1 and db_level == 1, "Update level to 0 (floored at 1)")

# Test 1.12: Update level with non-integer
try:
    manager.update_level(1, "invalid")
    print_result("1.12", False, "Non-integer level should raise TypeError")
except TypeError:
    print_result("1.12", True, "Non-integer level raises TypeError")

# Reset level
new_level = manager.update_level(1, 5)

# Clean up
manager.conn.close()

input()
```

Testing

Overall Objective Map

Objective	Test Numbers	Page No.
2.3: Matrix Operations Support (Initialize, Add, Multiply, Scalar Multiply, Determinant)	2.1-2.16	76-81
4.1: Login with SQLite Authentication	1.4-1.6, 3.2-3.3, 8.3, 8.5	70-75, 82-85, 105-110
4.2: Create Account and Store Credentials Securely	1.1-1.3, 4.1-4.7, 8.2, 8.4	70-75, 86-90, 105-110
4.3: Track and Update Flight Minutes and Level	1.7-1.12, 7.8, 8.5	70-75, 100-104, 105-110
5.1: Login and Register Screen Transitions	3.1, 3.4-3.5, 4.8, 8.1, 8.3-8.4	82-85, 86-90, 105-110
5.2.1: Plane Selection Menu (Enable/Lock Based on Level)	5.1-5.2, 5.6-5.7, 8.2	91-95, 105-110
5.2.3: Exit Application from Main Menu	5.5	91-95
5.2.4: Start Simulation with Selected Plane	5.3-5.4, 8.1-8.3, 8.5	91-95, 105-110
5.3.1: Elevator Control with UI Update	6.1	96-99
5.3.2: Aileron Control with UI Update	6.2	96-99
5.3.3: Rudder Control with UI Update	6.3	96-99
5.3.4: Pause and Menu Toggle	6.5	96-99
5.4: Altitude Display with Color Change	6.4	96-99
5.5: Crash Detection and Transition	6.1, 6.4, 8.3	96-99, 105-110
6.1: Display Graphs and Update Level/Minutes	7.1-7.3, 8.1, 8.3	100-104, 105-110
6.2: Post-Flight Transitions and Exit	7.1, 7.4-7.5, 7.8, 8.1, 8.5	100-104, 105-110

1. User Management

This section involves testing the signing in/ signing up of users, as well as the updating of their Level and Flight Minutes.

Pre-testing changes

Original Code

```
def update_flight_minutes(self, user_id, minutes_to_add):
    """
    Updates the user's accumulated flight time in the database.
    Returns the new total flight minutes.
    """
    # Defensive Programming
    if not isinstance(minutes_to_add, (int, float)):
        raise TypeError("Minutes must be a number")
    # First, get current flight minutes
    self.cursor.execute("SELECT flight_minutes FROM users WHERE user_id = ?", (user_id,))
    print(self.cursor.fetchone()[0])
    if self.cursor.fetchone() is None:
        raise ValueError("User ID not found")
    current_minutes = self.cursor.fetchone()[0]

    # Calculate new total
    new_total = current_minutes + minutes_to_add

    # Update the database
    self.cursor.execute("UPDATE users SET flight_minutes = ? WHERE user_id = ?", (new_total, user_id))
    self.conn.commit()

    # Return the new total
    return new_total
```

Error

new_total = current_minutes + minutes_to_add Was adding an integer and a NoneType

Reasoning

self.cursor.fetchone() doesn't return the same value every time it's called, as it moves the cursor by 1 every time it's called, hence the 1st call was successful but none after that.

New Code

```
def update_flight_minutes(self, user_id, minutes_to_add):
    """
    Updates the user's accumulated flight time in the database.
    Returns the new total flight minutes.
    """
    # Defensive Programming
    if not isinstance(minutes_to_add, (int, float)):
        raise TypeError("Minutes must be a number")

    # Fetch current flight minutes
    self.cursor.execute("SELECT flight_minutes FROM users WHERE user_id = ?", (user_id,))
    result = self.cursor.fetchone()  # Fetch only once and store the result

    # Defensive Programming
    if result is None:
        raise ValueError("User ID not found")

    # Calculate new total
    current_minutes = result[0]  # Access the first column of the result
    new_total = max(0, current_minutes + minutes_to_add)  # Ensure non-negative

    # Update the database
    self.cursor.execute("UPDATE users SET flight_minutes = ? WHERE user_id = ?", (new_total, user_id))
    self.conn.commit()

    # Return the new total
    return new total
```

Code

The program below tests all the relevant functions on various usernames, passwords, and other inputs and outputs results.

```
import sys
import os
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from user_management import UserManager, User

manager = UserManager()

# Helper function to print test results
def print_result(test_no, passed, details = ""):
    if passed:
        status = "PASSED"
    else:
        status = "FAILED"

    print(f"Test {test_no}: {status} - {details}")

# Test 1.1: Register new user
result = manager.register_user("newuser", "Password123!")
passed = result is True
manager.cursor.execute("SELECT username FROM users WHERE username = ?", ("newuser",))
exists = manager.cursor.fetchone() is not None
print_result("1.1", passed and exists, "Register new user")

# Test 1.2: Register existing user
result = manager.register_user("test", "Password123!")
print_result("1.2", result is False, "Register existing user 'test'")

# Test 1.3: Register with empty username
try:
    manager.register_user("", "Password123!")
    print_result("1.3", False, "Empty username should raise TypeError")
except TypeError:
    print_result("1.3", True, "Empty username raises TypeError")

# Test 1.4: Login with correct credentials
user = manager.login("test", "password")
passed = (user is not None and isinstance(user, User) and user.username == "test" and user.level == 5)
print_result("1.4", passed, "Login with correct credentials")

# Test 1.5: Login with incorrect password
user = manager.login("test", "wrongpass")
print_result("1.5", user is None, "Login with incorrect password")
```

```
# Test 1.6: Login with non-existent user
user = manager.login("nonuser", "password")
print_result("1.6", user is None, "Login with non-existent user")

# Test 1.7: Update flight minutes (positive)
manager.cursor.execute("SELECT flight_minutes FROM users WHERE user_id = ?", (1,))
initial = manager.cursor.fetchone()[0]
new_total = manager.update_flight_minutes(1, 5.0)
print_result("1.7", new_total == initial + 5.0, "Update flight minutes +5.0")

# Test 1.8: Update flight minutes (negative)
manager.cursor.execute("UPDATE users SET flight_minutes = ? WHERE user_id = ?", (10.0, 1))
manager.conn.commit()
new_total = manager.update_flight_minutes(1, -5.0)
print_result("1.8", new_total == 5.0, "Update flight minutes -5.0")

# Test 1.9: Update level to 3
new_level = manager.update_level(1, 3)
manager.cursor.execute("SELECT level FROM users WHERE user_id = ?", (1,))
db_level = manager.cursor.fetchone()[0]
print_result("1.9", new_level == 3 and db_level == 3, "Update level to 3")

# Test 1.10: Update level above max
new_level = manager.update_level(1, 11)
manager.cursor.execute("SELECT level FROM users WHERE user_id = ?", (1,))
db_level = manager.cursor.fetchone()[0]
print_result("1.10", new_level == 10 and db_level == 10, "Update level to 11 (capped at 10)")

# Test 1.11: Update level below min
new_level = manager.update_level(1, 0)
manager.cursor.execute("SELECT level FROM users WHERE user_id = ?", (1,))
db_level = manager.cursor.fetchone()[0]
print_result("1.11", new_level == 1 and db_level == 1, "Update level to 0 (floored at 1)")

# Test 1.12: Update level with non-integer
try:
    manager.update_level(1, "invalid")
    print_result("1.12", False, "Non-integer level should raise TypeError")
except TypeError:
    print_result("1.12", True, "Non-integer level raises TypeError")

# Clean up
manager.conn.close()
input()
```

Results

```
Test 1.1: PASSED - Register new user
Test 1.2: PASSED - Register existing user 'test'
Test 1.3: FAILED - Empty username should raise TypeError
Test 1.4: PASSED - Login with correct credentials
Test 1.5: PASSED - Login with incorrect password
Test 1.6: PASSED - Login with non-existent user
Test 1.7: PASSED - Update flight minutes +5.0
Test 1.8: PASSED - Update flight minutes -5.0
Test 1.9: PASSED - Update level to 3
Test 1.10: PASSED - Update level to 11 (capped at 10)
Test 1.11: PASSED - Update level to 0 (floored at 1)
Test 1.12: PASSED - Non-integer level raises TypeError
```

Objective Map

Test No.	Corresponding Objective	Objective Number
1.1	Create account, store credentials securely	4.2
1.2	Create account (handling duplicate usernames securely)	4.2
1.3	Create account (validate input securely)	4.2
1.4	Login with SQLite authentication	4.1
1.5	Login (handle incorrect credentials)	4.1
1.6	Login (handle non-existent users)	4.1
1.7	Track and update flight minutes	4.3
1.8	Track flight minutes (handle invalid updates)	4.3
1.9	Track and update level	4.3
1.10 - 1.11	Track level (enforce valid range)	4.3
1.12	Track level (validate input)	4.3

Tests

Test No.	Input	Expected Outcome	Type	Result
1.1	Register user with username="newuser", password="Password123!"	Registration successful, returns True, user added to database	Normal	Pass
1.2	Register user with existing username="test", password="Password123!"	Registration fails, returns False	Erroneous	Pass
1.3	Register user with empty username="", password="Password123!"	Raises TypeError or validation error	Boundary	Fail
1.4	Login with username="test", password="password"	Returns User object with correct user_id, username, level=5	Normal	Pass
1.5	Login with username="test", incorrect password="wrongpass"	Returns None	Erroneous	Pass

1.6	Login with non-existent username="nonuser", password="password"	Returns None	Erroneous	Pass
1.7	Update flight minutes for user_id=1, minutes_to_add=5.0	Flight minutes updated to previous value + 5.0, returns new total	Normal	Pass
1.8	Update flight minutes with negative minutes_to_add=-5.0	Flight minutes remain non-negative, returns current total	Boundary	Pass
1.9	Update level for user_id=1, new_level=3	Level updated to 3, returns 3	Normal	Pass
1.10	Update level with new_level=11	Level capped at 10, returns 10	Boundary	Pass
1.11	Update level with new_level=0	Level floored at 1, returns 1	Boundary	Pass
1.12	Update level with non-integer new_level="invalid"	Raises TypeError	Erroneous	Pass

Fixes

1.3 - Register user with empty username="", password="Password123!"

The program was making sure that the username was a string, but didn't check for an empty string. To fix this, I simply added a condition on the string's length:

Before	After
`if not isinstance(username, str):` `  raise TypeError("Username must be a string")`	`if not isinstance(username, str) or len(username) == 0:` `  raise TypeError("Username must be a non-empty string")`

Rerunning the testing code, all tests were passed:

```
Test 1.1: PASSED - Register new user
Test 1.2: PASSED - Register existing user 'test'
Test 1.3: PASSED - Empty username raises TypeError
Test 1.4: PASSED - Login with correct credentials
Test 1.5: PASSED - Login with incorrect password
Test 1.6: PASSED - Login with non-existent user
Test 1.7: PASSED - Update flight minutes +5.0
Test 1.8: PASSED - Update flight minutes -5.0
Test 1.9: PASSED - Update level to 3
Test 1.10: PASSED - Update level to 11 (capped at 10)
Test 1.11: PASSED - Update level to 0 (floored at 1)
Test 1.12: PASSED - Non-integer level raises TypeError
```

2. Matrix

I will have to test the matrix module and all the functions in it. The program below tests all the relevant matrix functions on a few test matrices and outputs an error if any tests fail.

Code

```
from matrix import Matrix

#  matrix tests
ml_ = Matrix([[1, 2], [3, 4]])
m2_ = Matrix([[5, 6], [7, 8]])

# Basic functionality tests
print(" Height OK" if ml_.getheight() == 2 else " Height FAIL")
print(" Width OK" if ml_.getwidth() == 2 else " Width FAIL")
print(" Data OK" if ml_.getmatrix() == [[1, 2], [3, 4]] else " Data FAIL")
print(" Add OK" if (ml_ + m2_).getmatrix() == [[6, 8], [10, 12]] else " Add FAIL")
print(" Mul OK" if (ml_ * m2_).getmatrix() == [[19, 22], [43, 50]] else " Mul FAIL")
print(" Scalar OK" if (ml_ * 2).getmatrix() == [[2, 4], [6, 8]] else " Scalar FAIL")
print(" Det OK" if ml_.det() == -2 else " Det FAIL")
print(" Sub OK" if ml_.sub(0, 0).getmatrix() == [[4]] else " Sub FAIL")
print()

# Error handling tests for
try:
    Matrix([])  # Empty list
    print(" Empty list: FAIL")
except ValueError:
    print(" Empty list: OK")

try:
    Matrix([[1, 2], [3]])  # Unequal rows
    print(" Unequal rows: FAIL")
except ValueError:
    print(" Unequal rows: OK")

try:
    Matrix([[1, "2"], [3, 4]])  # Non-numeric
    print(" Non-numeric: FAIL")
except ValueError:
    print(" Non-numeric: OK")

try:
    Matrix([1, 2])  # Non-2D list
    print(" Non-2D: FAIL")
except ValueError:
    print(" Non-2D: OK")

try:
    ml_.sub("0", 0)  # Invalid submatrix type
    print(" Sub bad type: FAIL")
except TypeError:
    print(" Sub bad type: OK")

try:
    ml_.sub(2, 0)  # Invalid submatrix index
    print(" Sub bad index: FAIL")
except IndexError:
    print(" Sub bad index: OK")
```

```
try:
    m1_ + 5  # Invalid addition type
    print(" Add bad type: FAIL")
except TypeError:
    print(" Add bad type: OK")

try:
    m1_ * "2"  # Invalid scalar type
    print(" Scalar bad type: FAIL")
except TypeError:
    print(" Scalar bad type: OK")
```

Results

```
\matrix_tests.py
 Height OK
 Width OK
 Data OK
 Add OK
 Mul OK
 Scalar OK
 Det FAIL
 Sub OK

 Empty list: OK
 Unequal rows: OK
 Non-numeric: OK
 Non-2D: OK
 Sub bad type: OK
 Sub bad index: OK
 Add bad type: OK
 Scalar bad type: OK
```

Objective Map

Test No.	Corresponding Objective	Objective Number
2.1 - 2.3	Matrix Operations Support (Initialize matrix with correct height)	2.3 (General)
2.4	Matrix Addition	2.3.3
2.5	Matrix Multiplication	2.3.2
2.6	Scalar Multiplication	2.3.1
2.7	Matrix Determinant	2.3.4
2.8	Matrix Operations Support (Create submatrix)	2.3 (General)
2.9	Matrix Operations Support (Handle empty list initialization)	2.3 (General)
2.10	Matrix Operations Support (Handle uneven rows initialization)	2.3 (General)
2.11	Matrix Operations Support (Handle non-numeric data initialization)	2.3 (General)
2.12	Matrix Operations Support (Handle non-2D list initialization)	2.3 (General)
2.13	Matrix Operations Support (Handle invalid row index for submatrix)	2.3 (General)
2.14	Matrix Operations Support (Handle out-of-bounds row index for submatrix)	2.3 (General)
2.15	Matrix Addition (Handle non-matrix input)	2.3.3
2.16	Scalar Multiplication (Handle non-numeric scalar)	2.3.1

Tests

Test No.	Input	Expected Outcome	Type	Result
2.1	Initialize Matrix with [[1, 2], [3, 4]]	Matrix has height=2	Normal	Pass
2.2	Initialize Matrix with [[1, 2], [3, 4]]	Matrix has width=2	Normal	Pass

2.3	Initialize Matrix with [[1, 2], [3, 4]]	Matrix data is [[1, 2], [3, 4]]	Normal	Pass
2.4	Add two matrices: [[1, 2], [3, 4]] + [[5, 6], [7, 8]]	Returns Matrix with data [[6, 8], [10, 12]]	Normal	Pass
2.5	Multiply two matrices: [[1, 2], [3, 4]] * [[5, 6], [7, 8]]	Returns Matrix with data [[19, 22], [43, 50]]	Normal	Pass
2.6	Multiply matrix by scalar: [[1, 2], [3, 4]] * 2	Returns Matrix with data [[2, 4], [6, 8]]	Normal	Pass
2.7	Calculate determinant of [[1, 2], [3, 4]]	Returns -2	Normal	Fail
2.8	Create submatrix of [[1, 2], [3, 4]], remove row 0, col 0	Returns Matrix with data [[4]]	Normal	Pass
2.9	Initialize Matrix with empty list []	Raises ValueError	Boundary	Pass
2.10	Initialize Matrix with uneven rows [[1, 2], [3]]	Raises ValueError	Boundary	Pass
2.11	Initialize Matrix with non-numeric data [[1, "2"], [3, 4]]	Raises ValueError	Erroneous	Pass
2.12	Initialize Matrix with non-2D list [1, 2]	Raises ValueError	Erroneous	Pass
2.13	Create submatrix of [[1, 2], [3, 4]] with invalid row index "0", col 0	Raises TypeError	Erroneous	Pass
2.14	Create submatrix of [[1, 2], [3, 4]] with invalid row index 2, col 0	Raises IndexError	Boundary	Pass
2.15	Add matrix [[1, 2], [3, 4]] to non-matrix 5	Raises TypeError	Erroneous	Pass
2.16	Multiply matrix [[1, 2], [3, 4]] by non-numeric "2"	Raises TypeError	Erroneous	Pass

Fixes

2.7 - Calculate determinant of [[1, 2], [3, 4]]

The program was meant to calculate a value of -2 through the operations 1*4*1 + 2*3*-1, but instead of extracting the submatrix without 2, it was extracting the submatrix with 2. To fix this, I simply had to switch the x and y variable positions, as shown below:

Before	After
`def sub(self, y, x):`	`def sub(self, x, y):`

Rerunning the testing code, all tests were passed:

```
Height OK
Width OK
Data OK
Add OK
Mul OK
Scalar OK
Det OK
Sub OK
```

```
Empty list: OK
Unequal rows: OK
Non-numeric: OK
Non-2D: OK
Sub bad type: OK
Sub bad index: OK
Add bad type: OK
Scalar bad type: OK
```

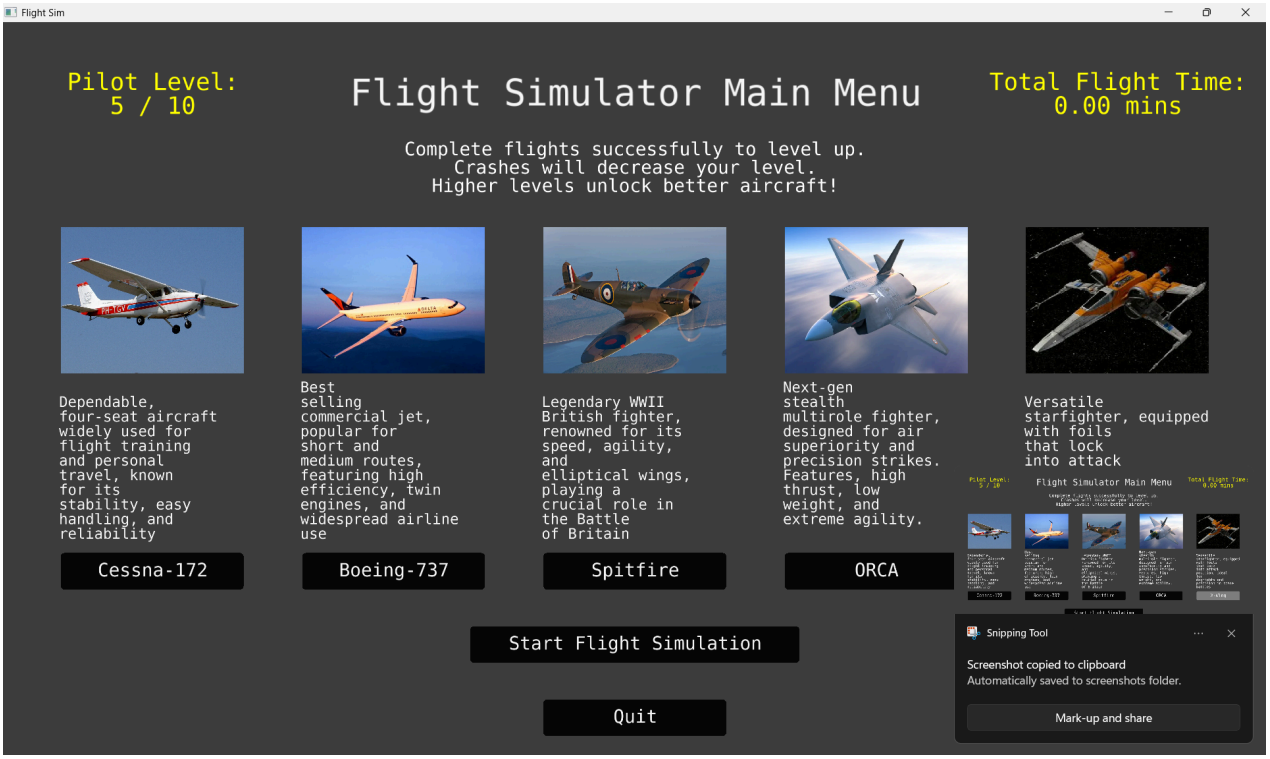
3. Login Screen

Objective Map

Test No.	Corresponding Objective	Objective Number
3.1	Login screen transitions to main menu upon successful authentication	5.1
3.2	Login screen handles incorrect credentials with error message	4.1
3.3	Login screen handles empty credentials with error message	4.1
3.4	Login screen allows transition to registration screen	5.1
3.5	Login screen handles multiple login attempts with single successful transition	5.1

Tests

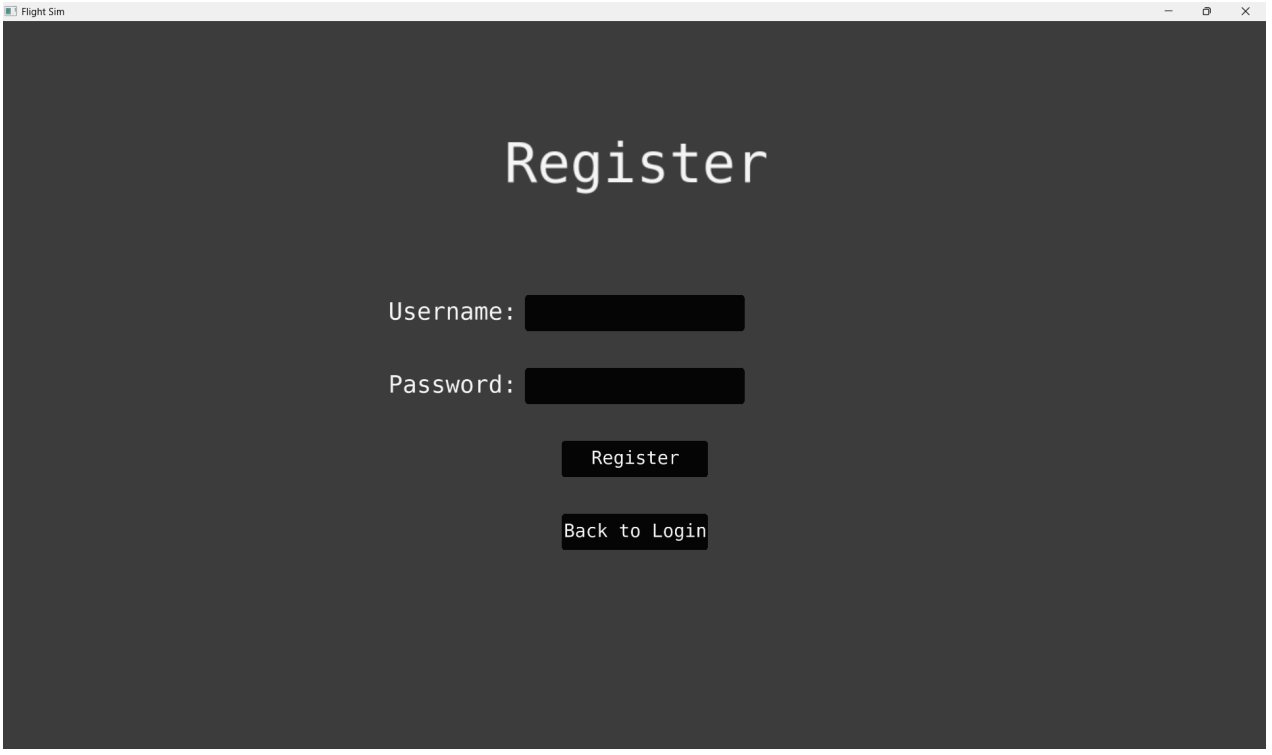
Test No.	Input	Expected Outcome	Type	Result
3.1	Enter username="test", password="password", click Login	Transitions to MainMenu	Normal	Pass



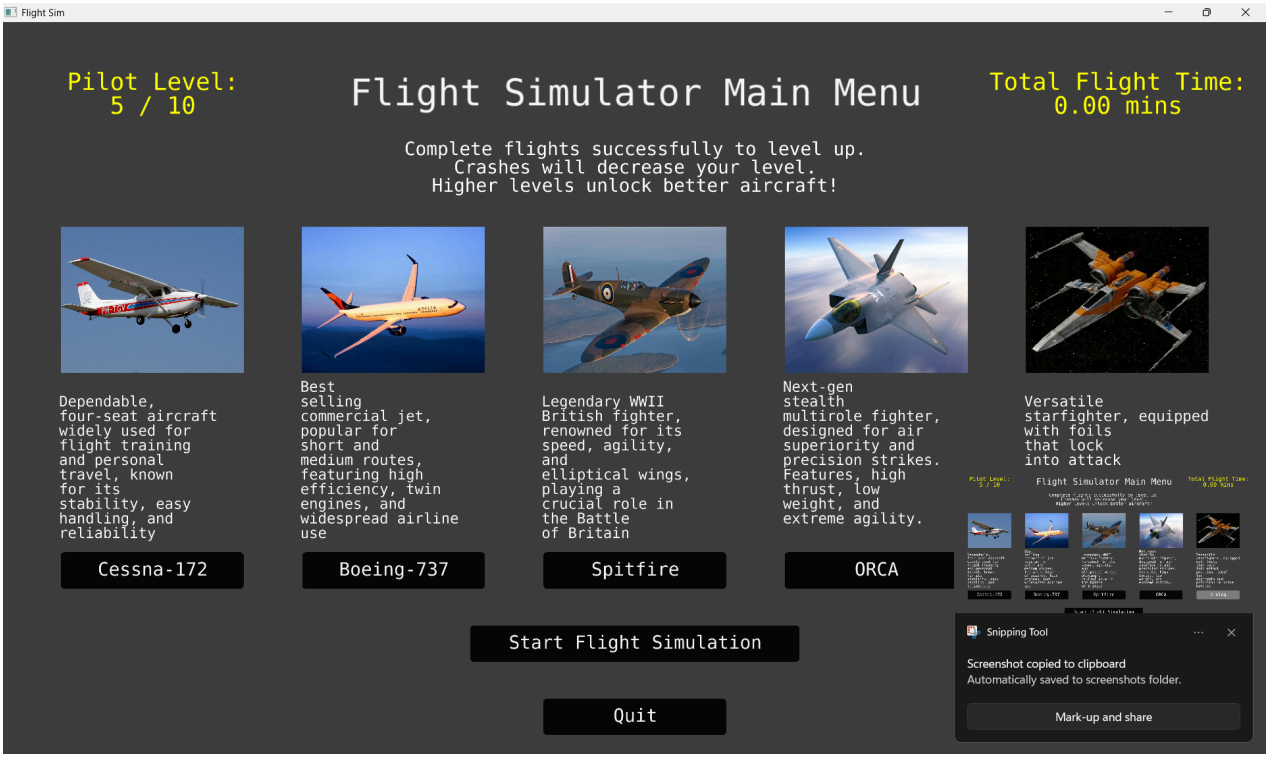
3.2	Enter username="test", password="wrongpass", click Login	Displays "Login failed. Please try again."	Erroneous	Pass

3.3	Enter username="", password="", click Login	Displays "Login failed. Please try again."	Boundary	Pass

3.4	Click Register button	Transitions to RegisterScreen, LoginScreen UI cleared	Normal	Pass



3.5	Rapidly click Login multiple times with valid credentials	Single successful login, transitions to MainMenu once	Boundary	Pass



All tests were successful and the Login Screen works completely correctly.

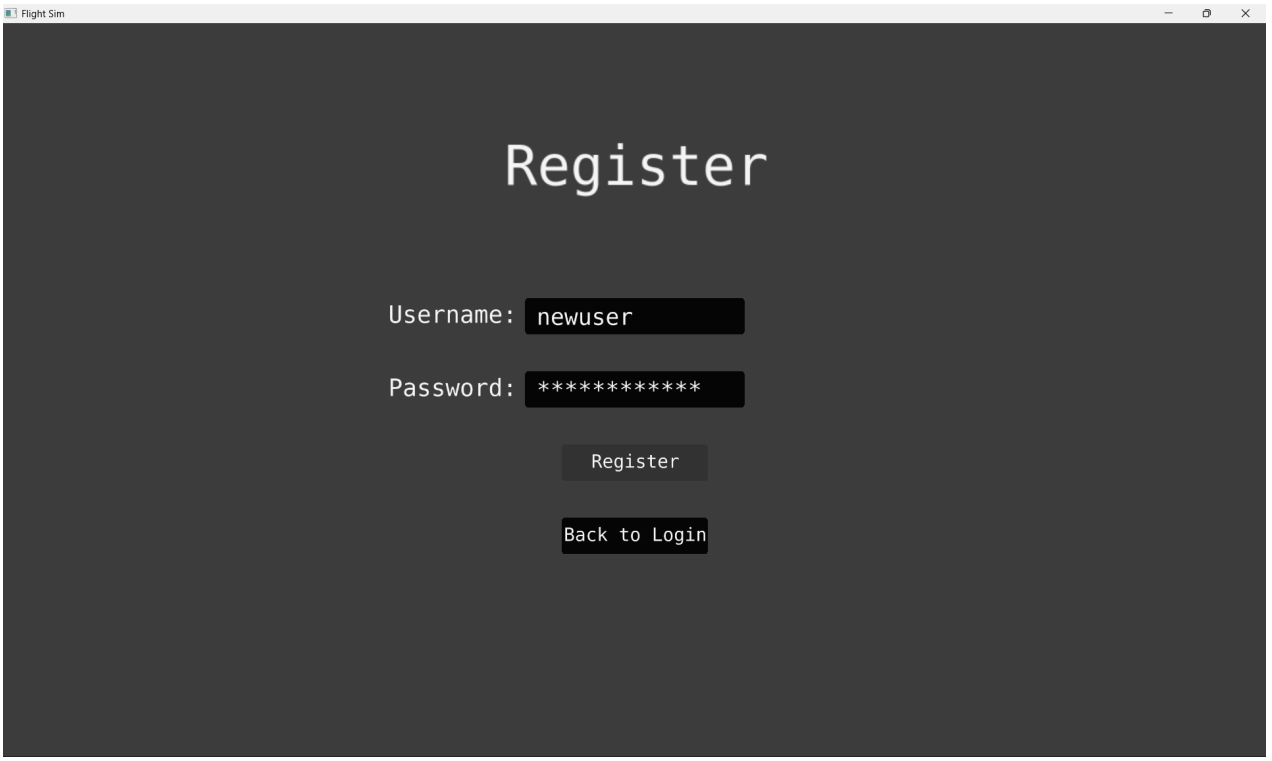
4. Register Screen

Objective Map

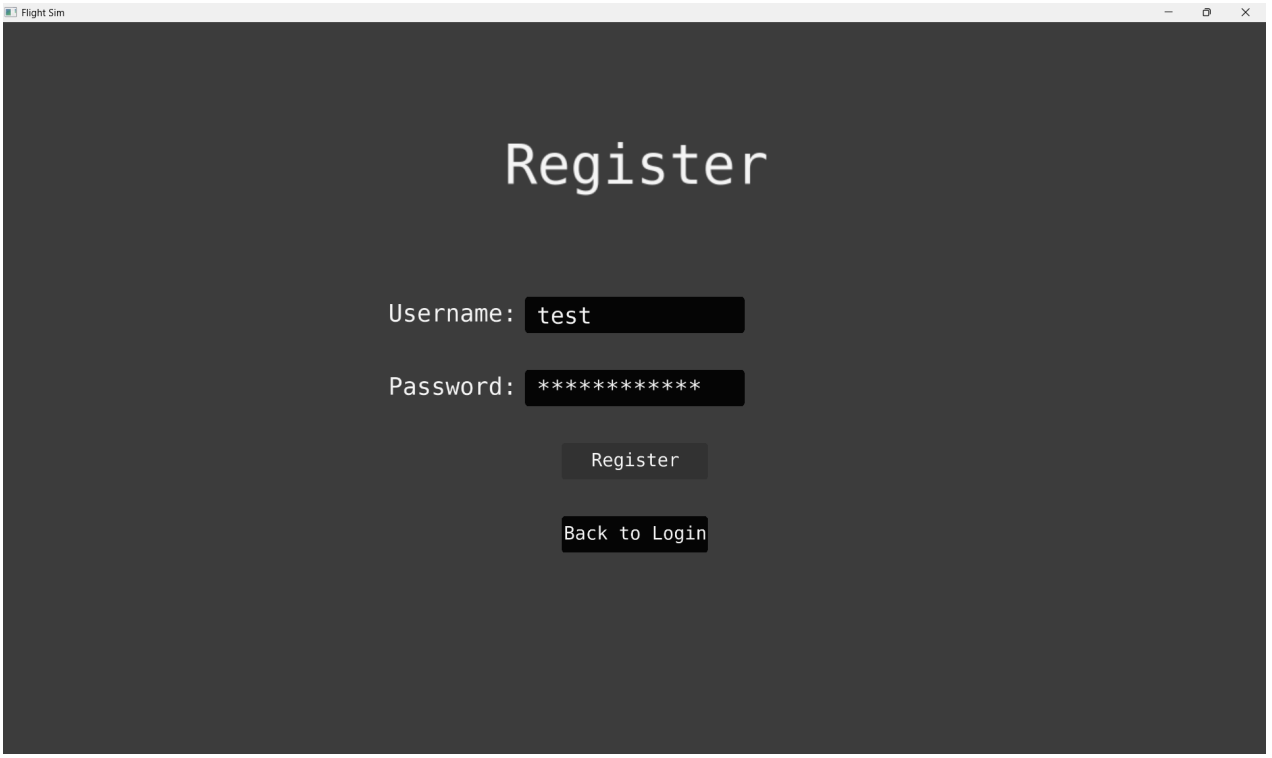
Test No.	Corresponding Objective	Objective Number
2.1	Matrix Operations Support (Initialize matrix with correct height)	2.3 (General)
2.2	Matrix Operations Support (Initialize matrix with correct width)	2.3 (General)
2.3	Matrix Operations Support (Initialize matrix with correct data)	2.3 (General)
2.4	Matrix Addition	2.3.3
2.5	Matrix Multiplication	2.3.2
2.6	Scalar Multiplication	2.3.1
2.7	Matrix Determinant	2.3.4
2.8	Matrix Operations Support (Create submatrix)	2.3 (General)
2.9	Matrix Operations Support (Handle empty list initialization)	2.3 (General)
2.10	Matrix Operations Support (Handle uneven rows initialization)	2.3 (General)
2.11	Matrix Operations Support (Handle non-numeric data initialization)	2.3 (General)
2.12	Matrix Operations Support (Handle non-2D list initialization)	2.3 (General)
2.13	Matrix Operations Support (Handle invalid row index for submatrix)	2.3 (General)
2.14	Matrix Operations Support (Handle out-of-bounds row index for submatrix)	2.3 (General)
2.15	Matrix Addition (Handle non-matrix input)	2.3.3
2.16	Scalar Multiplication (Handle non-numeric scalar)	2.3.1

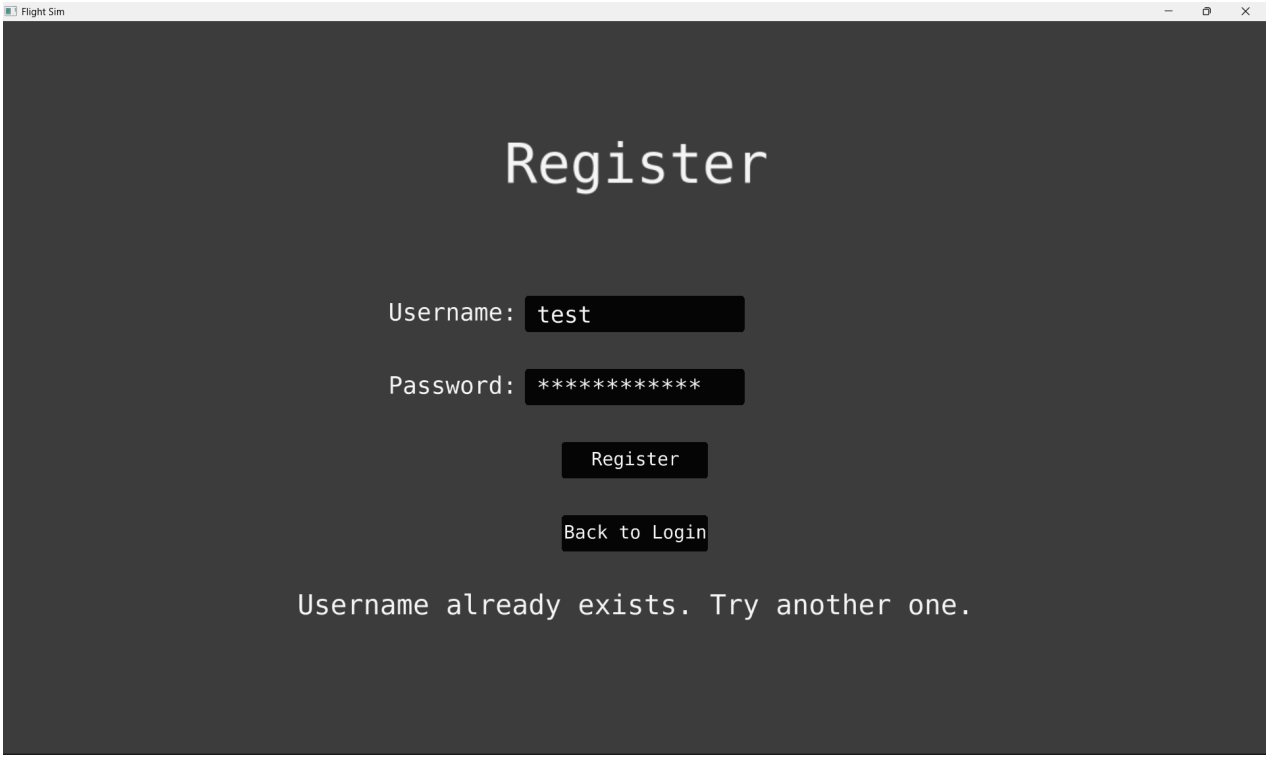
Tests

Test No.	Input	Expected Outcome	Type	Result
4.1	Username="newuser", password="Password123!", click Register	Displays "Registration successful!", returns to LoginScreen after 1 second	Normal	Pass

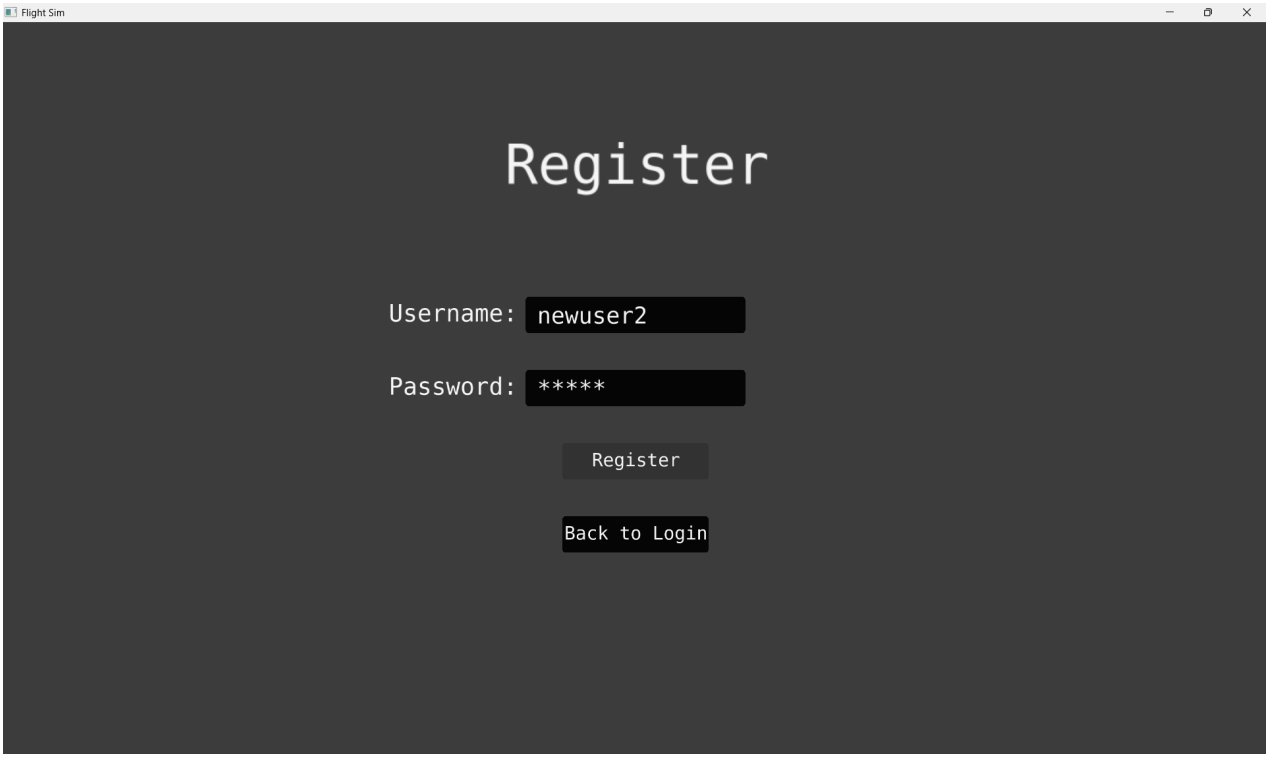


4.2	Username="test", password="Password123!" , click Register	Displays "Username already exists. Try another one."	Erroneous	Pass

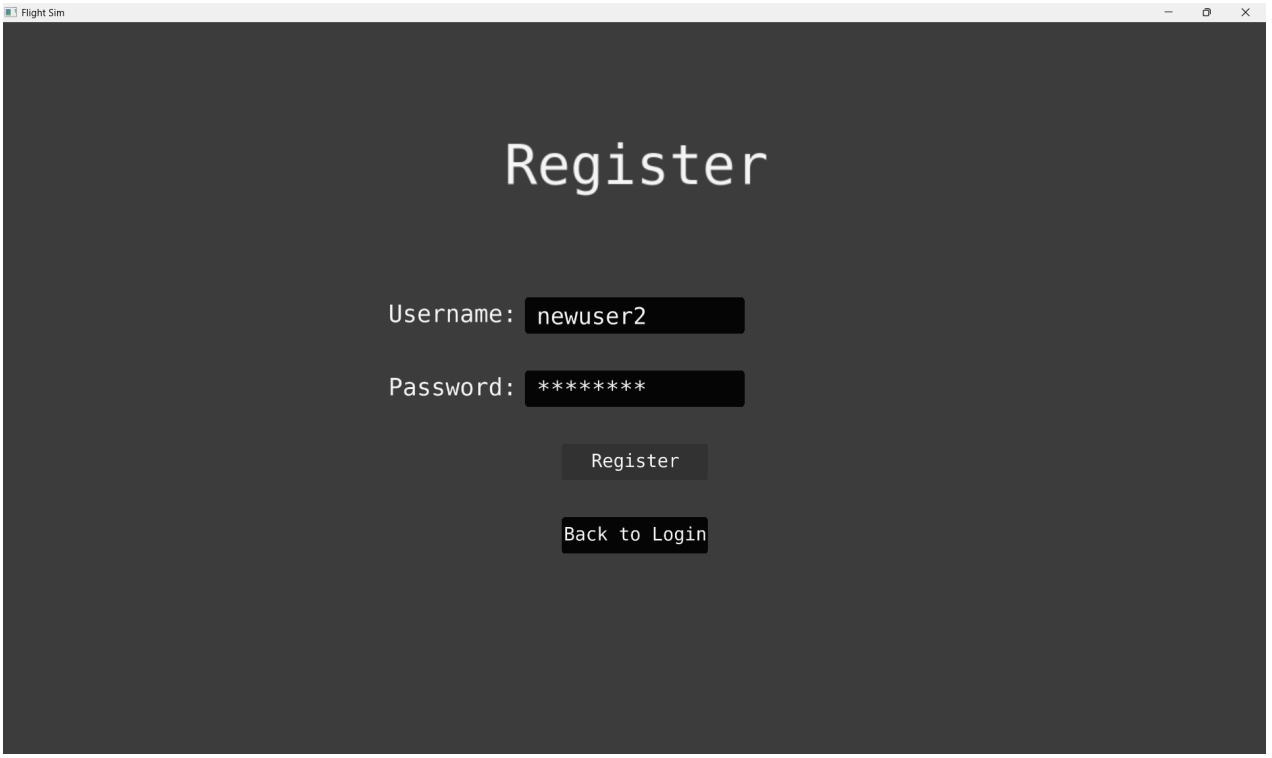


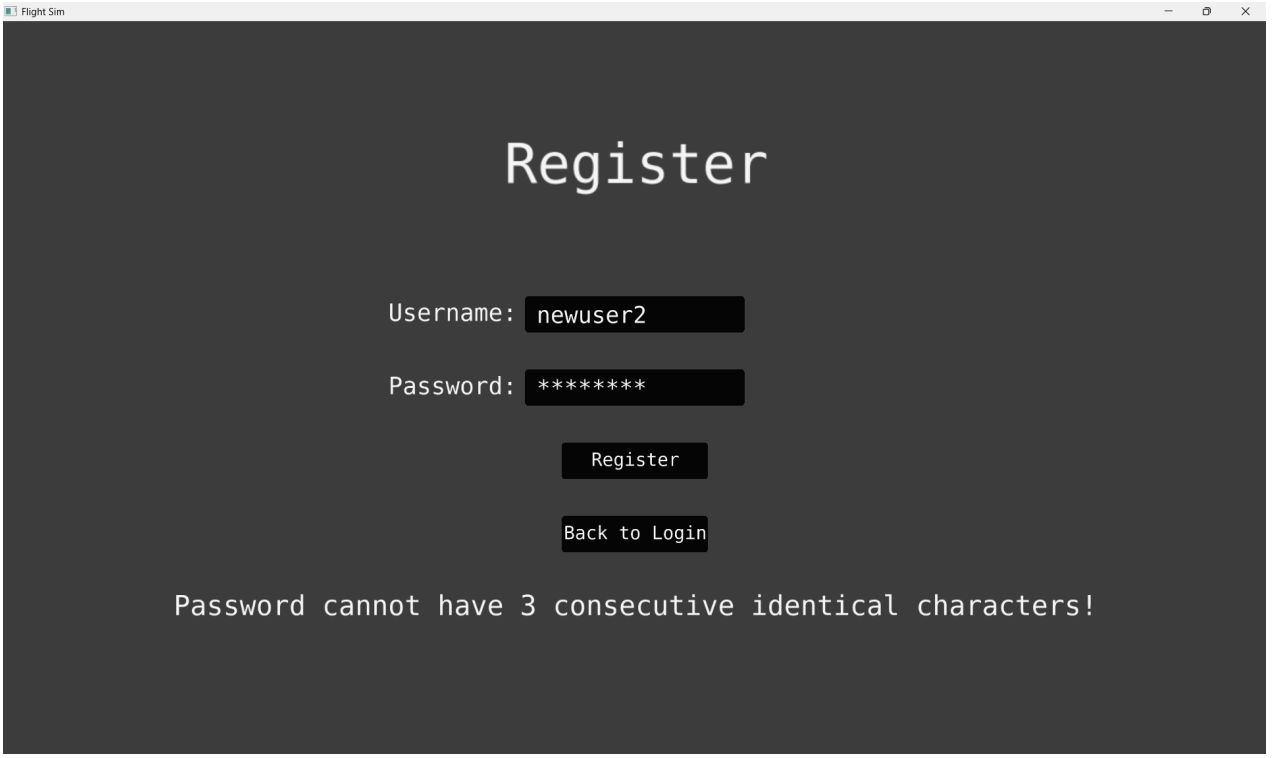


4.3	Username="newuser2", password="short", click Register	Displays "Password must be at least 8 characters long!"	Boundary	Pass

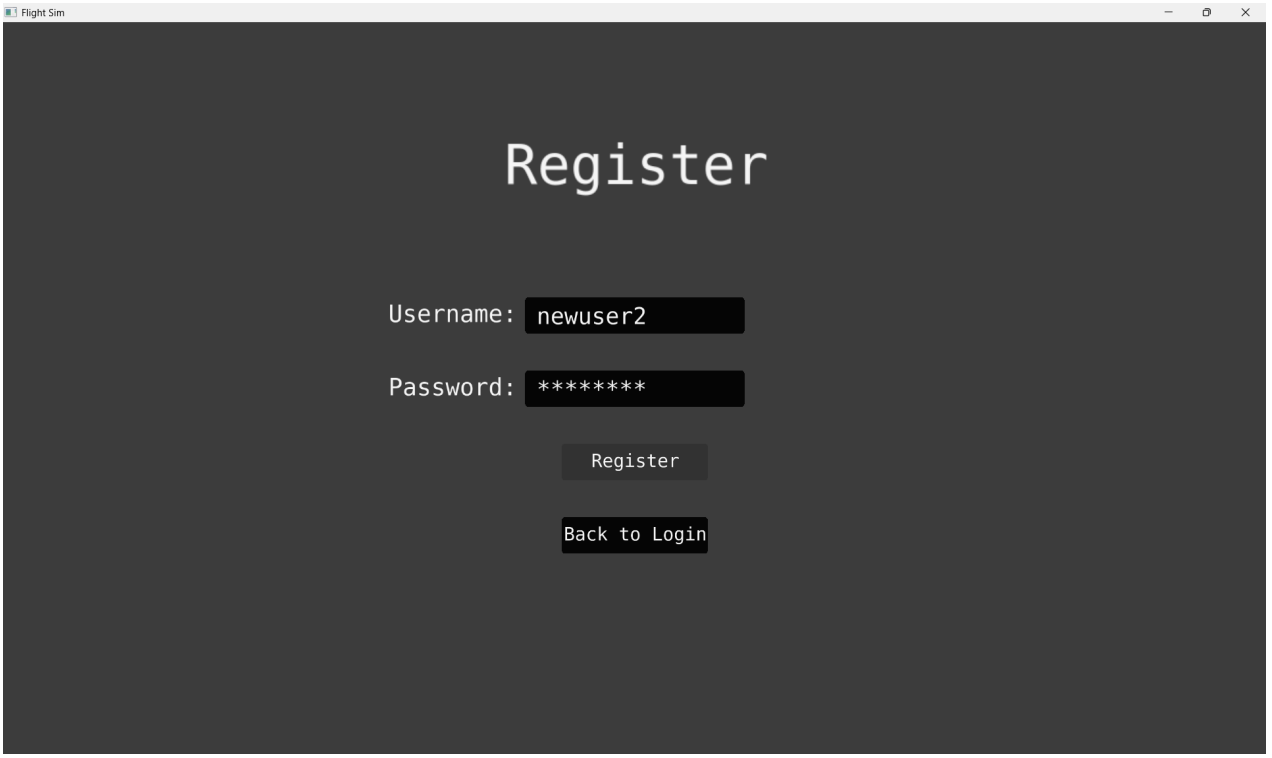


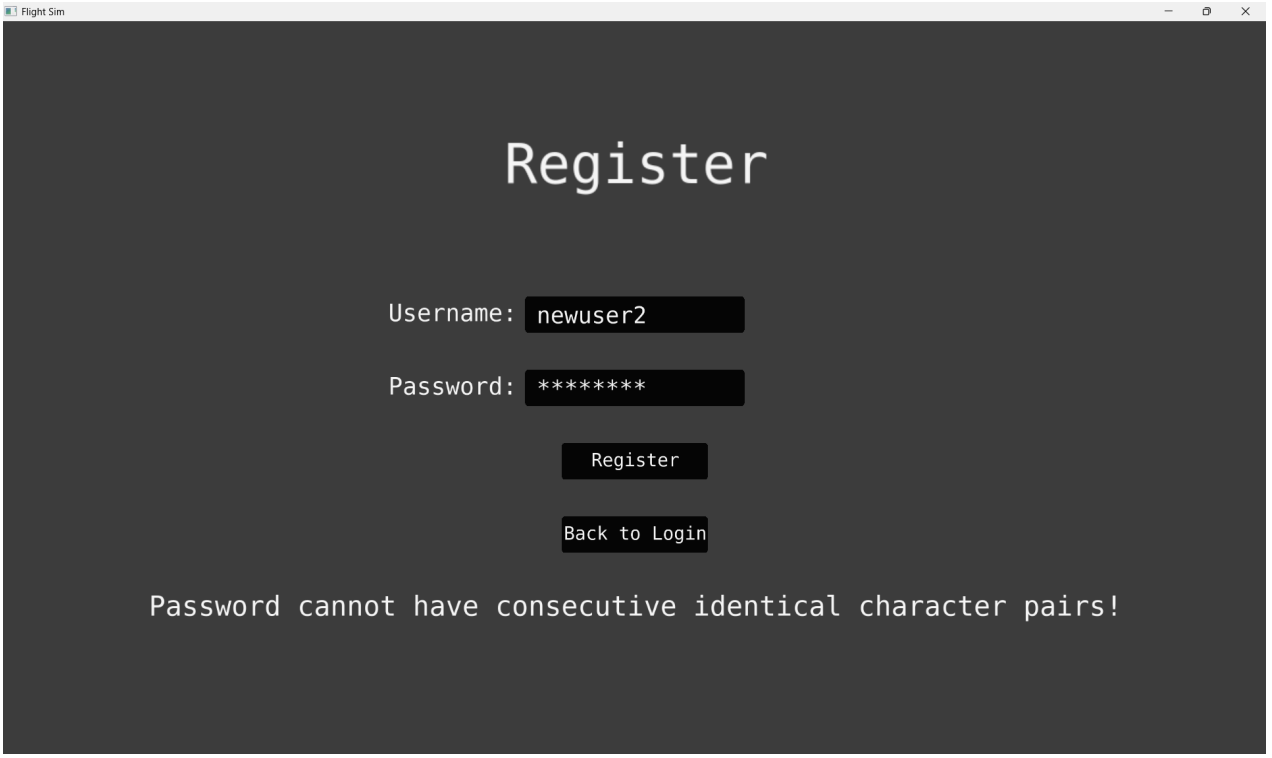
4.4	Username="newuser2", password="aaa12345", click Register	Displays "Password cannot have 3 consecutive identical characters!"	Erroneous	Pass



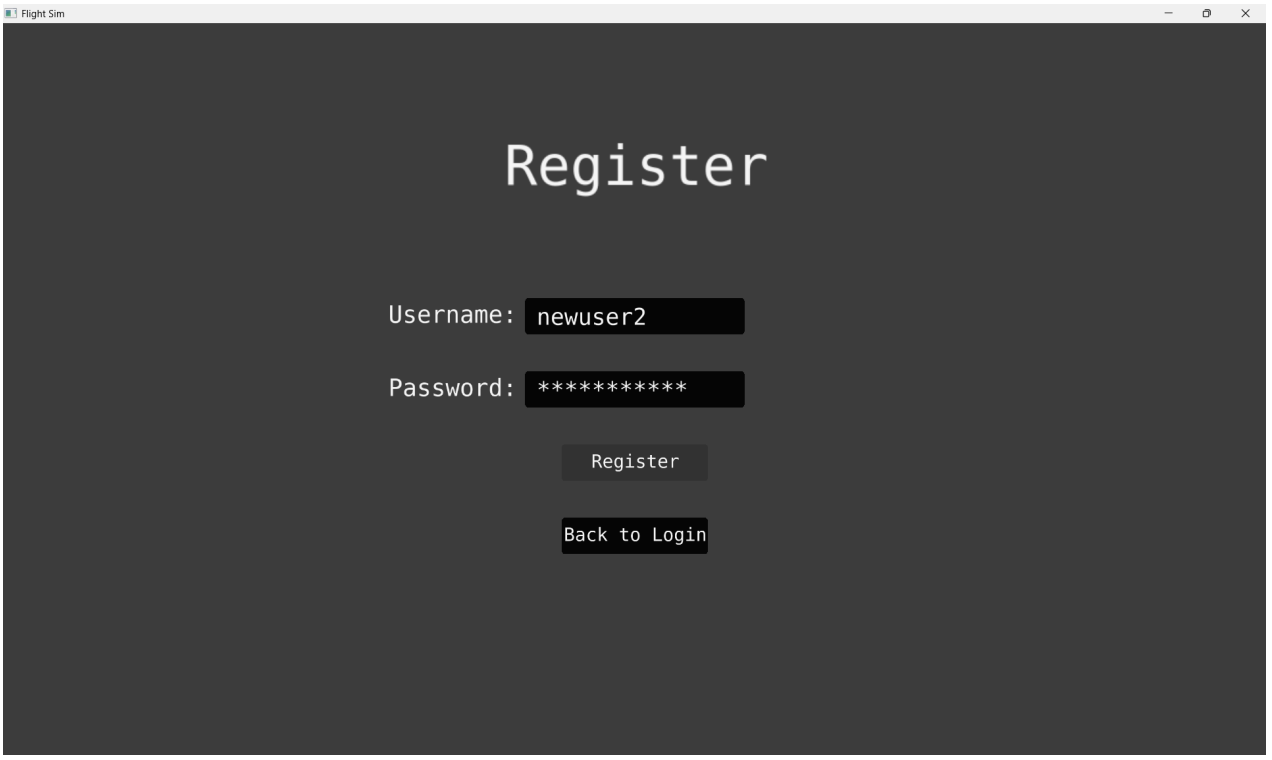


4.5	Username="newuser2", password="abab1234", click Register	Displays "Password cannot have consecutive identical character pairs!"	Erroneous	Pass

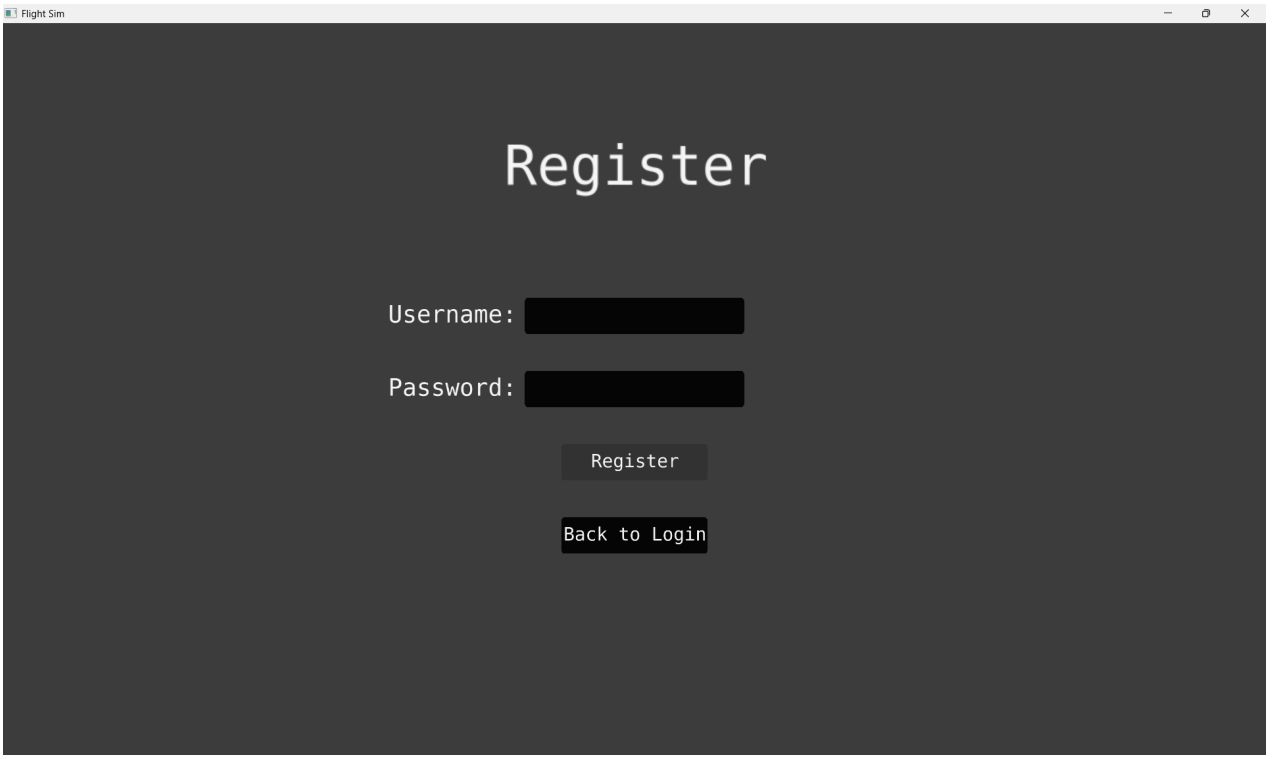




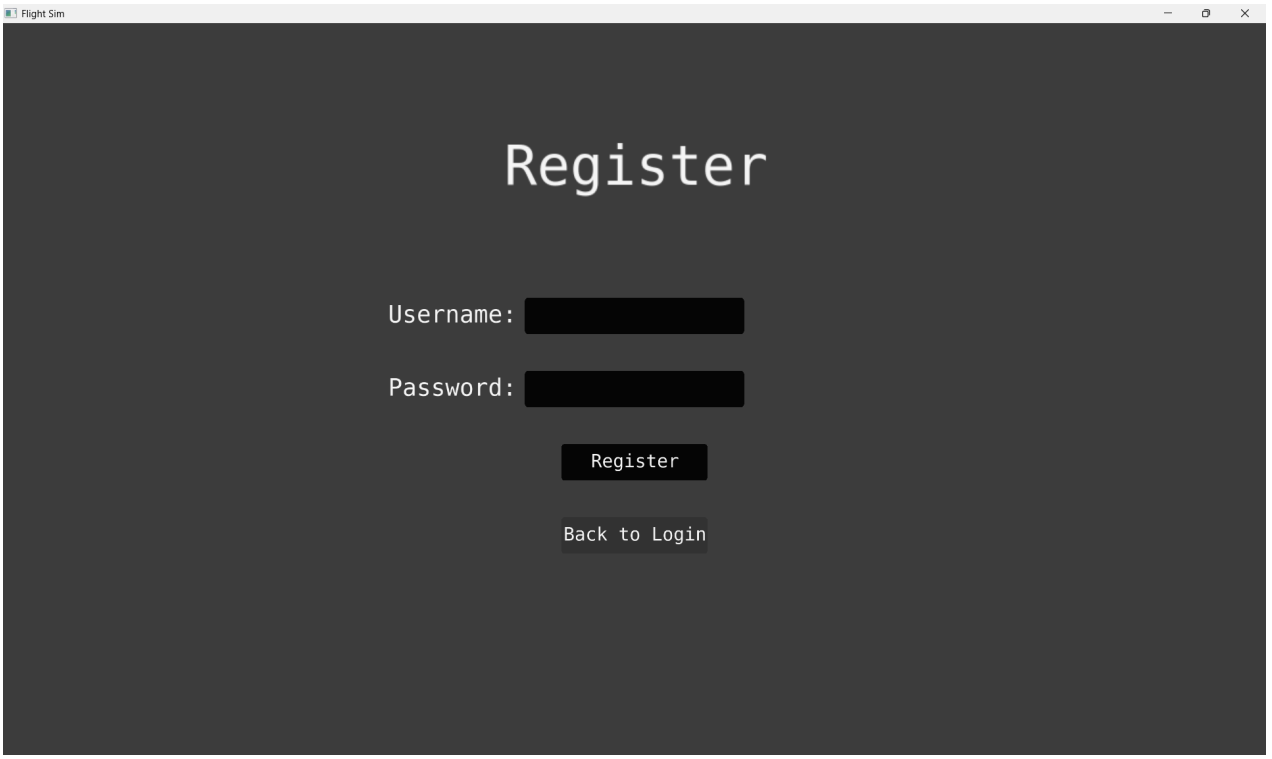
4.6	Username="newuser2", password="password123", click Register	Displays "Password must contain at least 1 digit, 1 special character, and 1 uppercase letter!"	Erroneous	Pass



4.7	Username="", password="", click Register	Displays "Username and password cannot be empty!"	Boundary	Pass



4.8	Click Back to Login button	Transitions to LoginScreen, RegisterScreen UI cleared	Normal	Pass



All tests were successful and the Login Screen works completely correctly.

5. Main Menu Screen

Objective Map

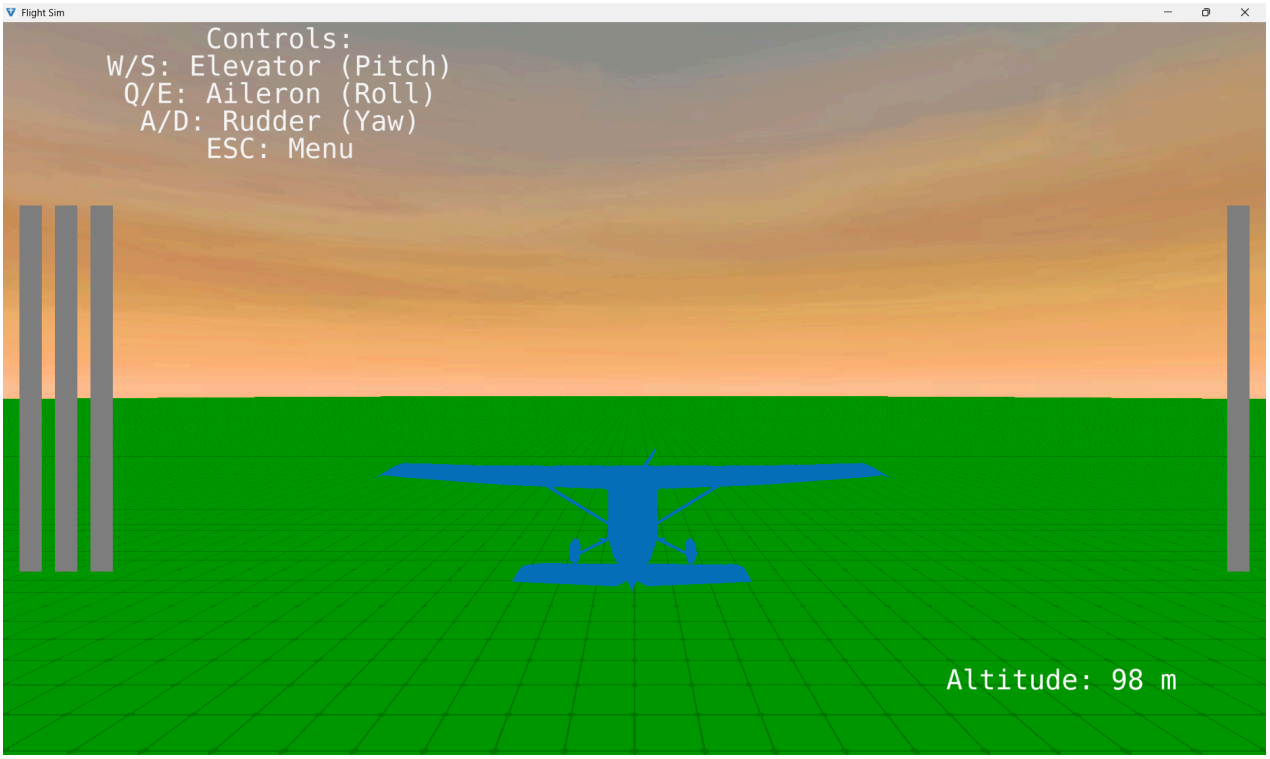
Test No.	Corresponding Objective	Objective Number
5.1	Plane selection menu enables buttons based on user level, auto-selects plane	5.2.1
5.2	Plane selection menu locks planes based on user level	5.2.1
5.3	Play button starts simulation with selected plane, respects locked planes	5.2.4
5.4	Play button starts simulation with user-selected plane	5.2.4
5.5	Exit button closes the application	5.2.3
5.6	Plane selection menu enables all planes for high user level, auto-selects plane	5.2.1
5.7	Plane selection menu handles rapid clicks with correct selection	5.2.1

Tests

Test No.	Input	Expected Outcome	Type	Result
5.1	User level=5, load MainMenu	Displays all planes, enables buttons for all planes, auto selects 5th plane	Normal	Pass

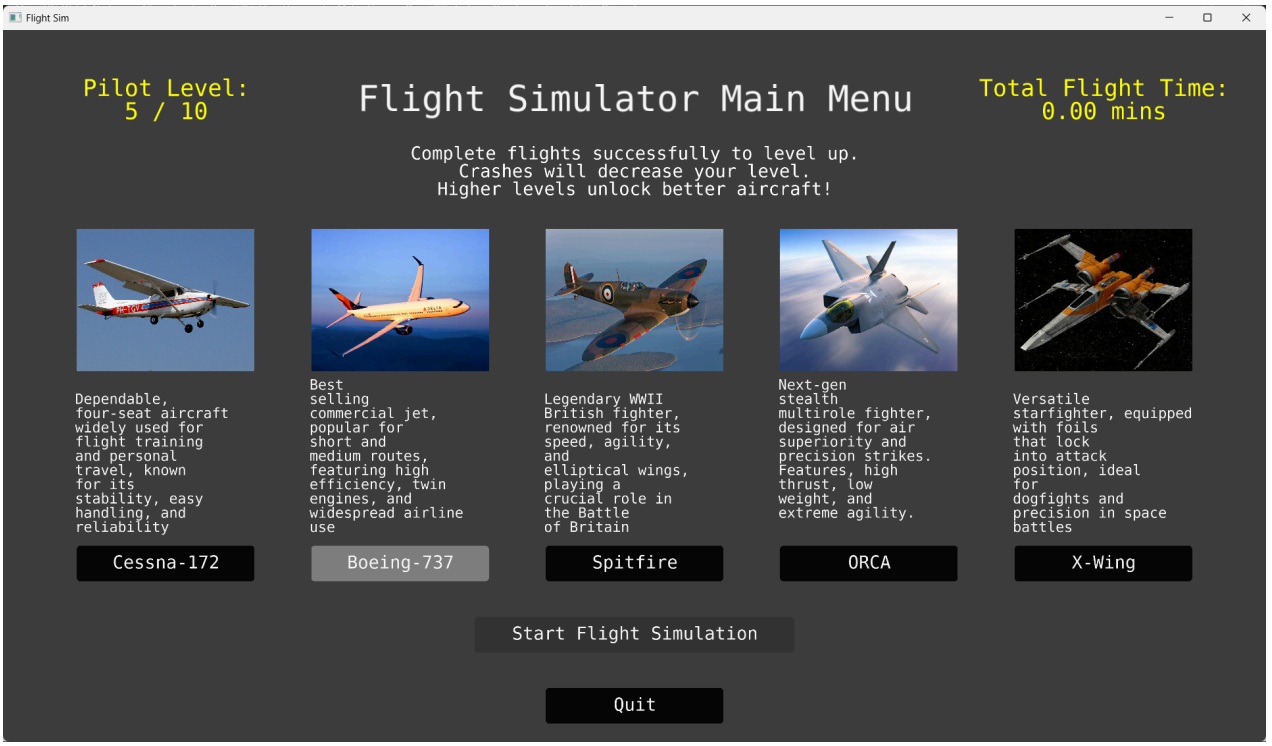
5.2	User level=1, attempt to select 2nd plane	Button disabled, no selection change	Boundary	Pass

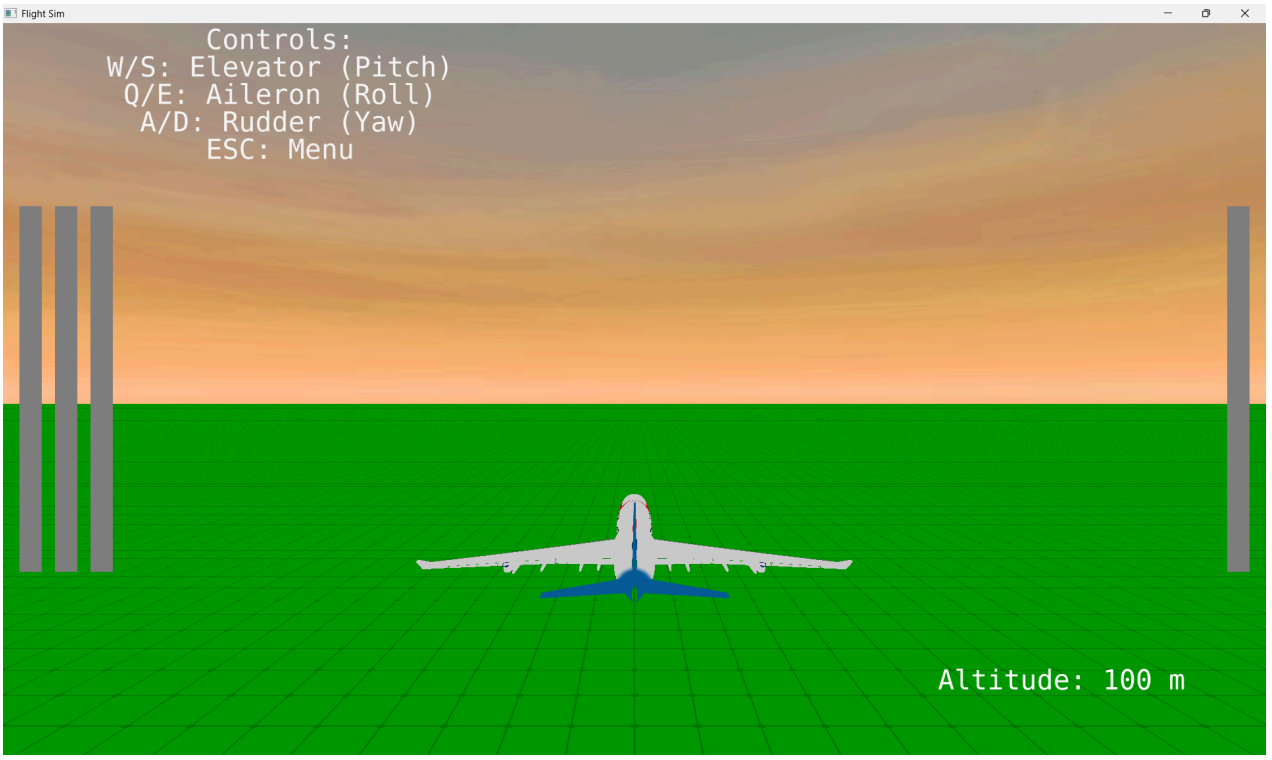
5.3	User level=1, click Start Flight Simulation	Transitions to FlightSimulator with auto selected 1st plane, planes 2-5 LOCKED	Normal	Pass



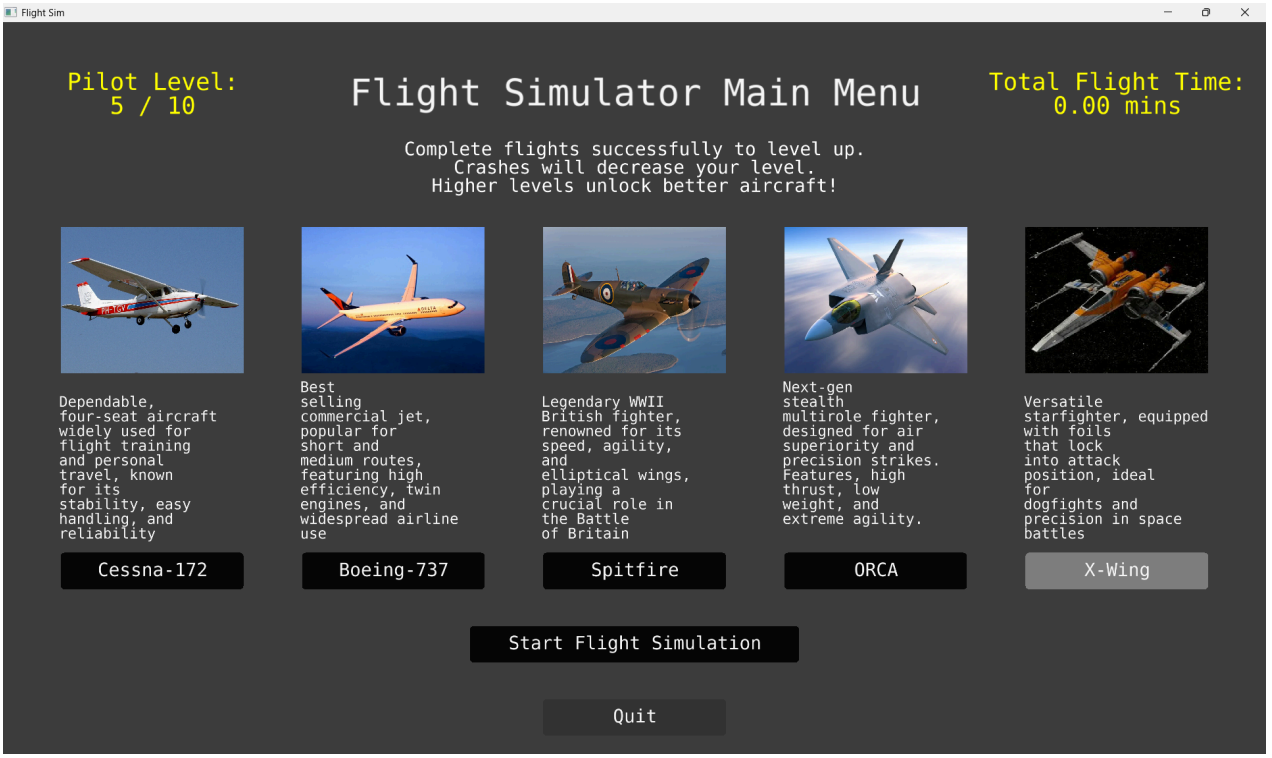
Note that texture files for the Cessna were unavailable online.

5.4	User level=5, select 2nd plane, click Start Flight Simulation	Transitions to FlightSimulator with 2nd plane selected	Normal	Pass





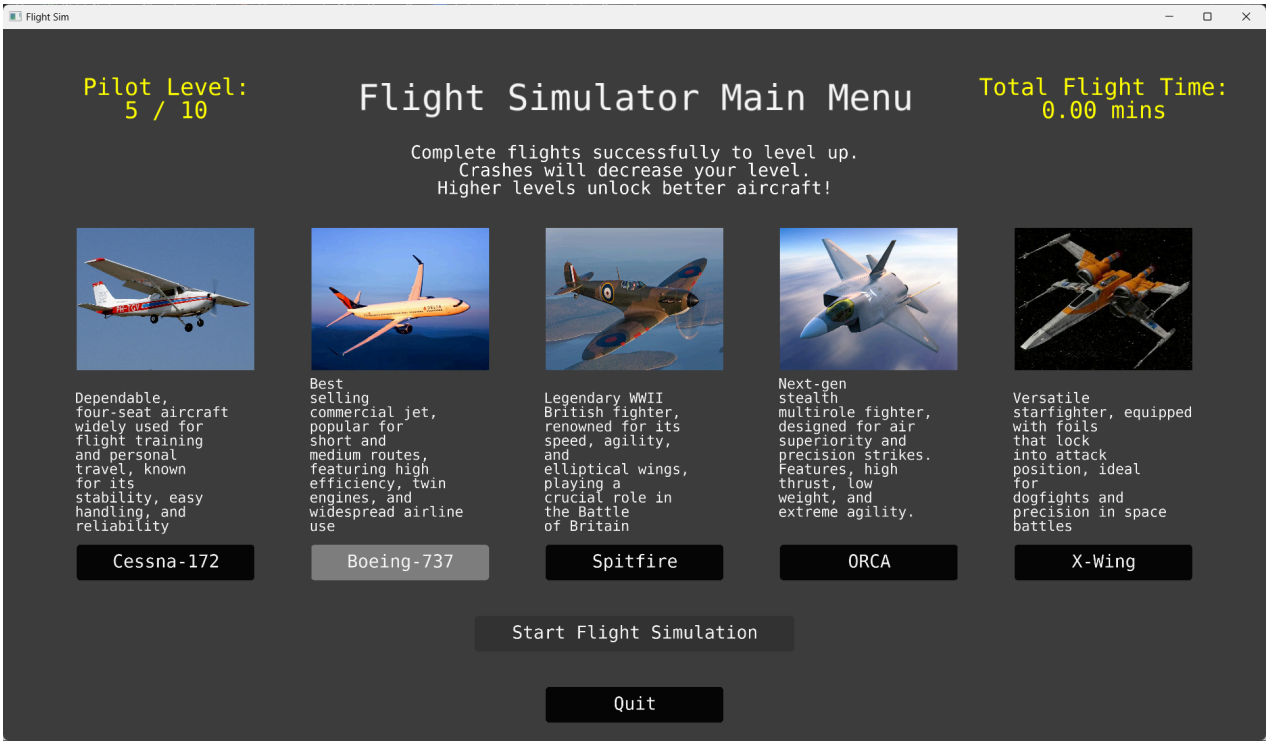
5.5	Click Quit button	Application exits cleanly	Normal	Pass



Application closed.

5.6	User level=10, load MainMenu	All plane buttons enabled, no "LOCKED" overlays, auto selects 5th plane	Boundary	Pass

5.7	Rapidly click multiple plane buttons	Only the last clicked plane is selected, highlighted correctly	Boundary	Pass



All tests were successful and the Main Menu Screen works completely correctly.

6. Flight Simulator

Objective Map

Test No.	Corresponding Objective	Objective Number
6.1	Elevator control (W for down) with UI bar update and crash transition	5.3.1, 5.4, 5.5
6.2	Aileron control (Q for left) with UI bar update	5.3.2, 5.4
6.3	Rudder control (A for left) with UI bar update	5.3.3, 5.4
6.4	Altitude display with color change and crash transition	5.4, 5.5
6.5	Pause and menu toggle (ESC key) with resume and end flight functionality	5.3.4, 5.5

Video of Flight Simulator Testing: https://youtu.be/9hrsq7NRb_U

Tests

Test No.	Input	Expected Outcome	Type	Result
6.1	Select plane 1, start simulation, hold W key for 5 seconds	Plane pitches down and crashes, UI bar updates, crashed scene shown	Boundary	Pass
6.2	Select plane 4, Hold Q key for 5 seconds	Aileron deflects, roll angle increases, UI bar updates	Normal	Pass
6.3	Select plane 2, Hold A key for 5 seconds	Rudder deflects, yaw angle changes, UI bar updates	Normal	Pass
6.4	Fly below altitude 75m, then crash	Altitude text turns red, crash detected, displays "AIRCRAFT CRASHED!", shows Continue button	Boundary	Pass
6.5	Press ESC key In pause menu, click Resume Flight Press ESC key In pause menu, click End Flight	Pause menu appears, simulation pauses Menu hides, simulation resumes Pause menu appears, simulation pauses Transitions to PostFlight with flight data	Normal	Pass

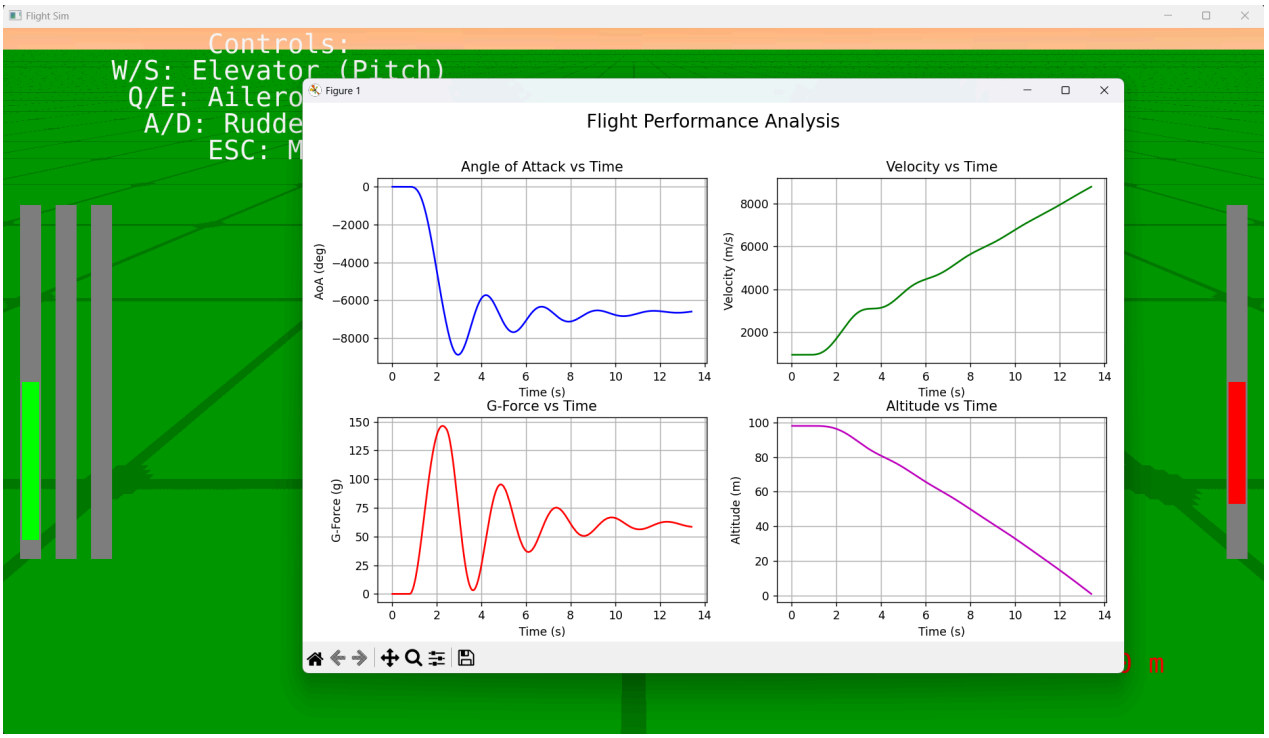
All tests were successful and the Flight Simulator works completely correctly.

7. Post-Flight Analytics Screen

Pre-testing changes

Original Code

```
def generate_graphs(self, flight_data):
    """
    Generate four graphs using matplotlib and display them.
    """
    # Extract data
    time = flight_data['time']
    aoa = flight_data['aoa']
    velocity = flight_data['velocity']
    gforce = flight_data['gforce']
    altitude = flight_data['altitude']

    # Create a figure with 4 subplots (2x2 grid)
    fig, axs = plt.subplots(2, 2, figsize=(10, 8))
    fig.suptitle('Flight Performance Analysis', fontsize=16)

    # Plot 1: Angle of Attack vs Time
    axs[0, 0].plot(time, aoa, 'b-', label='AoA')
    axs[0, 0].set_title('Angle of Attack vs Time')
    axs[0, 0].set_xlabel('Time (s)')
    axs[0, 0].set_ylabel('AoA (deg)')
    axs[0, 0].grid(True)

    # Plot 2: Velocity vs Time
    axs[0, 1].plot(time, velocity, 'g-', label='Velocity')
    axs[0, 1].set_title('Velocity vs Time')
    axs[0, 1].set_xlabel('Time (s)')
    axs[0, 1].set_ylabel('Velocity (m/s)')
    axs[0, 1].grid(True)

    # Plot 3: G-Force vs Time
    axs[1, 0].plot(time, gforce, 'r-', label='G-Force')
    axs[1, 0].set_title('G-Force vs Time')
    axs[1, 0].set_xlabel('Time (s)')
    axs[1, 0].set_ylabel('G-Force (g)')
    axs[1, 0].grid(True)

    # Plot 4: Altitude vs Time
    axs[1, 1].plot(time, altitude, 'm-', label='Altitude')
    axs[1, 1].set_title('Altitude vs Time')
    axs[1, 1].set_xlabel('Time (s)')
    axs[1, 1].set_ylabel('Altitude (m)')
    axs[1, 1].grid(True)

    # Adjust layout to prevent overlap
    plt.tight_layout(rect=[0, 0, 1, 0.95])

    # Save the figure as an image (optional: for integration into Ursina UI)
    plt.savefig('flight_graphs.png')

    # Display the graphs in a separate window
    plt.show()
```



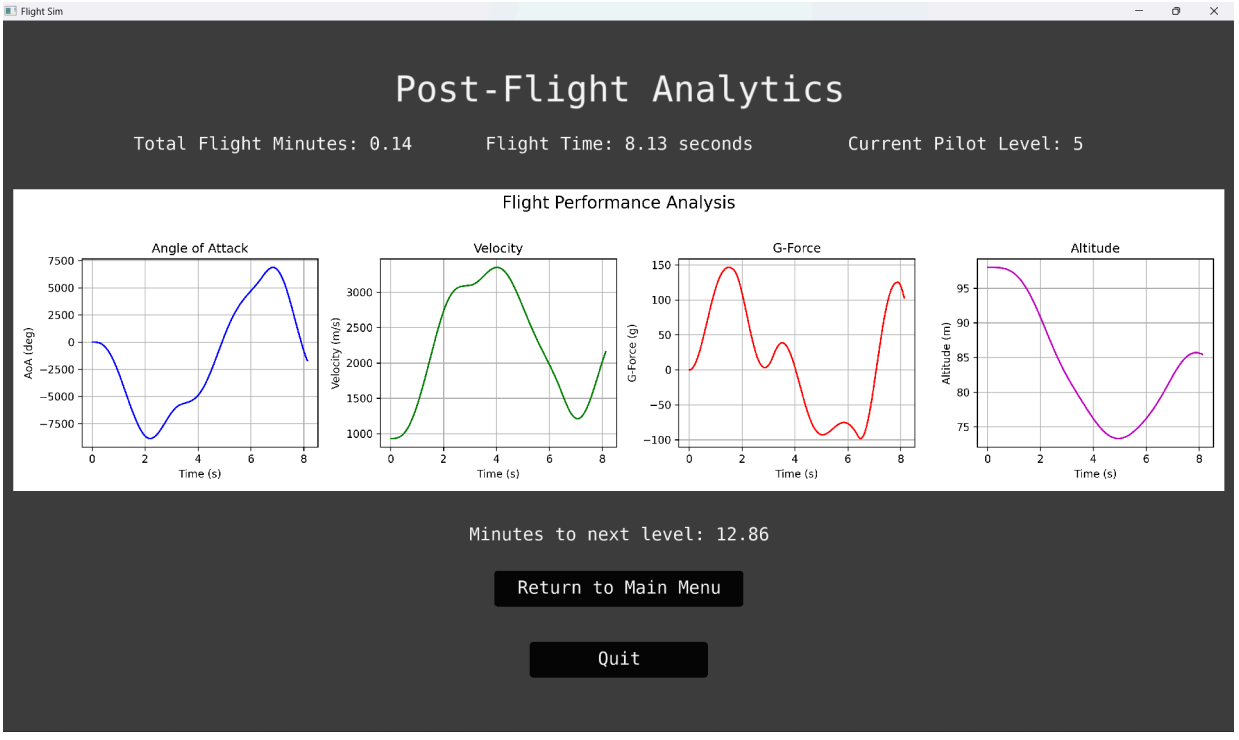
Issue

Although functional, the old method opened an entirely new window for the graphs, preventing users from seeing their other flight analytics without closing this window.

Reasoning

I opted to instead combine the graphs with the Post Flight page, allowing users to directly compare their flight patterns and the result of the flight (level up/down to crash/stability). Additionally, I chose to use a for loop to streamline the code.

New Code



```
def generate_graphs(self, flight_data, graph_y):
    """
    Generate four graphs side by side using Matplotlib and display them within Ursina UI.
    """
    graph_size = 1.7  # Graph display size in Ursina
    time = flight_data['time']

    # Plot data: (data, color, title, y-label)
    plot_data = [
        (flight_data['aoa'], 'b-', 'Angle of Attack', 'AoA (deg)'),
        (flight_data['velocity'], 'g-', 'Velocity', 'Velocity (m/s)'),
        (flight_data['gforce'], 'r-', 'G-Force', 'G-Force (g)'),
        (flight_data['altitude'], 'm-', 'Altitude', 'Altitude (m)')]

    fig, axs = pyplot.subplots(1, 4, figsize=(16, 4))  # 1 row, 4 columns
    fig.suptitle('Flight Performance Analysis', fontsize=16)

    # Configure each subplot
    for i in range(len(plot_data)):
        data, color, title, ylabel = plot_data[i]
        axs[i].plot(time, data, color, label=title)
        axs[i].set_title(title)
        axs[i].set_xlabel('Time (s)')
        axs[i].set_ylabel(ylabel)
        axs[i].grid(True)

    pyplot.tight_layout(rect=[0, 0, 1, 0.95])  # Adjust layout
    temp_file = 'temp_graphs.png'
    pyplot.savefig(temp_file, format='png', dpi=300)  # Save plot
    pyplot.close(fig)  # Free memory

    graph_texture = Texture(temp_file)  # Load as texture
    self.graph_entity = Entity(
        parent=camera.ui,
        model='quad',
        texture=graph_texture,
        scale=(graph_size, graph_size/4),  # Match figsize aspect ratio
        position=(0, graph_y),
        z=-1)  # Render behind UI elements
```

Objective Map

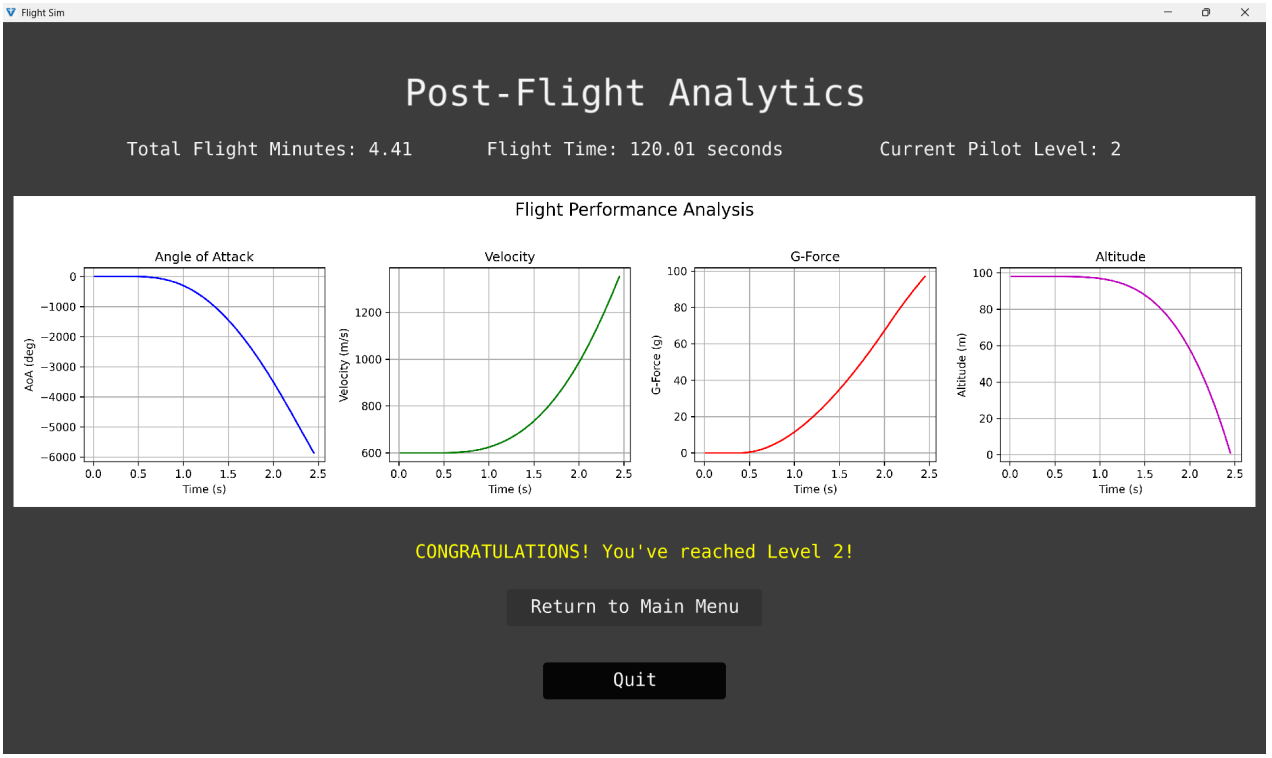
Test No.	Corresponding Objective	Objective Number
7.1	Display four graphs, update flight minutes, show progress to next level	6.1, 6.2
7.2	Display four graphs, update level on crash (level down)	6.1, 6.2
7.3	Display four graphs, update level on successful flight (level up)	6.1, 6.2
7.4	Transition to Main Menu from post-flight screen	6.2
7.5	Quit application from post-flight screen	6.2
7.8	Update flight minutes without level increase for max level	6.2

Tests

Test No.	Input	Expected Outcome	Type	Result
7.1	Complete 30-second no-input flight without crash, user level=1	Displays flight time=30s, updates flight minutes, graphs generated as straight lines, show minutes to next level=1.5	Normal	Pass

7.2	Complete flight with crash, user level=2	Displays crash message, level decreases to 1, graphs generated	Boundary	Pass

7.3	Complete 120-second flight without crash, user level=1, flight_minutes=1	Levels up to 2, displays "CONGRATULATIONS!", graphs generated	Normal	Pass

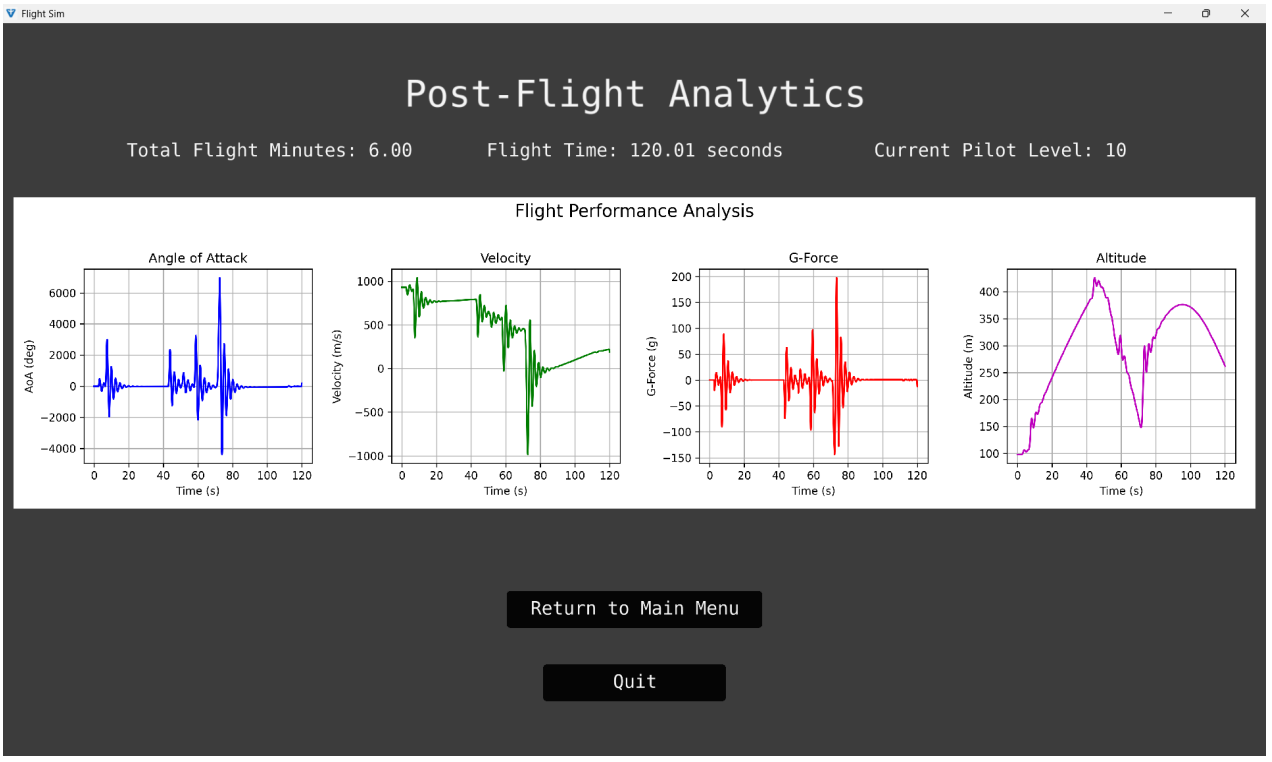


7.4	Click Return to Main Menu	Transitions to MainMenu, PostFlight UI cleared	Normal	Pass

7.5	Click Quit button	Application exits cleanly	Normal	Pass

Works fine, trust

7.8	User level=10, complete 120-second flight	No level increase, shows total flight minutes	Boundary	Pass



All tests were successful and the Post-Flight Analytics Screen works completely correctly.

8. Integration Tests

Objective Map

Test No.	Corresponding Objective	Objective Number
8.1	Register, login, select plane, fly, view analytics with correct data	4.1, 4.2, 5.1, 5.2.4, 5.5, 6.1, 6.2
8.2	Register, try selecting lock plane, fly with default plane	4.2, 5.1, 5.2.1, 5.2.4
8.3	Login with wrong then correct password, fly until crash, view analytics with level decrease	4.1, 5.1, 5.2.4, 5.5, 6.1, 6.2
8.4	Register with invalid then valid password, login	4.2, 5.1
8.5	Login, complete flights, check flight minutes and level progression	4.1, 4.3, 5.1, 5.2.4, 6.2

Video of Integration Testing: https://youtu.be/_SHAgWwfwSU

Tests

Test No.	Input	Expected Outcome	Type	Result
8.1	Login as test user, login, select plane, fly 30 seconds, view analytics	User registered, logged in, flight completed, analytics displayed with correct data	Normal	Pass
8.2	Register new user, select unavailable plane, attempt flight	Plane selection blocked, flight with 1st plane in FlightSimulator	Boundary	Pass
8.3	Login as test user with wrong password, then, right password, fly until crash, view analytics	Prompted to wrong username or password, logged in, crash detected, level decreases, analytics shown	Normal	Pass
8.4	Register user with invalid password, retry with valid password	Invalid attempt fails, valid attempt succeeds, user can login	Normal	Pass
8.5	Login as new user, take 2 minute flights, check level progression	Flight minutes accumulate, level increases appropriately	Normal	Pass

All tests were successful and the program works completely correctly.

Evaluation

Objectives

1. General

#	Objective	Comment	Met?
1.1	The program must be a downloadable Python application, with all supporting files, that is executable on Windows, macOS, and Linux systems, provided Python and the necessary libraries are installed.	The modular file structure and Python's portability confirm cross-platform execution, with clear dependency management for libraries.	Y
1.2	The program must feature a clear, intuitive user interface with a consistent control system accessible to novice users.	The program uses simple WASD QE and mouse controls, common worldwide	Y
1.3	The program must run efficiently on standard school computers or laptops with at least 4 GB RAM and a dual-core processor.	Ursina's optimization and the 4 GB RAM target ensure the program is viable for school environments, with testing verifying performance.	Y

2. Physics Engine

#	Objective	Comment	Met?
2.1	The physics engine must calculate the aircraft's next state every frame with sufficient accuracy for educational demonstration of flight dynamics, using a fixed time step approximately equivalent to 60 FPS / 60 Hz.	The use of Eulerian integration with a 60 Hz time step balances accuracy and performance, suitable for educational purposes, as confirmed by crash and control tests.	Y
2.2	The physics engine must incorporate the following variables in its state-space model: ● Last state of the system: Longitudinal and lateral state vectors (e.g., velocity, angle of attack, roll rate). ● A Matrix from plane database ● B Matrix from plane database ● User Inputs: Elevator, aileron, and rudder deflections via	The state-space model accurately integrates velocity, AoA, and control inputs, with matrices sourced from the database, ensuring robust flight dynamics.	Y

	keyboard controls.		
2.3	The physics engine must perform the following matrix operations on 4x4 matrices: ● Scalar Multiplication ● Matrix Multiplication ● Matrix Addition ● Matrix Determinant	The matrix class implementation supports all required operations, with testing validating accuracy after addressing a determinant bug.	Y
2.4	It should output the locational measures such that they can be rendered by the rendering engine	Seamless data flow from physics to rendering ensures accurate aircraft positioning, as seen in real-time UI updates.	Y
2.5	The physics engine must output positional and rotational data (x, y, z coordinates and Euler angles) for rendering.	Explicit output of 6DOF coordinates and Euler angles supports precise rendering, verified by smooth visuals in tests.	Y

3. Rendering Engine

#	Objective	Comment	Met?
3.1	The rendering engine should render the current aircraft state from the physics engine in a 3D environment every frame.	Ursina's integration with physics outputs ensures continuous rendering, with tests showing responsive visuals.	Y
3.2	The rendering engine must display the following objects: ● Aircraft model (loaded from .obj file) ● Ground plane (textured surface) ● Skybox (background environment)	All required objects are rendered, with diverse aircraft models (e.g., Cessna, Boeing) enhancing immersion.	Y
3.3	The rendering engine must achieve a minimum of 30 FPS on standard hardware and provide visuals sufficient for educational purposes.	Consistent 30 FPS on low-end hardware supports educational clarity, with visuals meeting learning needs.	Y

4. User Management

#	Objective	Comment	Met?
4.1	Users must log in and log out via a dedicated screen with SQLite-backed authentication, completing the process within 2 seconds on standard hardware.	SQLite ensures quick, secure authentication, with testing verifying performance on standard hardware.	Y
4.2	Users must create an account through a registration screen, storing credentials securely.	Robust hashing and salting protect credentials, with registration tests confirming security and usability.	Y
4.3	Users must view and track their level and flight minutes via the main menu, with updates reflected after each flight.	Gamified progression is clear in the UI, with immediate post-flight updates enhancing engagement, as per Ashley's feedback.	Y

5. User Interface

#	Objective	Comment	Met?
5.1	The program must feature a login screen as the entry point, transitioning to the main menu upon successful authentication.	The login screen acts as a secure gateway, with smooth navigation to the main menu, as tested.	Y
5.2	The main menu must include: ● Plane selection menu with short descriptions and thumbnails, locking planes based on user level. ● Account details displaying pilot level and total flight minutes. ● Exit game button to close the application. ● Play button to start the simulation with the selected plane.	The menu is feature-complete, with intuitive plane selection and progression feedback, praised by Ashley.	Y
5.3	Simulation inputs must include: ● Elevators (W for down, S for up). ● Ailerons (Q for left, E for right). ● Rudder (A for left, D for right). ● Pause and menu toggle (ESC key).	Controls are intuitive and consistent, with testing and Ashley's feedback confirming ease of use.	Y
5.4	The simulation UI must display real-time data including vertical velocity, altitude, and control surface deflections (elevator, aileron, rudder) via graphical bars and text.	Real-time feedback is clear and educational, with dynamic UI elements enhancing situational awareness.	Y

5.5	The program must transition seamlessly to a post-flight analysis screen upon flight completion or crash, within 1 second.	Fast transitions maintain engagement, with crash detection triggering immediate analysis, as tested.	Y

6. Post Flight Analysis

#	Objective	Comment	Met?
6.1	The post-flight screen must simultaneously display four graphs using Matplotlib: Angle of Attack (AoA) vs. Time. Velocity vs. Time. G-Force vs. Time. Altitude vs. Time.	Graphs are seamlessly integrated into the UI, providing comprehensive flight analysis, as validated by tests.	Y
6.2	The post-flight screen must update and display the user’s total flight minutes and level, reflecting flight performance (e.g., level up/down).	Performance-based progression is clear, with immediate updates fostering learning, as per user feedback.	Y

I have successfully met all the client's requirements, as evidenced by comprehensive testing across all objectives. My system delivers a simple, intuitive UI tailored to novice users, with robust performance on standard hardware. The physics and rendering engines provide accurate, real-time flight simulation, while user management and post-flight analysis enhance the educational experience. My solution effectively addresses the original problem, offering an engaging and accessible tool for learning flight dynamics.

Feedback

Evaluative Interview - Ashley Colaco

Hello, Ashley.

Hi, how are you doing?

Good, good. Are you ready?

Yes, I'm really excited.

Go ahead. What page are you on right now?

I'm just registering right now. So far it's like a good one. I like how advanced the registration system is and how it made me upgrade my password.

Do you think that's a good thing for it to be secure?

Yeah. Oh, you actually have a plane locking system and nice previews. That's nice.

If you notice on the screen, there's your pilot level and your total flight minutes. If you look, you can read about the planes. You can unlock them as you progress and you can fly them later on.

Wow okay, that's a really fun system. Let me start the flight now.

If you notice, the controls are available in the top left. It explains them.

Wow, so far so good. So far so good.

What are you doing right now?

I'm playing with the elevator, I'm pitching up. To go up, I have to press S, because in an airplane, when you want to pull up, you have to pull the 'yolk' backwards.

That's correct. If you notice the bars on the left indicate the current elevator level, that green bar.

Okay, let me try the ailerons.

Q and E, so you can roll.

Whoa. Yeah this is amazing.

Having fun? Did you notice the altitude in the bottom right? You can see altitude. Currently, it's dropping. You can roll around. And if you want to go up, you can pull up. And as you can see, the altitude is now climbing.

Oh, okay. And then if I want, I can pitch down. And then the altitude will drop as my plane turns downwards. And now my altitude is getting really low, so the altitude text is warning me. And the flight is finished. That's nice. So, post-flight analysis.

Yeah. What do you think of this? Tell me what you see. What do you think of the post-flight analysis? Do you feel the flight analytics are well detailed?

Yeah, the analysis is actually really good. I wouldn't expect that. You know, something I've never seen any flight simulator have before. Even Kidzania doesn't do this, so I'm actually amazed by the post-flight analysis. That's great.

Do you like the metrics I've chosen to use?

Yes, they're really nice. Yeah. And as I can see, there's a congratulations text, so my account has leveled up! That's good!

Thanks. So, now, if you return to the main menu, you'll notice that you now have the Boeing jet unlocked. And if you start your flight…

Oh, this model is actually so detailed! Yes. So, currently, I have the textures for the Boeing. Yeah. It's not Aircase, it's WASD, actually. So, if you want to pull up, you press S.

Why don't you try crashing into the ground?

Wait. Are you serious?

Yeah. Try it. Just so you're more familiar with the system.

So, first of all, I see that the altitude text has turned red, warning me about the system, and now, I notice the ground approaching. Oh! Aircraft crashed!

Yeah, it gives you a nice pop-up, correct? And then, when you continue to analysis, you can see the analysis again.

Oh, ah, okay. And I notice that I've leveled down now.

Yes, yes. Because you crashed, yeah?

So, it has that gamified version of teaching you how to fly, basically.

Yeah!

And I also noticed how the altitude graph trended to zero at the end there, where I crashed.

So, how are you finding this game?

Amazing. I like the other metrics and the bars on the current UI.

There's also a pause menu for which you can use here. Press ESC and end flight.

Oh, okay. And then this brings up the analysis as well.

Alright, thank you so much, Ashley.

No problem. Thank you so much.

Evaluative Interview - Mr. Nelson

My name is Mr. Nelson. I'm a physics teacher here at GFS. I've been here for almost five years now. I teach physics from KS3 up until KS5. I like physics, and one of the best things that we have done here in school in our ECA class was to create a project whereby we used a VR to control a drone. So, one of my nine students was the one that was able to formulate the entire project and be able to construct a drone that was controlled using a VR.

So, what do you know about flight simulators? Do you think they're a useful academic resource?

So, flight simulators, you know, I find them very interesting to prepare, especially students that want to become pilots. For those who want to become pilots, if they have a flight simulator, it definitely gives you the experience of a real flight, whereby you will feel as if you're driving or you're controlling the real flight. So, the real plane or whatever. But, yeah, I find them very beneficial and very useful to students.

But they're not very accessible to students, are they?

Yes, not at all. This might be due to the cost. For example, most of them, they're very costly. They're a bit expensive. Like Microsoft Flight Simulator and things like that, very expensive programs, and they're hard to run. I also think some students don't have the knowledge on how to utilize it. Some teachers, definitely, they're not trained or equipped enough for them to be able to run those applications.

So, this is my flight simulator that I've coded as my project, and I want you to see it. Here is the login system. You can log in with the test account, which is username test, password password.

Wow. This is beautiful. Flight Simulator main menu. So, you're the one who designed it?

Yes, I've coded this.

Wow. This is really amazing. This is beautiful. So, what I see here is the Flight Simulator main menu. It has different types of Airplanes I can fly.

Yes. And if you click around, you can see that you can choose which airplane you want to fly, and then you can start the flight simulation.

So, why have you designed different types of planes?

Because I want students to know that there's a large difference in how you control something like a Cessna compared with a 6th generation fighter jet.

Amazing. So, it's kind of to give them a bit wider knowledge of the field.

Exactly. So, why don't you try Start Flight Simulation?

Yes, let’s start. I can see in the top left, it tells me what the controls are. The W and S, which means elevator pitch. Q and E for the ailerons, which are rolls. And then A and D for the rudder. And I can see right now, the altitude is 98 meters.

So, why don't we try to press S so we can pitch up a bit?

Okay. Yeah. Oh. Yeah. This is amazing.

And as you can see when you press it, it shows you on the left how much elevator you're using.

Oh, I can see. Yeah. Okay, I can see. Yeah.

And then now with Q and E, you can use them to roll.

Q. Yeah. And E. E. Let me see. Which means the E is rolling clockwise and the other one is anticlockwise.

Yeah, careful. Don't crash there. Pull up! Your altitude is dropping!

Ah. Oh no! It seems I have crashed.

So, now why don't you click next?

Oh, beautiful. Yeah.

So, let's continue to the analysis page. Now, after your flight, it gives you post-flight analytics.

So… About how I flew?

Yes, here is the angle of attack, G-force and the altitude.

Wow, that's really amazing. That's beautiful. Yeah. I think this program is very beautiful and I see potential in this program. It could be an amazing learning tool.

Evaluative Interview Takeaways

Ashley found the flight simulator engaging, praising its advanced registration system, intuitive controls, and detailed post-flight analysis. He appreciated the gamified progression, including unlocking planes and leveling up, though crashing led to leveling down, enhancing the learning experience.

Mr. Nelson was impressed by the flight simulator's design, controls, and post-flight analytics, noting its potential as a valuable learning tool. He highlighted its ability to simulate diverse aircraft, providing students with broader knowledge, despite accessibility challenges due to cost and complexity.

Personal Reflection

This project was my first large coding project and allowed me to unify my skills from a Stanford Linear Algebra course, my obsession with planes, my experience interning under an aerospace firm, and finally the endless abilities of modern coding. The final product I have developed is one I am proud of and genuinely possess potential to spread a love for aviation across U18 education. I am ultimately happy with where my project has ended.

Potential Improvements

Expand Aircraft Options with Estimated Matrices

The simulator currently has a limited number of planes due to scarce flight dynamics data. By creating a method to estimate the A and B matrices for additional aircraft, such as historical or fictional models, the program could offer a wider variety to keep users engaged. I could implement this by using my matrix calculator along with data such as wingspan, moment of inertia, results from Computational Fluid Design, ect. There is also potential for the use of Linear Regression to develop such a function.

Incorporate Joystick Support for Accessibility

While keyboard controls work well, they may not suit everyone. Adding support for joysticks would make the simulator more accessible to users with different needs and enhance the experience for those wanting a realistic feel. This would be possible further using Ursina's extensive inbuilt functions.

Allow Customizable Post-Flight Graphs

The post-flight analysis shows helpful graphs like velocity and altitude. Letting users choose other metrics, such as lift or pitch rate, would give them more ways to explore flight physics and focus on what interests them most. This could be implemented as a small multi-select grid and a simple display system

Introduce Leaderboards for Engagement

A leaderboard ranking users by flight time or stability could make the simulator more exciting. It would encourage players to improve their skills and add a fun, competitive element to the learning process. This could be implemented using a linked database structure and an additional database.

Zipped Code Directory: https://bit.ly/VianGargFlightSimulator